Crackz Documentation

AI Image Analysis

Version: 0.70.1

Contents

Home

Crackz

	`crackz` `crackz-gui`


```
my-project/
├── images/          # Source images by split (raw/train/val/test)
├── masks/           # Ground truth for training
├── artifacts/       # Detection results and training metrics
├── logs/            # Complete audit trail
└── project.yaml     # Configuration snapshot
```

```
# Create project
crackz init --name demo

# Import images
crackz import-images --project demo --src ./photos --type raw --size 512 512

# Run detection
crackz detect run --project demo

# View results in demo/artifacts/detections/
```

```
crackz-gui
# File → New Project → Import Images → Run Detection
```

Getting Started

Getting Started with Crackz

```
# Install UV package manager
curl -LsSf https://astral.sh/uv/install.sh | sh  # macOS/Linux
# OR
irm https://astral.sh/uv/install.ps1 | iex       # Windows PowerShell

# Authenticate with GitHub
gh auth login

# Download and install Crackz
gh release download --repo anaxiatech/crackz --pattern '*.whl'
uv pip install crackz-*.whl

# Verify installation
crackz --version
```

```
# Pull the Docker image
docker pull ghcr.io/anaxiatech-ab/crackz:latest

# Verify it works
docker run --rm ghcr.io/anaxiatech-ab/crackz:latest --version
```

```
# Create a new project
crackz init --name my-first-project

# This creates a structured directory:
# my-first-project/
```

```
#   ├── images/      # Your photos organized by type
#   ├── masks/       # Ground truth (for training)
#   ├── artifacts/   # Results and metrics
#   └── logs/        # Audit trail
```

```
crackz-gui
# Then: File → New Project (Ctrl/Cmd+N)
```

```
# Import images for crack detection
crackz import-images \
  --project my-first-project \
  --src ./your-photos \
  --type raw \
  --size 512 512
```

```
# Run detection using the pre-trained model
crackz detect run --project my-first-project
```

`my-first-project/artifacts/detections/` `my-first-project/artifacts/detections/metrics.json`

```
# 1. Create project
crackz init --name harbor-inspection

# 2. Get pre-trained model checkpoint
#    (Contact your Crackz provider or use a partner model)
#    Place it in: harbor-inspection/artifacts/training/checkpoints/best.ckpt

# 3. Import images
crackz import-images --project harbor-inspection --src ./photos --type raw --size 512 512

# 4. Run detection (uses the pre-trained model automatically)
crackz detect run --project harbor-inspection
```

```
# 1. Create project
crackz init --name custom-harbor

# 2. Import labeled training data
crackz import-images \
  --project custom-harbor \
  --src ./labeled/images \
  --src-masks ./labeled/masks \
  --type train \
  --size 512 512

# 3. Import validation data
crackz import-images \
  --project custom-harbor \
  --src ./labeled/val_imgs \
  --src-masks ./labeled/val_masks \
  --type val \
  --size 512 512

# 4. Train your custom model
crackz detect train --project custom-harbor --epochs 20

# 5. Run detection with your trained model
crackz detect run --project custom-harbor
```

```
my-first-project/
├── images/
│   ├── raw/         # Original images for detection
│   ├── train/       # Training images (if training a model)
│   ├── val/         # Validation images
│   └── test/        # Test images
├── masks/
│   ├── train/       # Ground truth masks for training
│   ├── val/         # Validation masks
│   └── test/        # Test masks
├── artifacts/
│   ├── training/    # Model checkpoints and training metrics
│   └── detections/  # Detection results (images + JSON)
├── logs/            # Project-specific logs
└── project.yaml     # Configuration file
```

`my-first-project/my-first-project_project.yaml`

```
ai:
  model:
    threshold: 0.5  # Increase to 0.7 for fewer false positives
                    # Decrease to 0.3 to catch more cracks
```

```
# Process several inspection campaigns
for project in site-A site-B site-C; do
    crackz detect run --project $project
done
```

```
# Run detection in container
docker run --rm \
  -v $(pwd)/my-first-project:/home/app \
  ghcr.io/anaxiatech-ab/crackz:latest \
  detect run --project /home/app
```

```
# Verify CUDA installation
nvidia-smi

# Check PyTorch CUDA support
python -c "import torch; print(torch.cuda.is_available())"

# Reinstall with CUDA support (see Installation Guide)
```

`images/raw/` `.jpg` `.jpeg` `.png` `.bmp` `.tif` `.tiff`

`project.yaml`

```
# Reduce batch size
crackz detect train --project my-project --batch-size 1

# Or reduce image size during import
crackz import-images --project my-project --src ./data --type train --size 256 256
```

`crackz init --name my-first-project`

Installation

Installation

`uv sync`

`LICENSE`

-
-
-
-

`gh auth login`

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

```
irm https://astral.sh/uv/install.ps1 | iex
```

```
# Login to GitHub
gh auth login

# Download the latest wheel
gh release download --repo anaxiatech/crackz --pattern '*.whl'

# Install with uv
uv pip install crackz-*.whl

# Verify installation
crackz --version
```

`crackz-VERSION-py3-none-any.whl`

```
uv pip install crackz-VERSION-py3-none-any.whl
```

`crackz` `crackz-gui`

```
# Using pip with GitHub authentication
pip install git+https://github.com/OWNER/crackz.git@vVERSION

# Or download the wheel from the private repository's Releases page
# (Requires GitHub authentication - use personal access token)
pip install https://USERNAME:TOKEN@github.com/OWNER/crackz/releases/download/vVERSION/crackz-VERSION-py3-none-any.whl
```

```
pip install git+https://github.com/OWNER/crackz.git
```

```
# Login with GitHub CLI
gh auth login

# Download release asset
gh release download vVERSION --repo OWNER/crackz --pattern '*.whl'

# Install the downloaded wheel
pip install crackz-VERSION-py3-none-any.whl
```

```
# Create a personal access token with 'repo' scope at:
# https://github.com/settings/tokens

# Set your token (keep it secure!)
export GITHUB_TOKEN="ghp_xxxxxxxxxxxx" # gitleaks:allow

# Install using token authentication
pip install git+https://${GITHUB_TOKEN}@github.com/OWNER/crackz.git@vVERSION
```

`https://github.com/OWNER/crackz`
`crackz-VERSION-py3-none-any.whl`

```
pip install crackz-VERSION-py3-none-any.whl
```

```
# Download wheel using GitHub token
curl -L \
  -H "Authorization: token ${GITHUB_TOKEN}" \
  -o crackz.whl \
  https://github.com/OWNER/crackz/releases/download/vVERSION/crackz-VERSION-py3-none-any.whl

# Install the downloaded wheel
pip install crackz.whl
```

```
# If your organization has configured a private index
pip install --index-url https://pypi.company.com/simple/ crackz
```

```
# Using the provided index URL and credentials
pip install crackz --index-url https://INDEX_URL/simple/

# Example with authentication (if required):
pip install crackz --index-url https://USERNAME:TOKEN@pypi.company.com/simple/
```

`~/.config/pip/pip.conf` `%APPDATA%\pip\pip.ini`

```
[global]
index-url = https://USERNAME:TOKEN@pypi.company.com/simple/
```

```
pip install crackz
pip install --upgrade crackz  # For updates
```

```
# Set credentials via environment variables
export PIP_INDEX_URL="https://pypi.company.com/simple/"
export PIP_EXTRA_INDEX_URL="https://pypi.org/simple"  # Fallback to PyPI for dependencies

pip install crackz
```

`https://dl.cloudsmith.io/TOKEN/org/repo/python/simple/`
`https://artifactory.company.com/artifactory/api/pypi/pypi-local/simple/`
`https://pypi.fury.io/TOKEN/USERNAME/`
`https://aws:TOKEN@domain.d.codeartifact.region.amazonaws.com/pypi/repo/simple/`
`https://pkgs.dev.azure.com/org/_packaging/feed/pypi/simple/` `https://devpi.company.com/username/index/+simple/`

```
chmod 600 ~/.config/pip/pip.conf  # Linux/macOS
```

```
pip install --trusted-host pypi.company.com crackz
# Or add CA cert:
pip install --cert /path/to/ca-bundle.crt crackz
```

```
pip install crackz --extra-index-url https://pypi.org/simple/
```

`crackz.exe`
`crackz-gui.exe` `crackz` `crackz-gui` `crackz` `crackz-gui`

```
REM Download the executables from GitHub Releases
REM Move them to a permanent location
mkdir C:\Program Files\Crackz
move crackz.exe "C:\Program Files\Crackz\"
move crackz-gui.exe "C:\Program Files\Crackz\"

REM Add to PATH (optional, for CLI access from anywhere)
setx PATH "%PATH%;C:\Program Files\Crackz"
```

```
# Download the executables from GitHub Releases
chmod +x crackz crackz-gui

# Move to a system directory (optional)
sudo mv crackz /usr/local/bin/
sudo mv crackz-gui /usr/local/bin/

# Or keep in a local directory and add to PATH
mkdir -p ~/Applications/crackz
mv crackz crackz-gui ~/Applications/crackz/
echo 'export PATH="$HOME/Applications/crackz:$PATH"' >> ~/.bashrc
source ~/.bashrc
```

```
# Run CLI directly (no Python needed)
crackz --help
crackz init --name myproject

# Run GUI by double-clicking crackz-gui.exe (Windows)
# Or run from command line:
crackz-gui
```

`crackz-cuda.exe` `crackz-gui-cuda.exe`

```
crackz --help
python -c "import crackz; import importlib.metadata as m; print(m.version('crackz'))"
```

- `crackz-VERSION-py3-none-any.whl`
- `crackz_cu118-VERSION-py3-none-any.whl`
- `crackz_cu128-VERSION-py3-none-any.whl`

```
pip install https://github.com/Anaxiatech-AB/crackz/releases/download/vVERSION/crackz-VERSION-py3-none-any.whl
```

```
# Step 1: Install PyTorch with CUDA 11.8
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu118

# Step 2: Install crackz CUDA variant
pip install https://github.com/Anaxiatech-AB/crackz/releases/download/vVERSION/crackz_cu118-VERSION-py3-none-any.whl
```

```
# Step 1: Install PyTorch with CUDA 12.8
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu128

# Step 2: Install crackz CUDA variant
pip install https://github.com/Anaxiatech-AB/crackz/releases/download/vVERSION/crackz_cu128-VERSION-py3-none-any.whl
```

```
nvidia-smi
```

```
nvcc --version
# Or check driver
nvidia-smi | grep "CUDA Version"
```

`crackz` `crackz_cu118` `crackz_cu128`

```
# Login first
gh auth login

# Download and install CPU version (one step)
gh release download vVERSION --repo OWNER/crackz --pattern 'crackz-*.whl'
pip install crackz-VERSION-py3-none-any.whl
```

```
# Login first
gh auth login
```

```
# Step 1: Install PyTorch with CUDA 12.8
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu128

# Step 2: Download and install crackz CUDA variant
gh release download vVERSION --repo OWNER/crackz --pattern 'crackz_cu128-*.whl'
pip install crackz_cu128-VERSION-py3-none-any.whl
```

```
# Check CUDA availability
python -c "import torch; print(f'CUDA available: {torch.cuda.is_available()}')"

# Check PyTorch version (should include +cu118 or +cu128 suffix)
python -c "import torch; print(f'PyTorch version: {torch.__version__}')"

# Check GPU device name
python -c "import torch; print(f'Device: {torch.cuda.get_device_name(0) if torch.cuda.is_available() else \"CPU\"}')"
```

```
CUDA available: True
PyTorch version: 2.5.0+cu128
Device: NVIDIA GeForce RTX 4090
```

```
# Step 1: Install specific PyTorch version
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121

# Step 2: Install crackz CUDA variant (cu118 or cu128, either works)
pip install https://github.com/Anaxiatech-AB/crackz/releases/download/vVERSION/crackz_cu118-VERSION-py3-none-any.whl
```

`cpu` `cu128`

```
# This installs CPU PyTorch even with the cu128 extra - BROKEN!
pip install "crackz-VERSION-py3-none-any.whl[cu128]"
```

```
# Step 1: Install PyTorch with CUDA
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu128

# Step 2: Install crackz
pip install crackz_cu128-VERSION-py3-none-any.whl
```

`keyring`

```
# With uv
uv add keyring

# With pip
pip install keyring
```

```
from crackz_gui.state.secure_storage import (
    store_api_key,
    retrieve_api_key,
    is_secure_storage_available,
)

# Check availability
if is_secure_storage_available():
    store_api_key("your-api-key")
    api_key = retrieve_api_key()
```

1.
```
mkdir crackz_wheels
cd crackz_wheels

# Download Crackz wheel
wget https://github.com/Anaxiatech-AB/crackz/releases/download/vVERSION/crackz-VERSION-py3-none-any.whl

# Download all dependencies
pip download -r <(pip show -f crackz | grep Requires) -d .
# Or download from the wheel itself:
pip download crackz-VERSION-py3-none-any.whl -d .
```
2.
3.
```
pip install --no-index --find-links crackz_wheels crackz-VERSION-py3-none-any.whl
```

```
pip install .
```

`pyproject.toml`

```
# Download the new version from GitHub Releases
pip install --upgrade https://github.com/Anaxiatech-AB/crackz/releases/download/vNEW_VERSION/crackz-NEW_VERSION-py3-none-any.whl

# Or if using a private index:
pip install -U --index-url https://pypi.company.com/simple/ crackz
```

```
pip uninstall crackz
```

```
crackz --version
crackz init --name sanity-test
crackz detect run --project sanity-test  # (Will use default config; may download model weights)
```


`torch.__version__`

`No module named crackz`

`CUDA not available` `torch.__version__`

`ModuleNotFoundError: No module named 'torch'`

`CRACKZ_LOG_DIR`

`pip install --force-reinstall crackz-VERSION.whl`

`pyproject.toml`

```
# Check if CUDA is available
python -c "import torch; print(f'CUDA available: {torch.cuda.is_available()}')"
python -c "import torch; print(f'CUDA device: {torch.cuda.get_device_name(0)}')"
```

```
CUDA available: True
CUDA device: NVIDIA GeForce RTX 3080
```

```
# These commands automatically use GPU if available:
crackz detect train --project my_project
crackz detect run --project my_project
crackz-gui  # GUI will use GPU automatically
```

`cuda` `mps`
`cpu`

```
# Force GPU usage (will error if CUDA not available)
crackz detect train --project my_project --device cuda

# Force CPU usage (useful for debugging)
crackz detect train --project my_project --device cpu

# Let Crackz auto-detect (default)
crackz detect train --project my_project
```

`<project>/<project>_project.yaml`

```
ai:
  model:
    device: cuda  # or cpu, or mps
```

```
crackz detect train --project my_project
```

```
GPU available: True, used: True
TPU available: False, using: 0 TPU cores
IPU available: False, using: 0 IPUs
HPU available: False, using: 0 HPUs
```

```
# NVIDIA GPUs
watch -n 1 nvidia-smi

# Should show Python process using GPU memory and compute
```

`crackz_cu118-VERSION-py3-none-any.whl`

`crackz_cu128-VERSION-py3-none-any.whl`

`crackz_cu118-VERSION-py3-none-any.whl`

```
nvcc --version  # If CUDA toolkit installed
nvidia-smi  # Shows driver version (different from CUDA toolkit)
```

1.
   ```
   python -c "import torch; print(torch.__version__)"
   # Should show: 2.5.0+cu118 or 2.5.0+cu128
   # If it shows just 2.5.0, you have CPU-only version!
   ```
2.
   ```
   pip uninstall torch torchvision
   pip install torch torchvision --index-url https://download.pytorch.org/whl/cu118
   ```
3.
   ```
   python -c "import torch; print(f'CUDA available: {torch.cuda.is_available()}')"
   ```

```
# GUI: Project → Settings → Training → Batch Size
# CLI:
crackz detect train --project my_project --batch-size 2
```

- `nvidia-smi`
-
-

```
CRACKZ_LOG_DIR=my_logs crackz detect run --project myproj
```

`$Env:CRACKZ_LOG_DIR='my_logs'`

- `pip install --require-hashes -r requirements.txt`

```
crackz migrate --project ./my_project         # applies changes + backup
crackz migrate --project ./my_project --dry-run  # preview only
```

`CRACKZ_AUTO_MIGRATE=1`

User Guide

User Guide

-
-
-
-
-
-

`crackz detect train`

`crackz detect run`

```
graph LR
    A[Create Project] --> B[Import Images]
    B --> C{Need Custom Model?}
    C -->|Yes| D[Train Model]
    C -->|No| E[Run Detection]
    D --> E
    E --> F[View Results]
    F --> G{Satisfied?}
    G -->|No| H[Adjust Config]
    H --> D
    G -->|Yes| I[Export/Package]
```

-
-
-

-
-

`nvidia-smi`

```
# Install from GitHub Release (replace VERSION with actual version)
pip install https://github.com/Anaxiatech-AB/crackz/releases/download/vVERSION/crackz-VERSION-py3-none-any.whl

# Or download first, then install
wget https://github.com/Anaxiatech-AB/crackz/releases/download/v0.30.2/crackz-0.30.2-py3-none-any.whl
pip install crackz-0.30.2-py3-none-any.whl

# Verify installation
crackz --help
```

```
# Install PyTorch with CUDA support first
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121

# Then install Crackz from GitHub Release
pip install https://github.com/Anaxiatech-AB/crackz/releases/download/vVERSION/crackz-VERSION-py3-none-any.whl
```

```
# CPU version
docker pull ghcr.io/anaxiatech-ab/crackz:latest

# GPU version (requires nvidia-docker)
docker pull ghcr.io/anaxiatech-ab/crackz:gpu

# Slim version (minimal runtime)
docker pull ghcr.io/anaxiatech-ab/crackz:slim
```

```
crackz --version
python -c "import crackz; print('Crackz installed successfully')"
```

```
crackz init --name my-first-project
cd my-first-project
```

```
my-first-project/
├── images/          # Image storage by split
│   ├── raw/         # Unprocessed images for inference
│   ├── train/       # Training images
│   ├── val/         # Validation images
│   └── test/        # Test images
├── masks/           # Ground truth masks (optional for raw)
│   ├── train/
│   ├── val/
│   └── test/
├── artifacts/       # Model outputs (metrics, plots)
├── detections/      # Detection results (annotated images, JSON)
├── models/          # Saved model weights (or checkpoints/ in older projects)
├── logs/            # Project-specific logs
└── project.yaml     # Project configuration
```

`checkpoints/` `models/` `crackz migrate --project <project> --migrate-files`

```
# Import sample images for detection (no training)
crackz import-images \
  --project my-first-project \
  --src ./samples \
  --type raw \
  --size 512 512
```

`images/raw/`

```
# Use pre-trained model
crackz detect run --project my-first-project
```

`detections/` `artifacts/metrics.json`

```
# Import training images with masks
crackz import-images \
  --project my-first-project \
  --src ./training-data/images \
  --src-masks ./training-data/masks \
  --type train \
  --size 512 512

# Quick training (1 epoch for testing)
crackz detect train --project my-first-project --epochs 1 --batch-size 2
```

```
# Check detection outputs
ls my-first-project/detections/

# View metrics
cat my-first-project/artifacts/metrics.json

# Open annotated images
open my-first-project/detections/*.png  # macOS
xdg-open my-first-project/detections/*.png  # Linux
```

```
# Launch Streamlit dashboard for advanced analysis
crackz dashboard --project my-first-project
```

```
# Open project in GUI and launch dashboard
crackz-gui
# Then: Tools → Launch Dashboard
```

`images/` `raw/`
`train/val/test/` **`masks/`**
`artifacts/`
`detections/` **`models/`** `checkpoints/`
`logs/` **`project.yaml`**

`metricsjson` `loss_curve.png`
`confusion_matrix.png`

`detections/` `metrics.json`

`CRACKZ_LOG_DIR`

`<project>/logs/`

```
# Set central log directory
export CRACKZ_LOG_DIR=~/crackz-logs

# Both central and project logs will be written
crackz detect run --project my-project
```

`--debug` `--quiet`

```
schema_version: "1.0"
project:
  name: my-project
  # ... rest of config
```

```
crackz migrate --project old-project
```

`num_classes: 1`

`num_classes: 3`

`efficientnet-b3`

`efficientnet-b0`				
`resnet34`				
`efficientnet-b7`				
`resnet50`				
`resnet101`				
`mobilenet_v2`				
`resnet18`				
`densenet121`				

`<project>/<project>_project.yaml`

```
ai:
 model:
   encoder: resnet34           # Choose encoder backbone
   encoder_weights: imagenet   # Use transfer learning (recommended)
   in_channels: 3              # RGB images (use 1 for grayscale)
   num_classes: 1              # Binary segmentation
   threshold: 0.5              # Detection sensitivity (0.3=more sensitive, 0.7=less)

 training:
   epochs: 20                  # Number of training iterations
   batch_size: 8               # Images per batch (reduce if out of memory)
   learning_rate: 0.001        # Optimizer learning rate
```


```
crackz model import --project ./my_project --model pretrained_model.zip
```

```
crackz model export --project ./my_project --out my_model.zip
```

`project.yaml` `ai.model.encoder` `crackz detect train --project <name>`

```
crackz init --name harbor-inspection-2025 --description "Q1 2025 harbor crack survey"
```

`crackz-gui`

```
# Create with custom settings
crackz init --name custom-project --force  # Overwrite if exists

# Initialize from template (future)
crackz init --name new-project --template harbor-config.yaml
```

```
crackz import-images \
  --project harbor-inspection-2025 \
  --src ~/Pictures/harbor-photos \
  --type raw \
  --size 512 512
```

```
crackz import-images \
  --project harbor-inspection-2025 \
  --src ~/labeled-data/images \
  --src-masks ~/labeled-data/masks \
  --type train \
  --size 512 512
```

```
# Training data
crackz import-images --project my-project --src train_imgs --src-masks train_masks --type train --size 512 512

# Validation data
crackz import-images --project my-project --src val_imgs --src-masks val_masks --type val --size 512 512

# Test data
crackz import-images --project my-project --src test_imgs --src-masks test_masks --type test --size 512 512
```

`.jpg` `.jpeg` `.png` `.bmp` `.tif` `.tiff` `.png`

```
crackz-gui
```

`images/train/`

`B`

`E`

`masks/train/`

-

-
-
-
-
- `B` `E` `Ctrl+Z`

`B` `E` `Ctrl+Z`


```
crackz detect train --project my-project
```

```
crackz detect train \
  --project my-project \
  --epochs 20 \
  --batch-size 8 \
  --learning-rate 0.001
```

`project.yaml`

```
# Automatically uses GPU if available
crackz detect train --project my-project --epochs 50

# Verify GPU is being used (check logs)
tail -f my-project/logs/crackz.log | grep -i cuda
```

```
# Watch loss in real-time
tail -f my-project/artifacts/metrics.json

# Or use TensorBoard (if enabled)
tensorboard --logdir my-project/artifacts/tensorboard
```

`project.yaml`

```
crackz detect run --project my-project
```

```
# Uses GPU automatically if available
crackz detect run --project my-project

# Force CPU (even with GPU available)
CUDA_VISIBLE_DEVICES="" crackz detect run --project my-project
```

`--model`

```
# Auto-select (default): best.ckpt > last.ckpt > final.ckpt
crackz detect run --project my-project

# Explicitly use best checkpoint
crackz detect run --project my-project --model best

# Use most recent checkpoint (useful during iterative development)
crackz detect run --project my-project --model last

# Use final checkpoint
crackz detect run --project my-project --model final

# Use specific epoch checkpoint (e.g., epoch-0.9234.ckpt)
crackz detect run --project my-project --model epoch-0.9234
```

`--model` **`--model best`**
`--model last` **`--model final`**
`--model epoch-X`

```
# Run detection with different checkpoints to compare results
crackz detect run --project my-project --model best
mv my-project/detections my-project/detections-best

crackz detect run --project my-project --model last
mv my-project/detections my-project/detections-last

crackz detect run --project my-project --model epoch-0.8532
mv my-project/detections my-project/detections-epoch-0.8532

# Compare detection outputs side by side
```

```
# Detect on raw images (default)
crackz detect run --project my-project

# Detect on test set (with metrics if masks available)
crackz detect run --project my-project --split test
```

```
# Process multiple projects
for project in project1 project2 project3; do
    crackz detect run --project $project
done
```

```
# CPU
docker run --rm \
  -v $(pwd)/my-project:/home/app/my-project \
  ghcr.io/anaxiatech-ab/crackz-cli:latest \
  detect run --project /home/app/my-project

# GPU
docker run --gpus all --rm \
  -v $(pwd)/my-project:/home/app/my-project \
  ghcr.io/anaxiatech-ab/crackz-cli:gpu \
  detect run --project /home/app/my-project
```

`project.yaml`

```
training:
  tensorboard: true
```

```
# tensorboard-config.yaml
training:
  tensorboard: true
```

```
# After training with tensorboard enabled
tensorboard --logdir my-project/artifacts/tensorboard

# Open browser to http://localhost:6006
```

```
# Quick summary
cat my-project/artifacts/metrics.json | python -m json.tool

# Extract specific metrics
cat my-project/artifacts/metrics.json | jq '.final_val_iou'
```

```
crackz clean --project my-project --dry-run
```

```
crackz clean --project my-project
```

```
crackz clean --project my-project --logs-only
```

`*.log`

`project.yaml`

```
training:
  checkpoint:
    save_top_k: 3  # Keep top 3 checkpoints by validation loss
```

```
# Remove all but the last checkpoint
rm my-project/checkpoints/*.ckpt
# (except last-*.ckpt)

# Or keep only best checkpoint
rm my-project/checkpoints/epoch*.ckpt
# (keep best-*.ckpt)
```

```
# Train on 50-100 manually labeled images
crackz detect train --project harbor-project --epochs 10
```

```
# Prioritize top 50 images from raw/unlabeled pool
uv run crackz active-learning run \
  --project harbor-project \
  --split raw \
  --top-n 50 \
  --method entropy \
  --out priorities.json
```

`entropy` `max_probability` `variance` `mean_confidence`

```
# Use GUI mask editor or external tools
# Focus on top-ranked images from priorities.json
```

```
# Move annotated images to train split
crackz import-images --project harbor-project \
  --src annotated_batch \
  --src-masks annotated_masks \
  --type train

# Retrain with expanded dataset
crackz detect train --project harbor-project --epochs 15
```

`wall_section_01.jpg`

```
# Preview similarity scores
uv run crackz propagate-masks find-similar \
  --project harbor-project \
  --reference wall_section_01.jpg \
  --split raw \
  --out similarities.json
```

```
# Copy mask to similar images (>80% similarity)
uv run crackz propagate-masks run \
  --project harbor-project \
  --reference wall_section_01.jpg \
  --mask wall_section_01_mask.png \
  --split raw \
  --threshold 0.8 \
  --out propagated_masks/
```

```
# Import propagated masks
crackz import-images --project harbor-project \
  --src images_with_propagated \
  --src-masks propagated_masks \
  --type train

# Review in GUI mask editor - correct errors before training
```

```
# More permissive (more propagations, less reliable)
--threshold 0.6 --min-matches 5

# Stricter (fewer propagations, higher quality)
--threshold 0.9 --min-matches 20 --sift
```

```
crackz detect train --project harbor-project
```

```
uv run crackz active-learning run \
  --project harbor-project \
  --top-n 20 \
  --method entropy \
  --out uncertain.json
```

```
# For each reference image
uv run crackz propagate-masks run \
  --reference ref1.jpg --mask ref1_mask.png \
  --project harbor-project --threshold 0.8

# Propagate remaining references...
```

```
# Import all propagated masks
crackz import-images --project harbor-project \
  --src propagated_images \
  --src-masks propagated_masks \
  --type train

# Retrain with 50 original + ~40 propagated = 90 annotations
crackz detect train --project harbor-project --epochs 15
```

```
crackz export-config \
  --project my-project \
  --out my-project-snapshot.yaml
```

```
# Create archive (excluding large files)
tar -czf my-project.tar.gz \
  --exclude='checkpoints' \
  --exclude='detections' \
  my-project/

# Send to collaborator
# They extract and run:
tar -xzf my-project.tar.gz
crackz detect run --project my-project  # Downloads default checkpoint if missing
```

`latest`			
`slim`			
`gpu`			

```
# Full image
docker build -t crackz-cli .

# Slim image
docker build -f Dockerfile.slim -t crackz-cli:slim .

# GPU image
docker build -f Dockerfile.gpu -t crackz-cli:gpu .
```

```
# CPU
docker run --rm \
  -v $(pwd)/my-project:/home/app/my-project \
  crackz-cli detect run --project /home/app

# GPU
docker run --gpus all --rm \
  -v $(pwd)/my-project:/home/app/my-project \
  crackz-cli:gpu detect run --project /home/app
```

```
docker run -it --rm \
  -v $(pwd)/my-project:/home/app/my-project \
  crackz-cli bash
```

`Makefile`

```
# Build documentation
make docs

# Run tests
make test

# Build Docker image
make docker-build

# Clean artifacts
make clean
```

```
# Add to $PROFILE
function Invoke-Crackz {
    docker run --rm -v ${pwd}:/home/app ghcr.io/anaxiatech-ab/crackz-cli:latest @args
}
Set-Alias crackz Invoke-Crackz

# Usage
crackz detect run --project /home/app/my-project
```

```
# process-all.ps1
$projects = Get-ChildItem -Directory -Filter "*-project"
foreach ($project in $projects) {
    Write-Host "Processing $($project.Name)..."
    crackz detect run --project $project.FullName
}
```

```
crackz-gui
```

```
python -m crackz_gui.qt.app
```

`project.yaml`

`images/raw`

`detections/`

```
# Install from GitHub Release
pip install https://github.com/Anaxiatech-AB/crackz/releases/download/vVERSION/crackz-VERSION-py3-none-any.whl
crackz init --name local-project
```

```
docker pull ghcr.io/anaxiatech-ab/crackz-cli:latest
docker run --rm -v $(pwd)/project:/home/app crackz-cli detect run --project /home/app
```

```
# Create Batch pool
./scripts/azure/azure_batch.sh \
  --batch-account myaccount \
  --storage-account mystorage
```

```
# Submit job
./scripts/azure/submit_batch_job.sh \
  --project my-project \
  --pool-id crackz-pool
```

Solution:

```
# Update pip and setuptools
pip install --upgrade pip setuptools

# Download and install from GitHub Release
wget https://github.com/Anaxiatech-AB/crackz/releases/download/vVERSION/crackz-VERSION-py3-none-any.whl
pip install crackz-VERSION-py3-none-any.whl
```

Solution:

```
# Verify CUDA installation
nvidia-smi

# Check PyTorch CUDA support
python -c "import torch; print(torch.cuda.is_available())"

# Reinstall PyTorch with CUDA
pip uninstall torch torchvision
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121
```

`No images found in training split`

Solution: `images/train/` `images/raw/` `.jpg` `.jpeg` `.png` `.bmp` `.tif` `.tiff` `--type train`

Solution: `masks/train/`

Solution:

```
# Reduce batch size
crackz detect train --project my-project --batch-size 1

# Or reduce image size
crackz import-images --project my-project --src ./data --type train --size 256 256

# Monitor GPU memory
nvidia-smi -l 1  # Update every second
```

Solution: `tail my-project/logs/crackz.log` `raw/` `ls my-project/models/` `my-project/checkpoints/` `crackz detect run --project my-project --debug`

Solution: project.yaml

```
detection:
  threshold: 0.5  # Pixel-level binarization threshold
                  # Increase to reduce false positives (0.7)
                  # Decrease to catch more cracks (0.3)

  # Severity categorization thresholds (crack area %)
  min_confidence: 0.005     # Min to save (0.5% crack area)
  medium_confidence: 0.02   # "Probable Crack" threshold (2%)
  high_confidence: 0.05     # "Crack Detected" threshold (5%)
  min_crack_area_px: 100    # Filter out detections < 100 pixels
```

Solution:

```
# Check permissions
ls -la $(python -c "import tempfile; print(tempfile.gettempdir())")/crackz_logs

# Set explicit log directory
export CRACKZ_LOG_DIR=~/crackz-logs
mkdir -p ~/crackz-logs
crackz detect run --project my-project
```

Solution:

```
# Clean old logs
rm -rf $CRACKZ_LOG_DIR/*.log

# Or set retention policy (future feature)
# See configuration.md for log rotation settings
```

Missing --project or --cfg		--project <name> --cfg <file>
CUDA out of memory		--batch-size
Schema version mismatch		crackz migrate --project <name>
Permission denied writing logs		CRACKZ_LOG_DIR
No checkpoint found		crackz detect train --project <name>

$CRACKZ_LOG_DIR /tmp/crackz_logs
<project>/logs/

```
crackz detect train --project my-project --epochs 50 --batch-size 8
```

`project.yaml`

```
crackz detect run --cfg my-custom-config.yaml
```

```
crackz export-config --project my-project --out snapshot.yaml
crackz detect run --cfg snapshot.yaml
```

`.jpg` `.jpeg` `.png` `.bmp` `.tif` `.tiff`

`segmentation-models-pytorch`
`--checkpoint <path>`

`project.yaml` `model.encoder`

`--batch-size` `--size 256 256`
`512 512` `mobilenet_v2` `resnet50`

`&`

```
for p in project1 project2 project3; do
    crackz detect run --project $p &
done
wait
```

`CRACKZ_LOG_DIR`

`.ckpt`

`project.yaml`

`raw` `train` `val` `test`

1.
2.
3.

1.
2.
3.

1.
2.
3.

1.
2.
3.

-
-
-

CLI Cookbook

CLI Cookbook

`crackz init --name demo-project`

```
demo-project/
  images/
    raw/ train/ val/ test/
  masks/
  artifacts/
  detections/
  checkpoints/
  project.yaml
```

- `crackz --help` `--help`
- `--project` `--cfg`
-
- `--debug` `--quiet`
-

```
crackz init --name demo-project
```

```
crackz init --name demo-project --force
```

```
# Preview what will be cleaned (safe)
crackz clean --project demo-project --dry-run

# Clean logs and artifacts
crackz clean --project demo-project

# Clean only logs, keep checkpoints and detections
crackz clean --project demo-project --logs-only
```

`*.log` `--logs-only`
`--logs-only` `--logs-only`

`--type`

```
crackz import-images \
  --project demo-project \
  --src ./samples \
  --type raw \
  --size 512 512 \
  --quality 90
```

```
crackz import-images --project demo-project --src ./samples --type train --size 512 512
crackz import-images --project demo-project --src ./samples --type val   --size 512 512
crackz import-images --project demo-project --src ./samples --type test  --size 512 512
```

```
crackz import-images \
  --project demo-project \
  --src ./samples \
  --split \
  --train-ratio 0.7 \
  --val-ratio 0.15 \
  --test-ratio 0.15 \
  --size 512 512
```

`--split`

`--split` `--type`

`--random-seed`

```
crackz import-images --project demo-project --src ./samples --split --random-seed 42
```

`--src/masks`

```
crackz import-images --project demo-project --src ./samples --masks_src ./samples/masks --type train
```

```
crackz import-images --project demo-project --src ./samples --type train --overwrite
```

`images/train` `val` `test` `raw/` `--type`
`train` `train/` `crackz detect train` `--split`

`scripts/import_masks.py`

```
# Basic usage - import from organized mask directories
python scripts/import_masks.py <project_dir> <masks_source_dir>

# Example: Import masks for skytteren_projekt from smasks directory
python scripts/import_masks.py ./skytteren_projekt ./smasks

# Custom size and quality
python scripts/import_masks.py ./my_project ./mask_sources --size 1024 1024 --quality 95
```

`masks_source_dir`

```
masks_source_dir/
  train/
    mask_001.jpg
    mask_002.png
    ...
  val/
    mask_101.jpg
    ...
  test/
    mask_201.png
    ...
```

`data.mask_threshold`
`_mask`

`--size WIDTH HEIGHT` `--quality QUALITY`

`data.mask_threshold`
`val/`

```
# Basic usage - edit an image (mask path auto-derived)
crackz masks edit images/train/crack_001.jpg

# With custom mask path
```

```
crackz masks edit photo.jpg --mask output/my_mask.png

# With project context (smart path derivation)
crackz masks edit images/raw/inspection.jpg --project my_harbor_project
```

`--mask`

1. `--project`

   ```
   Image:  project/images/train/crack_001.jpg
   Mask:   project/masks/train/crack_001.png  ← Auto-derived
   ```

2.

   ```
   Image:  data/images/train/photo.jpg
   Mask:   data/masks/train/photo.png  ← Auto-derived
   ```

3.

   ```
   Image:  /tmp/photo.jpg
   Mask:   /tmp/masks/photo.png  ← Auto-derived
   ```

`pyside6`

```
for img in images/train/*.jpg; do
    crackz masks edit "$img" --project my_project
done
```

```
ssh -X user@server
crackz masks edit /data/harbor/crack.jpg
```

```
# Convert all masks in a project (uses project threshold)
crackz masks convert --project my_harbor_project

# Convert with custom threshold
crackz masks convert --project my_harbor_project --threshold 128
```

-
- `train` `val` `test` `raw`
- `raw/`

1. `train` `val` `test` `image_01.jpg` `image_01.png`
2. `raw/`
3. `crackz detect train`
4. `crackz detect run --project <p>` `raw/`
5. `test`

		`--type train`

`train` `val` `test`

1. `labelme` `labelme` `pip install labelme`
2.
3. `image_001.png` `images/train/image_001.jpg` `masks/train/image_001.png`
4. `masks/<split>/` `masks/train/` `masks/val/` `masks/test/`
5.
6.
7. `crackz import-images` `--masks_src` `masks/` `--split`

`cv2.threshold`

```
crackz detect run --project demo-project
```

`--project`

```
crackz detect run --cfg config/detect.yaml
```

`best.ckpt` `last.ckpt` `final.ckpt` `--model`

```
# Use specific checkpoint by name
crackz detect run --project demo-project --model best
crackz detect run --project demo-project --model last
crackz detect run --project demo-project --model final

# Use specific epoch checkpoint (e.g., epoch-0.9234.ckpt)
crackz detect run --project demo-project --model epoch-0.9234
```

`--model` `best.ckpt` `last.ckpt`
`final.ckpt`

`--model best`
`--model last` **`--model epoch-X`**

```
crackz detect train --project demo-project
```

`train/val/test`

```
crackz detect train --project demo-project --epochs 1 --batch-size 1
```

```
# Auto-selects best checkpoint, outputs to <project-name>_model.zip
crackz model export --project demo-project

# Specify output path
crackz model export --project demo-project --out models/harbor_v1.zip

# Export specific checkpoint (best, last, or final)
crackz model export --project demo-project --checkpoint best --out best_model.zip
```

```
crackz model export -p demo-project -o my_model.zip
crackz model export -p demo-project -c final -o final_model.zip
```

`.ckpt`

`best.ckpt` `last.ckpt` `final.ckpt`
`<project_name>_model.zip` `--out`

```
# Import model package
crackz model import --project demo-project --model harbor_v1.zip

# Overwrite existing checkpoint with same name
crackz model import --project demo-project --model harbor_v1.zip --force
```

```
crackz model import -p demo-project -m my_model.zip
crackz model import -p demo-project -m my_model.zip --force
```

`checkpoints/`

`--force`

```
# Team member A: Export trained model
crackz model export -p training-project -o shared/harbor_crack_v1.zip

# Team member B: Import into their project
crackz model import -p deployment-project -m shared/harbor_crack_v1.zip
```

```
# Export base model trained on general cracks
crackz model export -p general-cracks -o pretrained/base_model.zip

# Import into specialized project for fine-tuning
crackz model import -p harbor-specific -m pretrained/base_model.zip

# Fine-tune on harbor-specific data
crackz detect train -p harbor-specific --epochs 10
```

```
# Backup current best model before trying new hyperparameters
crackz model export -p demo-project -c best -o backups/before_experiment.zip

# Run experiment
crackz detect train -p demo-project --epochs 50

# If results are worse, can import the backup
crackz model import -p demo-project -m backups/before_experiment.zip --force
```

```
# Export with timestamp
crackz model export -p demo-project -o models/model_2025-10-11.zip

# Or use git-style versioning
crackz model export -p demo-project -o models/v1.0.0.zip
```

```
# Export from training environment
crackz model export -p training-project -o production/model.zip

# Transfer to production server and import
crackz model import -p production-project -m production/model.zip

# Run detection in production
crackz detect run -p production-project
```

<checkpoint_name>.ckpt

metadata.json

```
# Linux/Mac
unzip -l my_model.zip

# Windows PowerShell
Expand-Archive -Path my_model.zip -DestinationPath temp -Force
cat temp\metadata.json
```

metadata.json

```
{
  "project_name": "harbor_detection",
  "checkpoint_name": "best.ckpt",
  "model_config": {
    "encoder": "resnet34",
    "encoder_weights": "imagenet",
    "in_channels": 3,
    "num_classes": 1,
    "threshold": 0.5,
    "loss_name": "combo",
    "dice_weight": 0.7,
    "focal_weight": 0.3
  }
}
```

```
# Generate map for raw split (default)
crackz geo create-map --project demo-project --output harbor_map.html

# Map for specific split (train/val/test)
crackz geo create-map --project demo-project --output train_map.html --split train
```

```
crackz geo create-map -p demo-project -o map.html -s raw
```

```
# Export all images with GPS data
crackz geo export-geojson --project demo-project --output detections.geojson

# Export specific split
crackz geo export-geojson --project demo-project --output train.geojson --split train
```

```
crackz geo export-geojson -p demo-project -o data.geojson -s raw
```

```
{
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "geometry": {
        "type": "Point",
        "coordinates": [longitude, latitude]
      },
      "properties": {
        "image_name": "IMG_0001.jpg",
        "image_path": "/path/to/image.jpg",
        "has_detection": true,
        "latitude": 59.123456,
        "longitude": 10.123456
      }
    }
  ],
  "metadata": {
    "project": "demo-project",
    "split": "raw",
    "total_images": 42,
    "images_with_detections": 15
  }
}
```

```
# Linux/Mac
exiftool image.jpg | grep GPS

# Python (Pillow)
from PIL import Image
img = Image.open("image.jpg")
exif = img.getexif()
gps_ifd = exif.get_ifd(0x8825)  # 0x8825 is GPS IFD tag
print(gps_ifd)
```

```
# 1. Import drone photos (GPS data preserved in EXIF)
crackz import-images --project harbor --src ./drone_photos --type raw --size 512 512

# 2. Run detection
crackz detect run --project harbor

# 3. Generate map to visualize results
```

```
crackz geo create-map --project harbor --output harbor_results.html

# 4. Export to GeoJSON for GIS analysis
crackz geo export-geojson --project harbor --output harbor.geojson

# 5. Open map in browser to review
# (or import GeoJSON into QGIS/ArcGIS)
```

```
crackz --debug detect run --project demo-project
crackz --quiet detect run --project demo-project
crackz --log-dir run_logs detect run --project demo-project
```

```
CRACKZ_LOG_DIR=logs_debug crackz detect run --project demo-project
```

`--debug`

```
# Start training
crackz detect train --project demo-project

# Press Ctrl+C to cancel at any time
# The operation will be cancelled gracefully
# Partial results (e.g., checkpoints) are preserved
```

```
crackz detect run --cfg configs/detect.yaml
crackz detect train --cfg configs/train.yaml
```

```
for P in demo-project other-project; do
  crackz detect run --project "$P" || echo "Failed: $P" >&2
done
```

`/home/app`

```
# Init project (host ./project mapped to /home/app)
mkdir -p project
docker run --rm -v %cd%\project:/home/app crackz-cli init --name demo-project

# Import images (mount samples to /data)
docker run --rm -v %cd%\project:/home/app -v %cd%\samples:/data \
  crackz-cli import-images --project /home/app/demo-project --src /data --type raw --size 512 512

# Detection
docker run --rm -v %cd%\project:/home/app crackz-cli detect run --project /home/app/demo-project
```

```
docker run --gpus all --rm -v %cd%\project:/home/app crackz-cli:gpu detect run --project /home/app/demo-project
```


`--project` `--cfg`

`--src`

`images/train` `raw`

`torch.cuda.is_available()`

```
# Generate report with default name (project_name_report.docx)
crackz report generate --project demo-project

# Specify custom output path
crackz report generate --project demo-project --output inspection_report.docx

# Use config file instead of project path
crackz report generate --cfg project.yaml --output report.docx
```

`crackz detect run`

```
# Find top 50 most uncertain images
uv run crackz active-learning run \
  --project demo-project \
  --top-n 50 \
  --method entropy \
  --out priorities.json
```

`entropy` `max_probability`

`variance` `mean_confidence`

```
uv run crackz active-learning info
```

```
# Prioritize images in 'raw' split (default)
uv run crackz active-learning run --project demo-project --split raw

# Prioritize images in 'train' split
uv run crackz active-learning run --project demo-project --split train
```

```
{
  "method": "entropy",
  "total_images": 500,
  "prioritized_count": 50,
  "images": [
    {
      "path": "image_042.jpg",
      "uncertainty_score": 0.8234,
      "rank": 1
    }
```

```
  ]
}
```

```
# Find similar images and propagate mask
uv run crackz propagate-masks run \
  --project demo-project \
  --reference wall_01.jpg \
  --mask wall_01_mask.png \
  --split raw \
  --threshold 0.8 \
  --out ./propagated_masks
```

`--threshold` `--min-matches` `--sift` `--orb`

```
uv run crackz propagate-masks find-similar \
  --project demo-project \
  --reference wall_01.jpg \
  --split raw \
  --out similarities.json
```

```
# More permissive matching (more propagations, less accurate)
uv run crackz propagate-masks run \
  --reference ref.jpg \
  --mask ref.png \
  --project demo-project \
  --threshold 0.6 \
  --min-matches 5

# Stricter matching (fewer propagations, more reliable)
uv run crackz propagate-masks run \
  --reference ref.jpg \
  --mask ref.png \
  --project demo-project \
  --threshold 0.9 \
  --min-matches 20 \
  --sift
```

```
uv run crackz propagate-masks run \
  --reference ref.jpg \
  --mask ref.png \
  --project demo-project \
  --orb
```

- `api.md` `detect()` `train()`

  `crackz export-config`

GUI

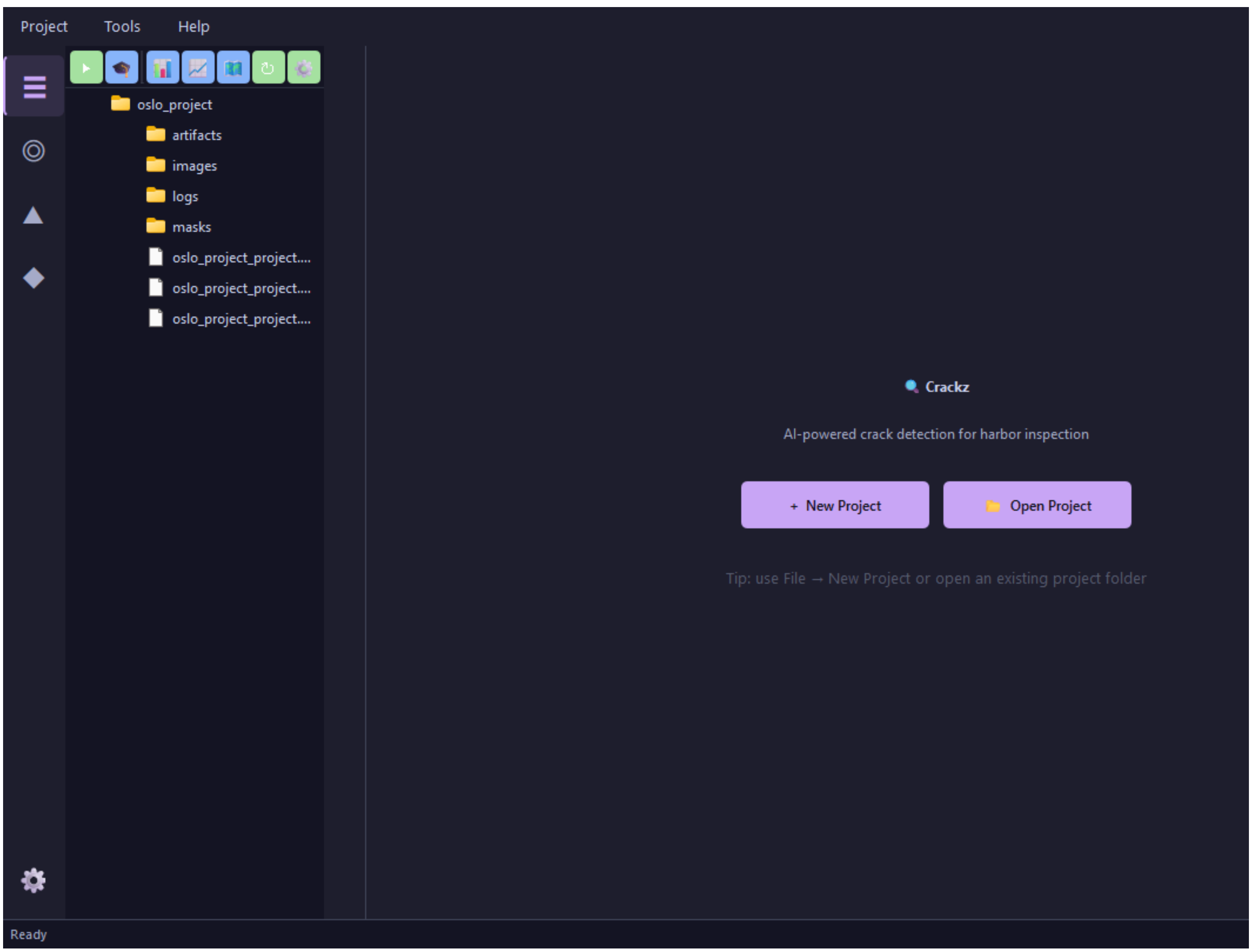
GUI Guide

`crackz-gui`

```
# After installing Crackz
crackz-gui
```

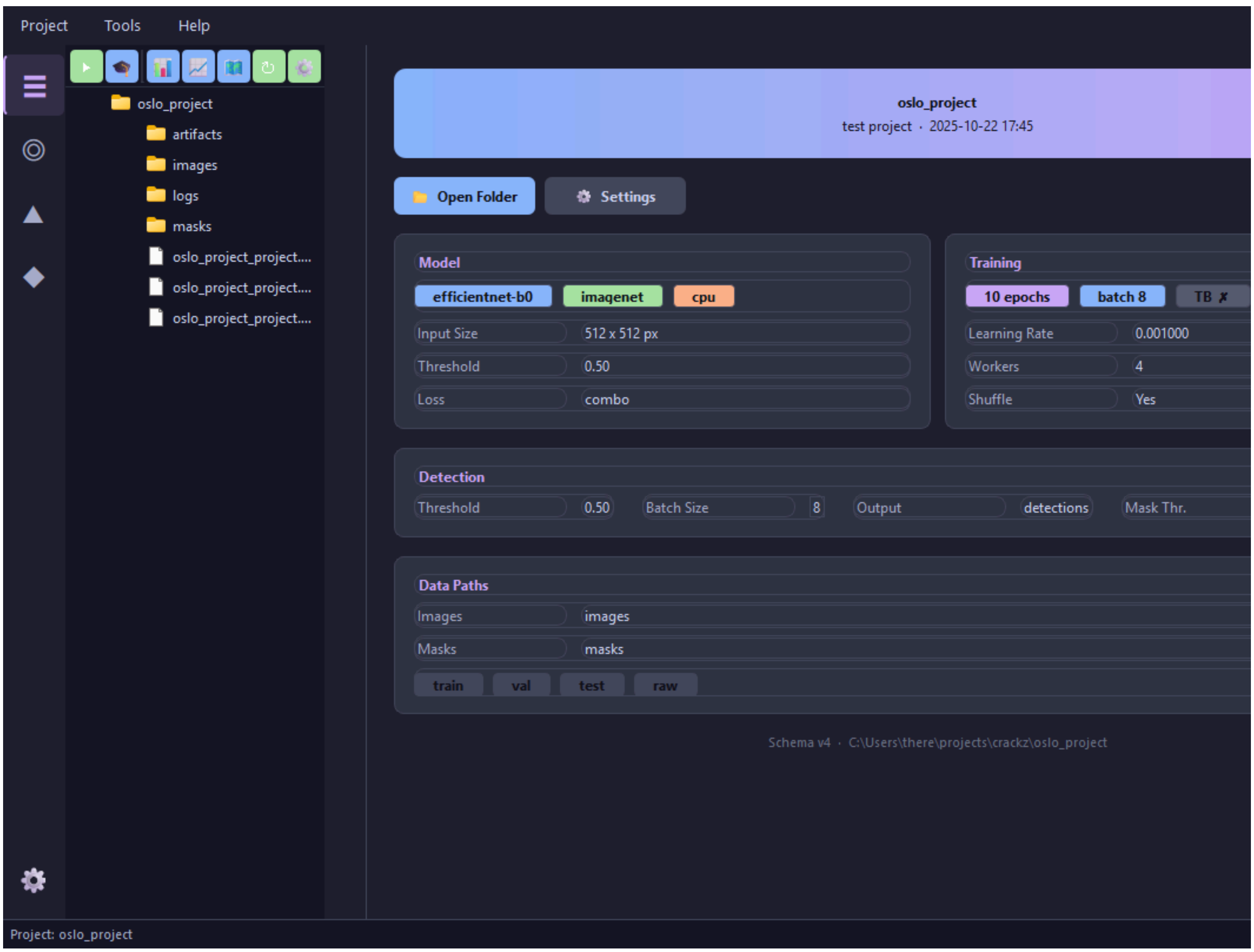
`crackz-gui.exe` `./crackz-gui`

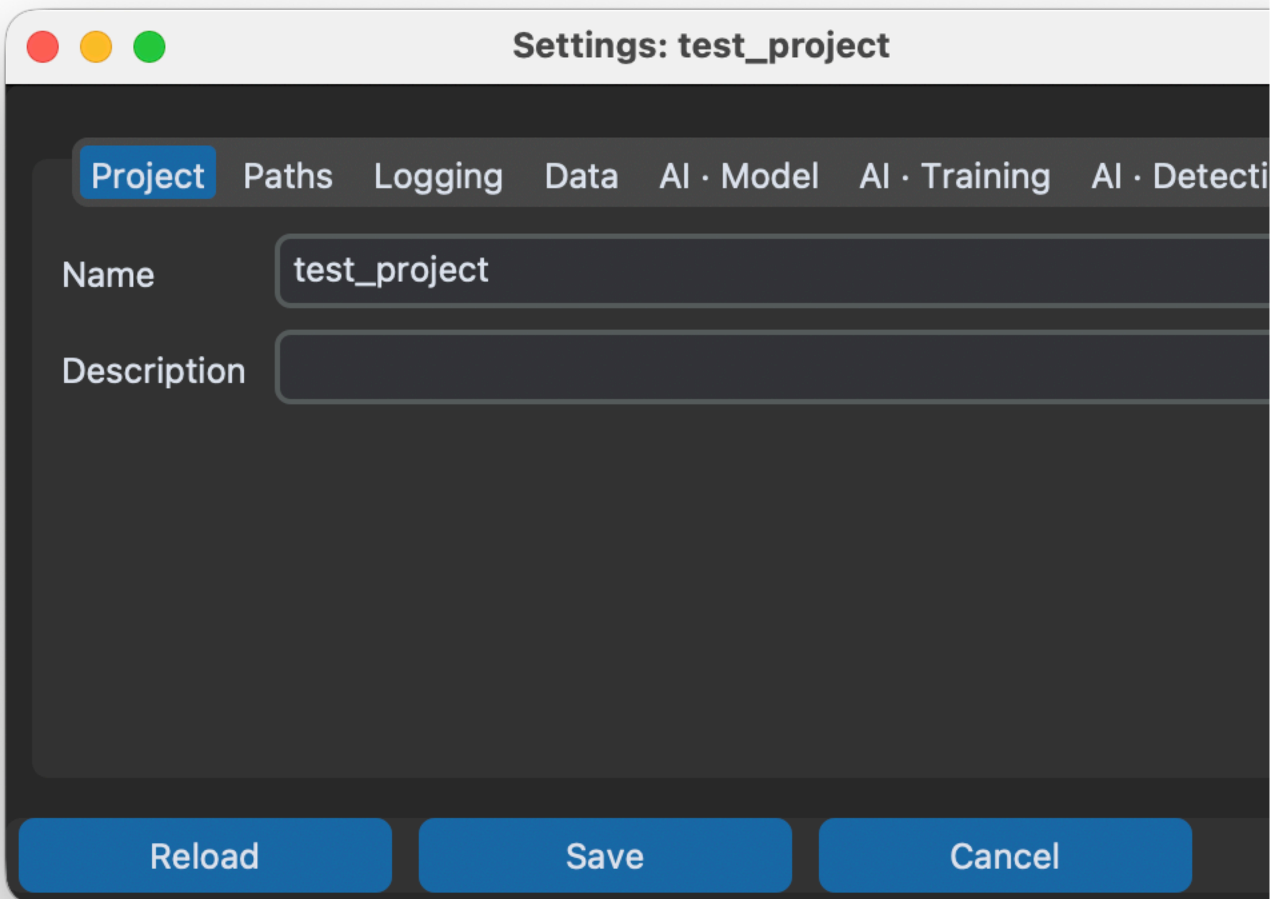
```
uv run python scripts/capture_qt_screenshots.py --project oslo_project
```

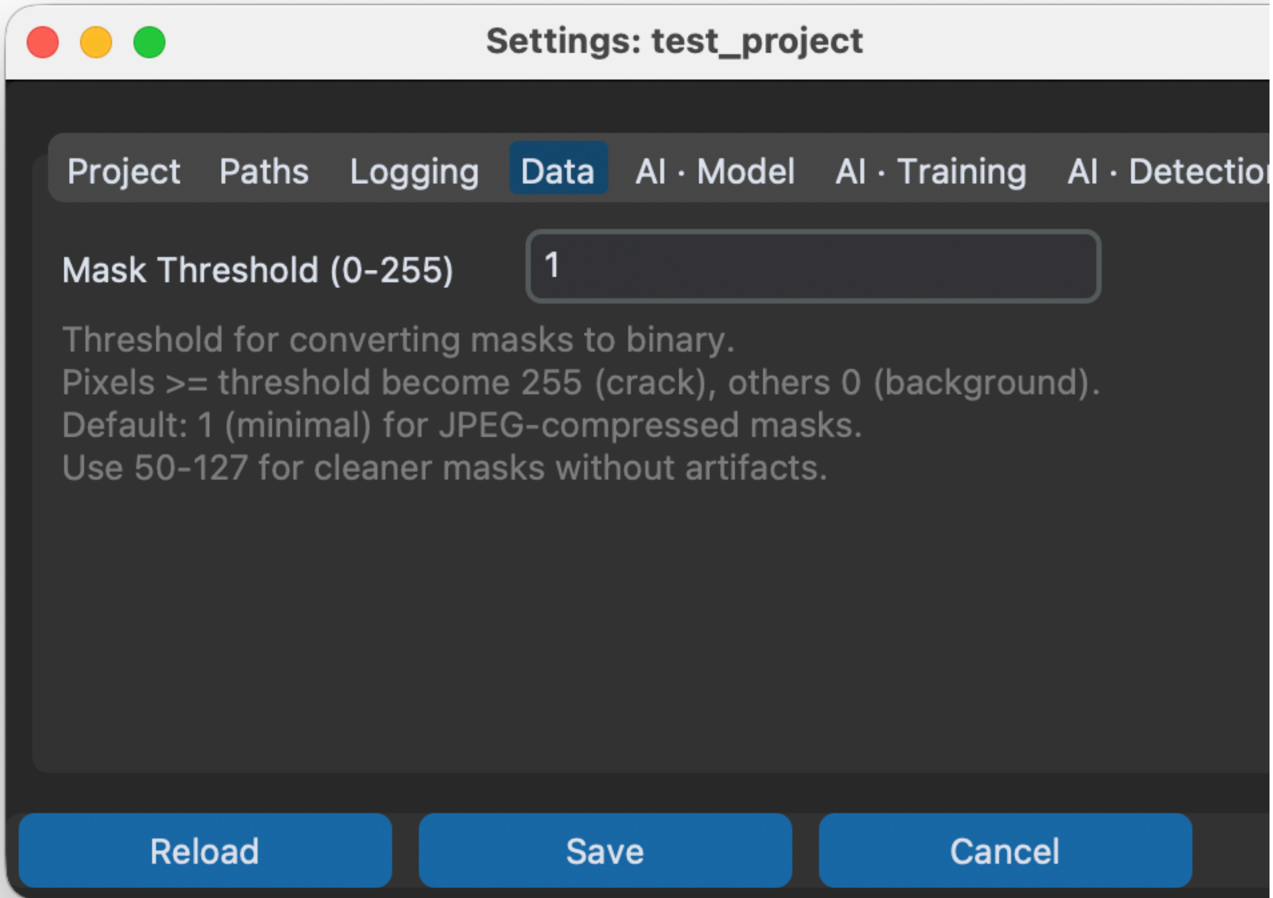


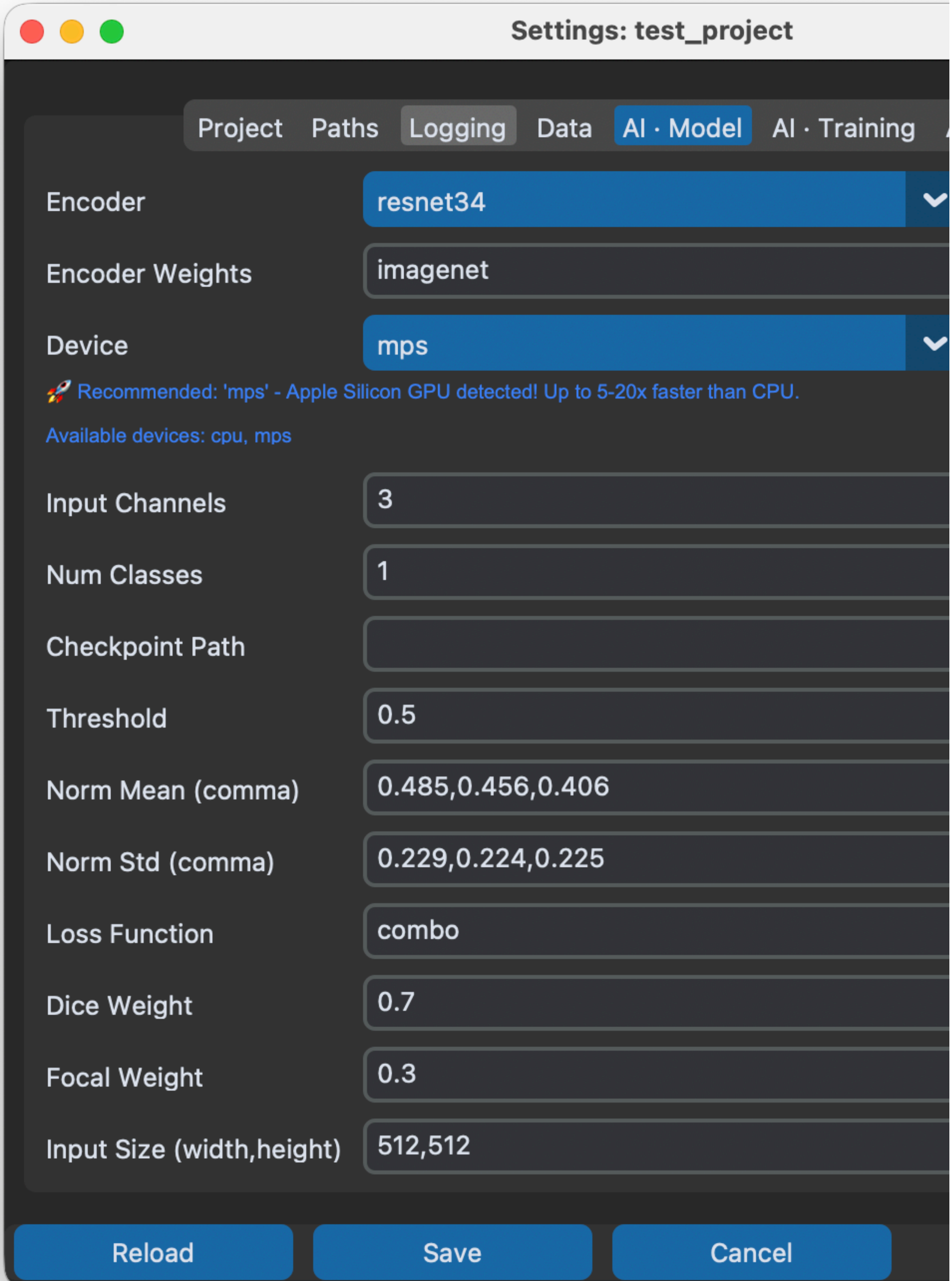
1.
2.
3.
4.

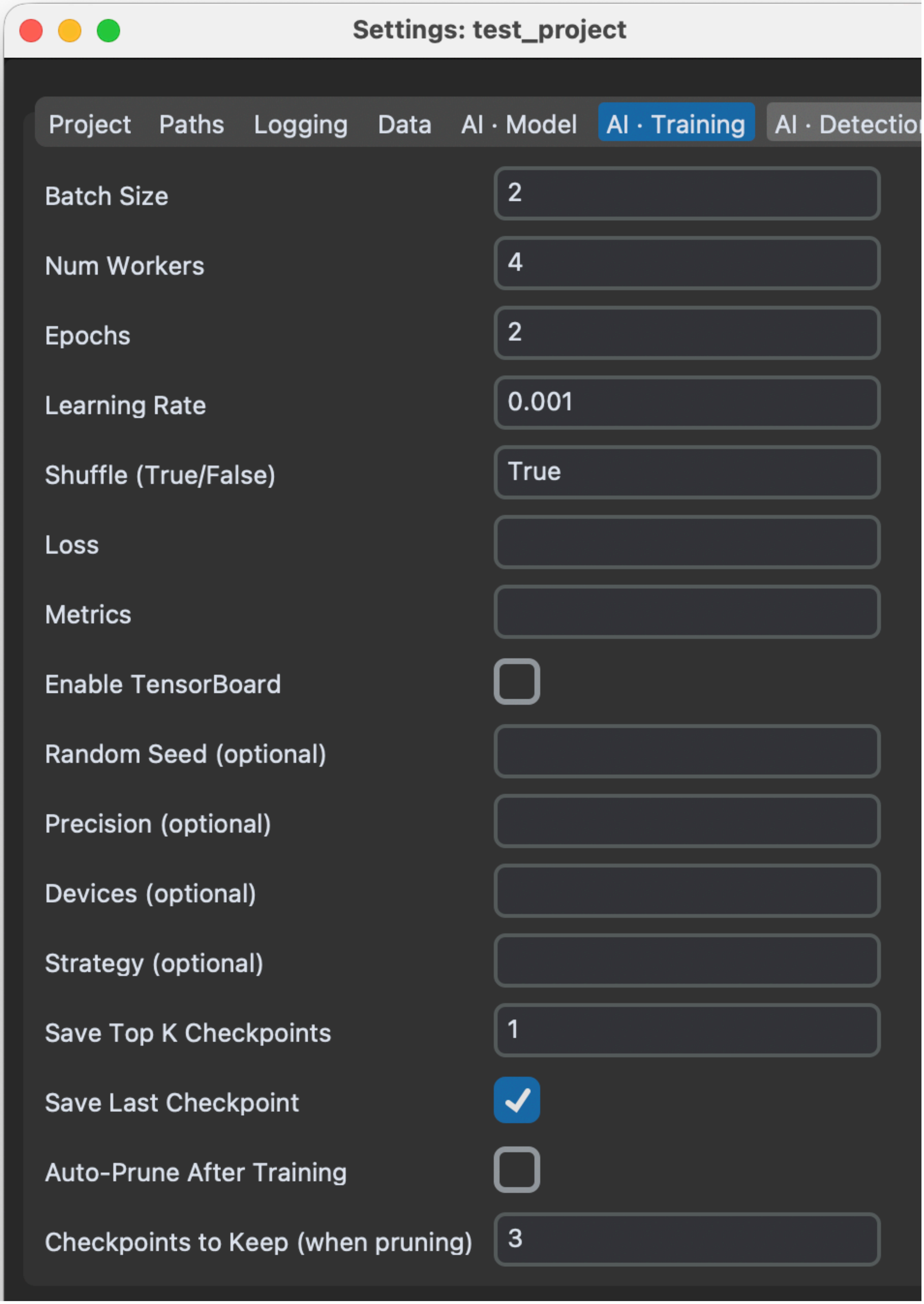
1.
2. `project.yaml`
3.

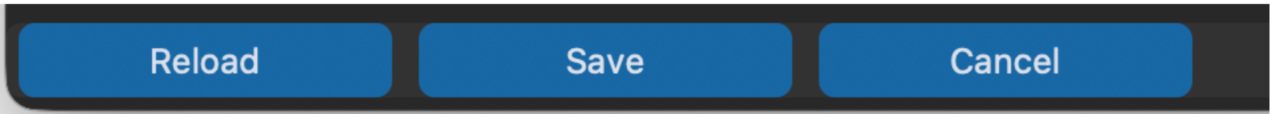


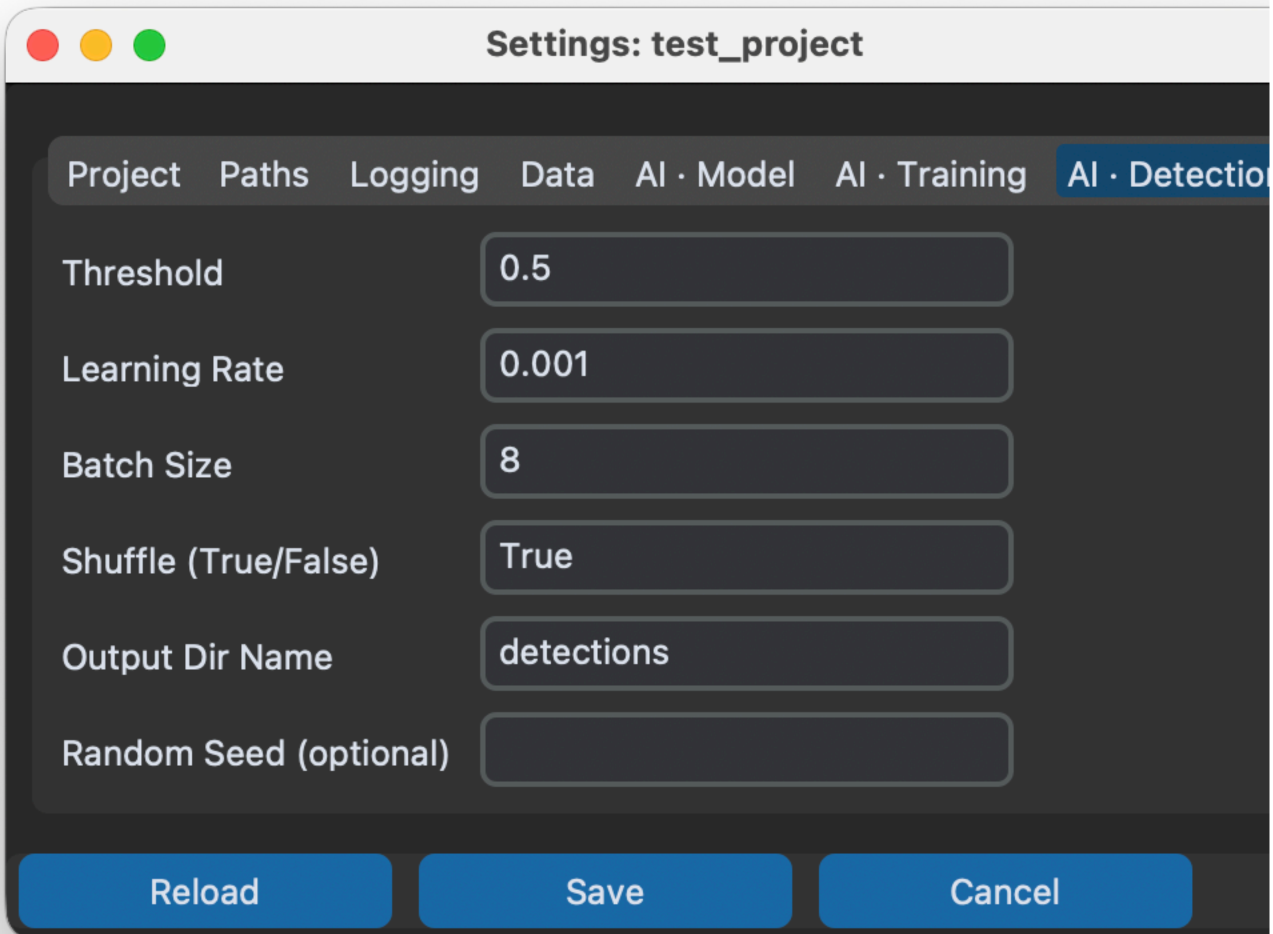

Settings: test_project
Project
Paths
Logging
Data
AI · Model
AI · Training
Name
test_project
Description
Reload
Save
Cancel

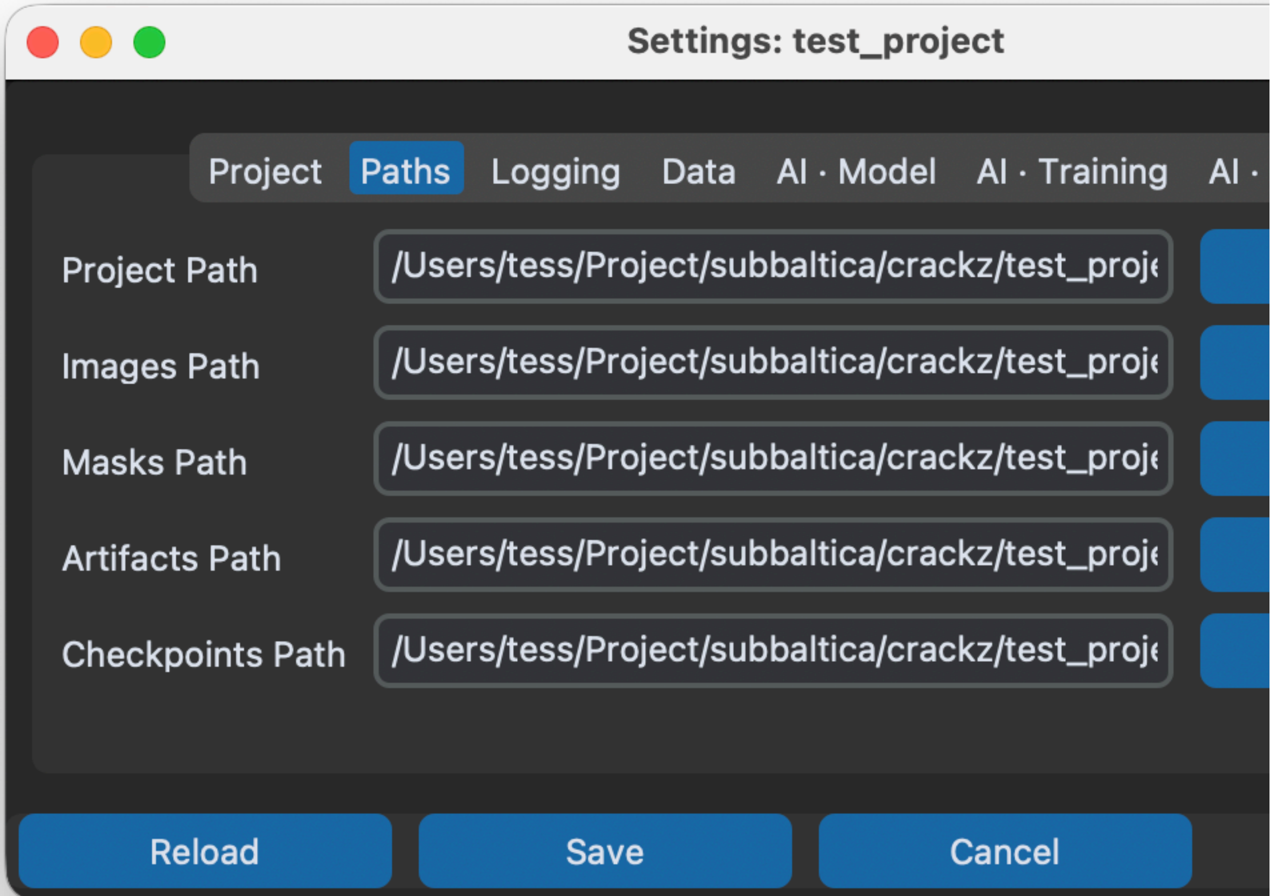

Settings: test_project
Project
Paths
Logging
Data
AI · Model
AI · Training
AI · Detectio
Mask Threshold (0-255)
1
Threshold for converting masks to binary.
Pixels >= threshold become 255 (crack), others 0 (background).
Default: 1 (minimal) for JPEG-compressed masks.
Use 50-127 for cleaner masks without artifacts.
Reload
Save
Cancel

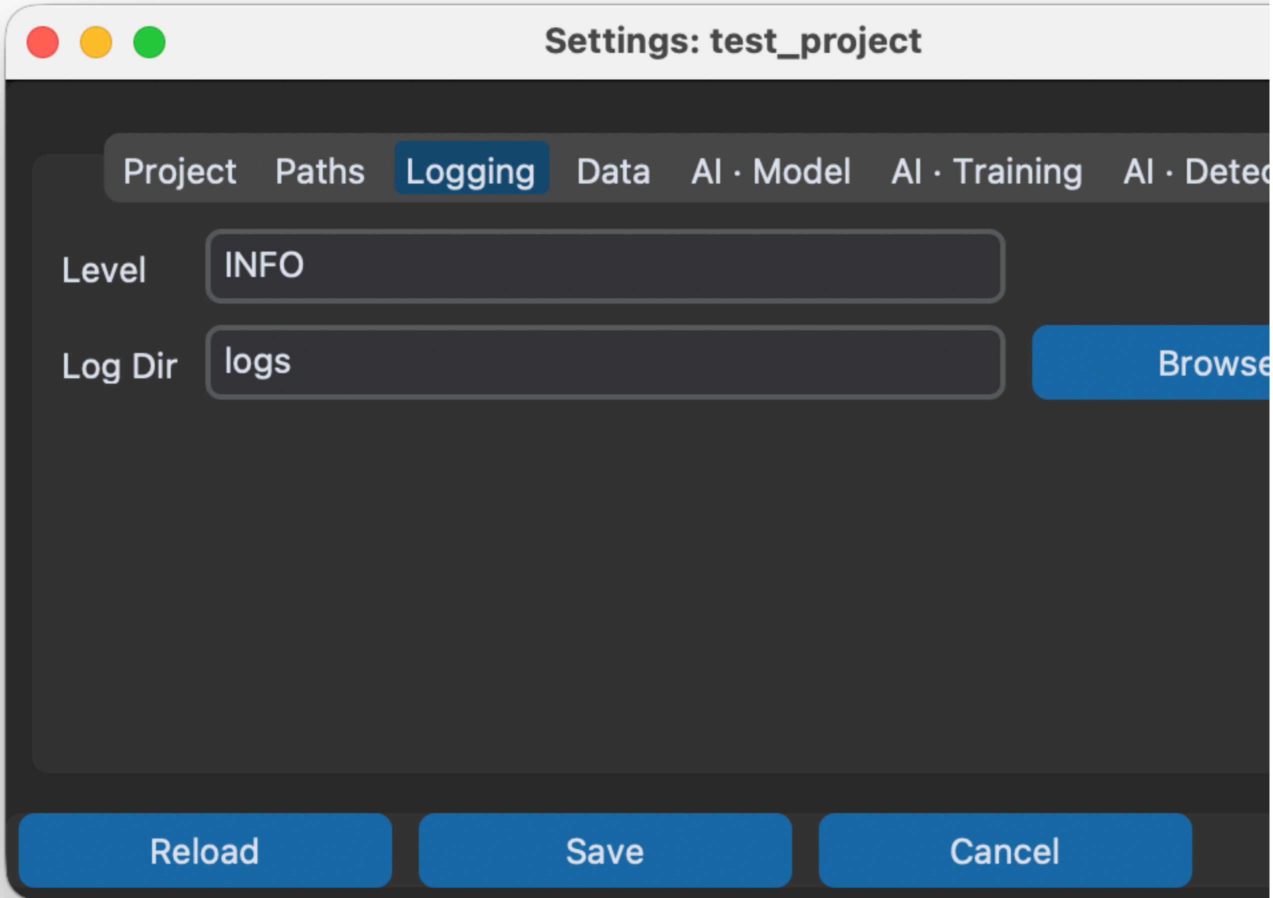

Settings: test_project
Project
Paths
Logging
Data
AI · Model
AI · Training
Encoder
resnet34
Encoder Weights
imagenet
Device
mps
Recommended: 'mps' - Apple Silicon GPU detected! Up to 5-20x faster than CPU.
Available devices: cpu, mps
Input Channels
3
Num Classes
1
Checkpoint Path
Threshold
0.5
Norm Mean (comma)
0.485,0.456,0.406
Norm Std (comma)
0.229,0.224,0.225
Loss Function
combo
Dice Weight
0.7
Focal Weight
0.3
Input Size (width,height)
512,512
Reload
Save
Cancel

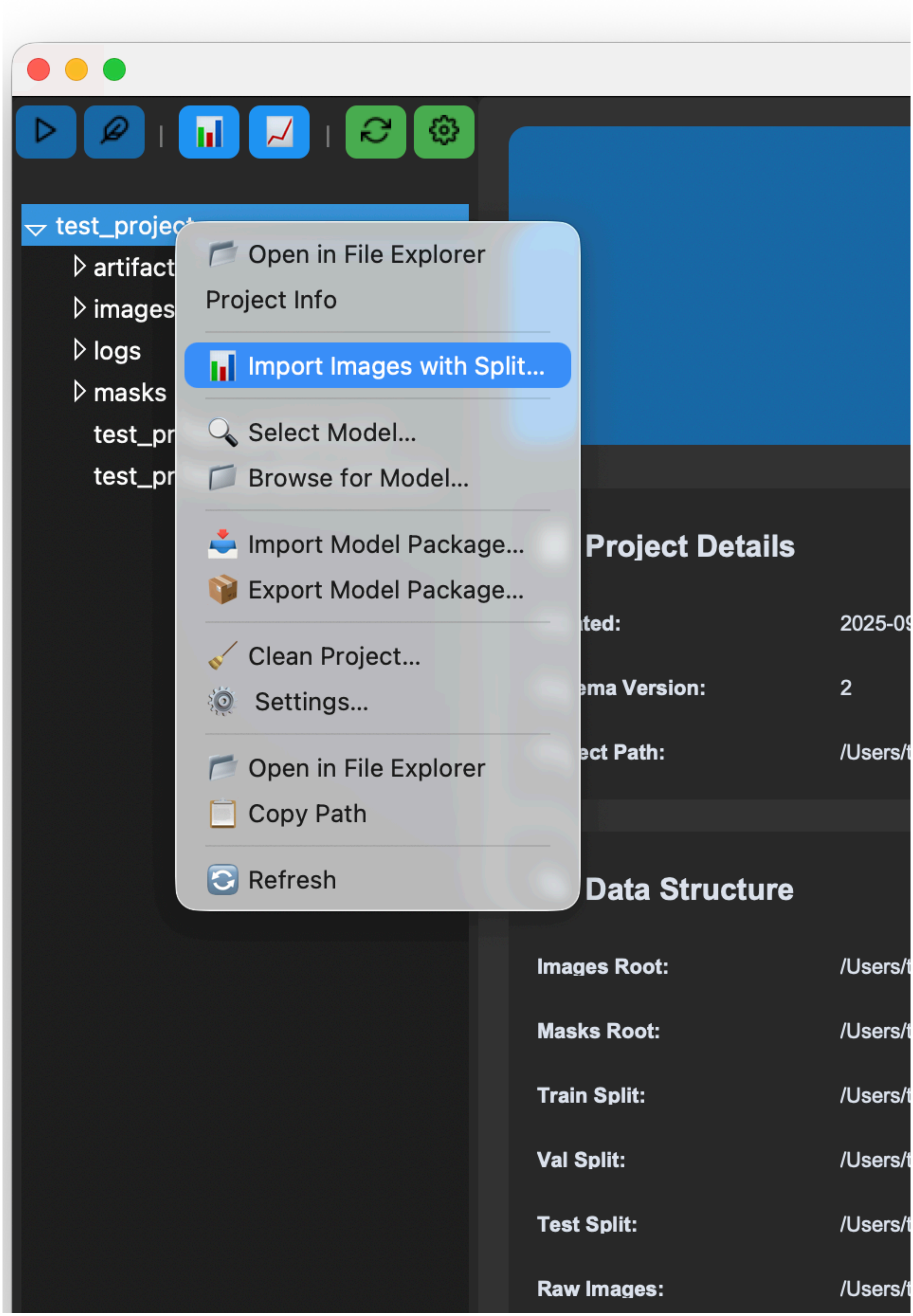

Settings: test_project
Project
Paths
Logging
Data
AI · Model
AI · Training
AI · Detectio
Batch Size
2
Num Workers
4
Epochs
2
Learning Rate
0.001
Shuffle (True/False)
True
Loss
Metrics
Enable TensorBoard
Random Seed (optional)
Precision (optional)
Devices (optional)
Strategy (optional)
Save Top K Checkpoints
1
Save Last Checkpoint
Auto-Prune After Training
Checkpoints to Keep (when pruning)
3

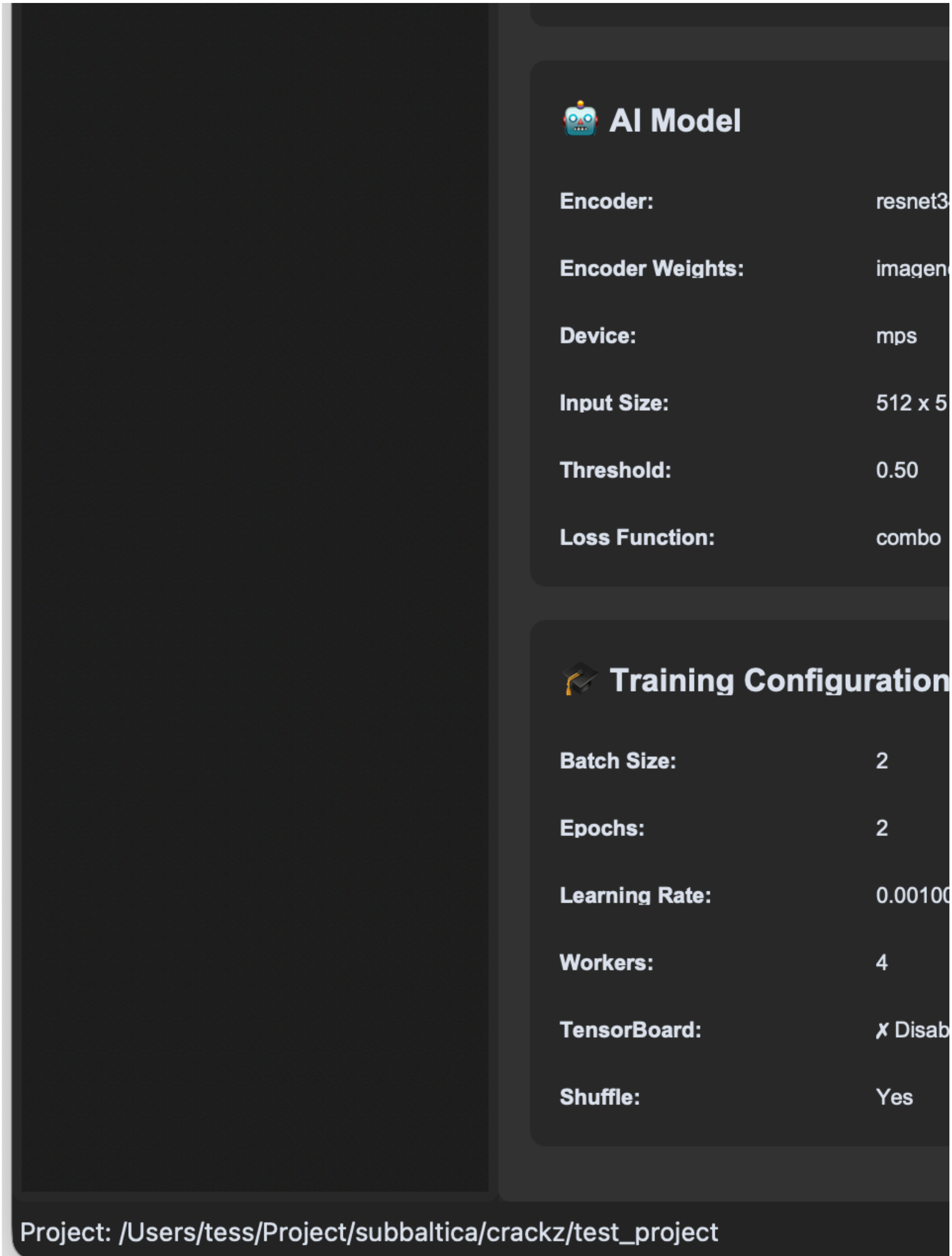

Reload
Save
Cancel


Settings: test_project
Project
Paths
Logging
Data
AI · Model
AI · Training
AI · Detectio
Threshold
0.5
Learning Rate
0.001
Batch Size
8
Shuffle (True/False)
True
Output Dir Name
detections
Random Seed (optional)
Reload
Save
Cancel


Settings: test_project
Project
Paths
Logging
Data
AI · Model
AI · Training
Project Path
/Users/tess/Project/subbaltica/crackz/test_proje
Images Path
/Users/tess/Project/subbaltica/crackz/test_proje
Masks Path
/Users/tess/Project/subbaltica/crackz/test_proje
Artifacts Path
/Users/tess/Project/subbaltica/crackz/test_proje
Checkpoints Path
/Users/tess/Project/subbaltica/crackz/test_proje
Reload
Save
Cancel

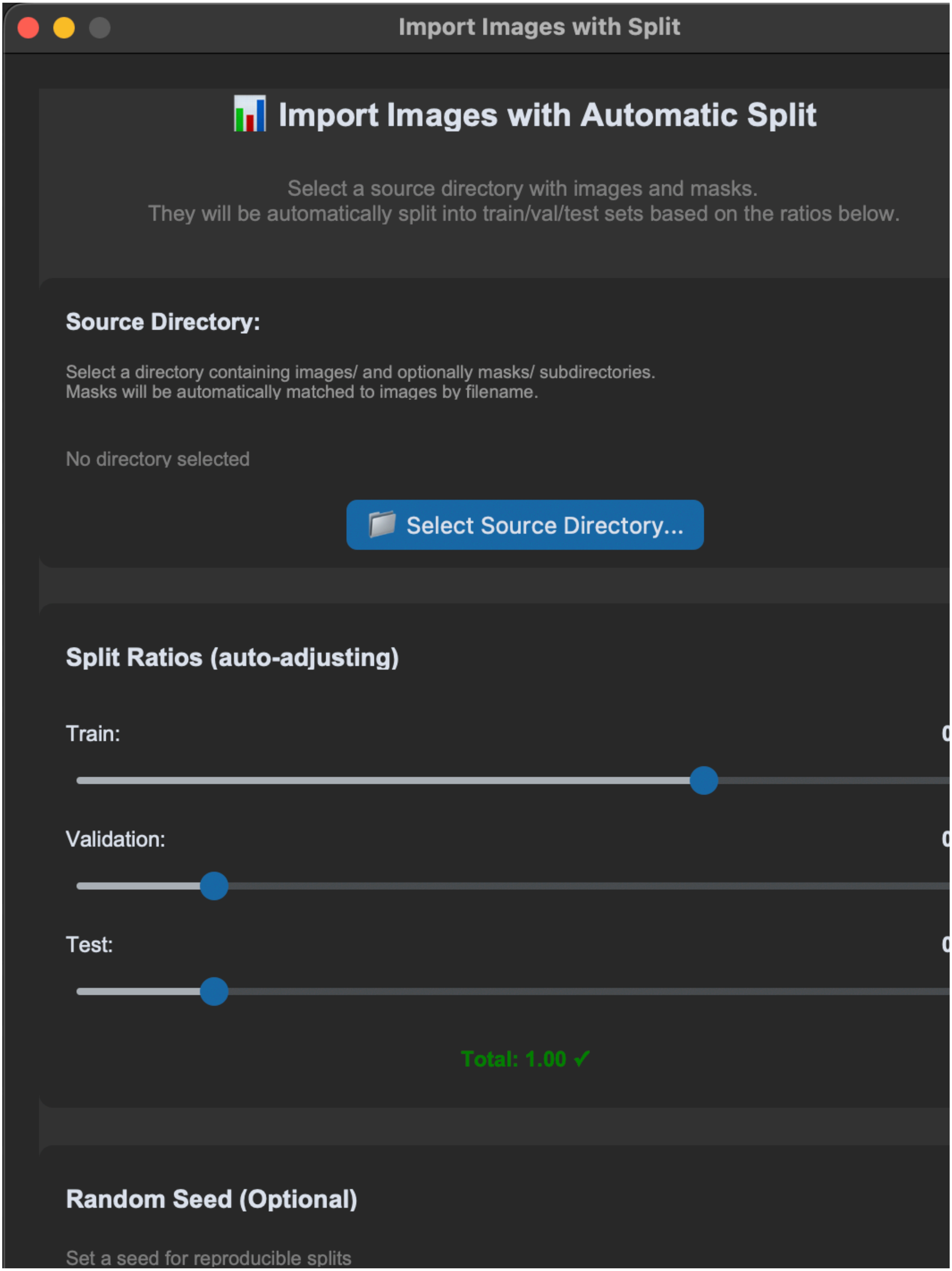

Settings: test_project
Project
Paths
Logging
Data
AI · Model
AI · Training
Level
INFO
Log Dir
logs
Reload
Save
Cancel


test_project
artifact
images
logs
masks
test_pr
test_pr
Open in File Explorer
Project Info
Import Images with Split...
Select Model...
Browse for Model...
Import Model Package...
Export Model Package...
Clean Project...
Settings...
Open in File Explorer
Copy Path
Refresh
Project Details
ted:
2025-0
ema Version:
2
ect Path:
/Users/
Data Structure
Images Root:
/Users/
Masks Root:
/Users/
Train Split:
/Users/
Val Split:
/Users/
Test Split:
/Users/
Raw Images:
/Users/

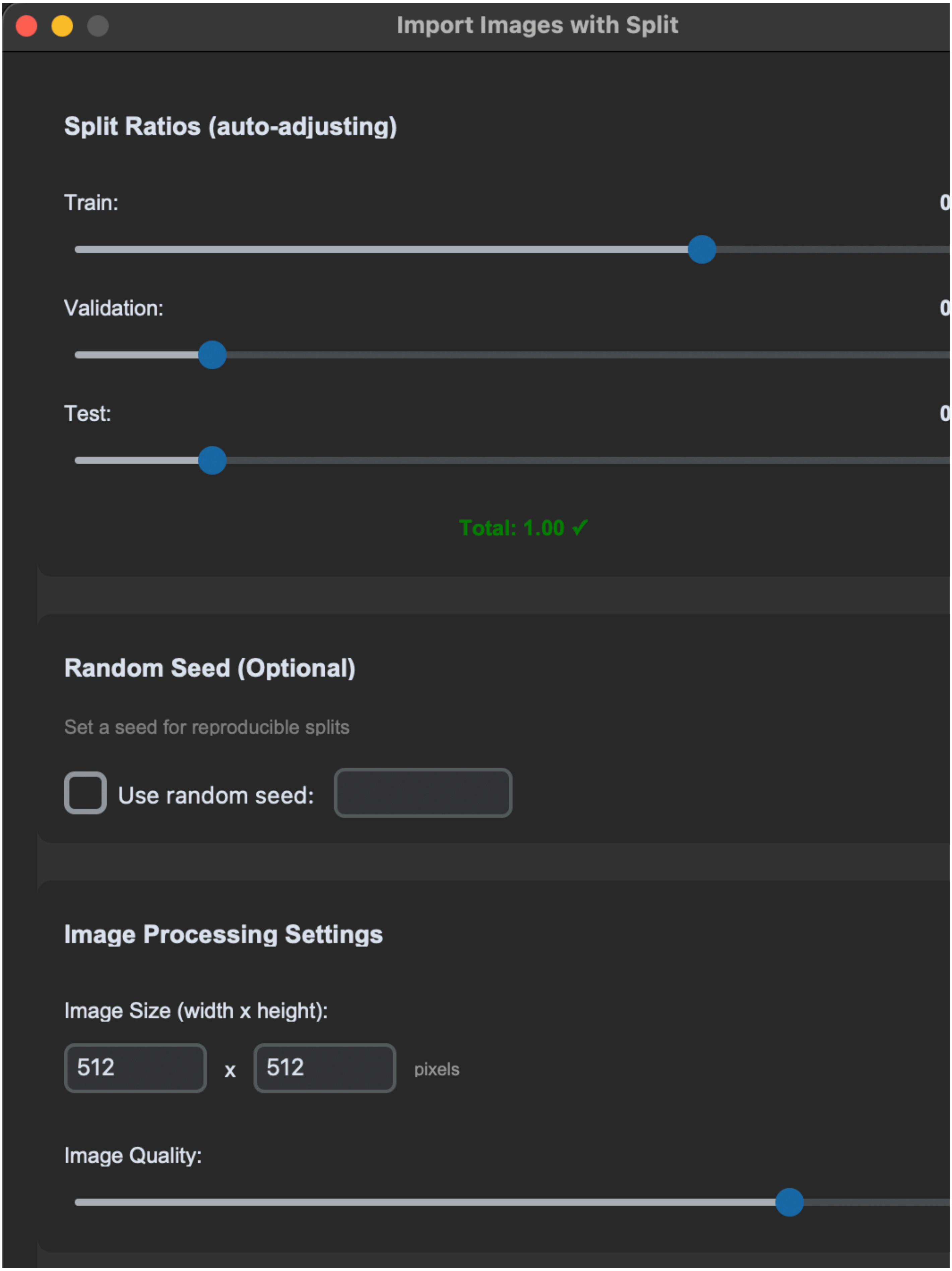

🤖 AI Model
Encoder:
resnet3
Encoder Weights:
imagen
Device:
mps
Input Size:
512 x 5
Threshold:
0.50
Loss Function:
combo
🎓 Training Configuration
Batch Size:
2
Epochs:
2
Learning Rate:
0.0010
Workers:
4
TensorBoard:
✗ Disab
Shuffle:
Yes
Project: /Users/tess/Project/subbaltica/crackz/test_project

1.
2.
3.
4.
5.
6.

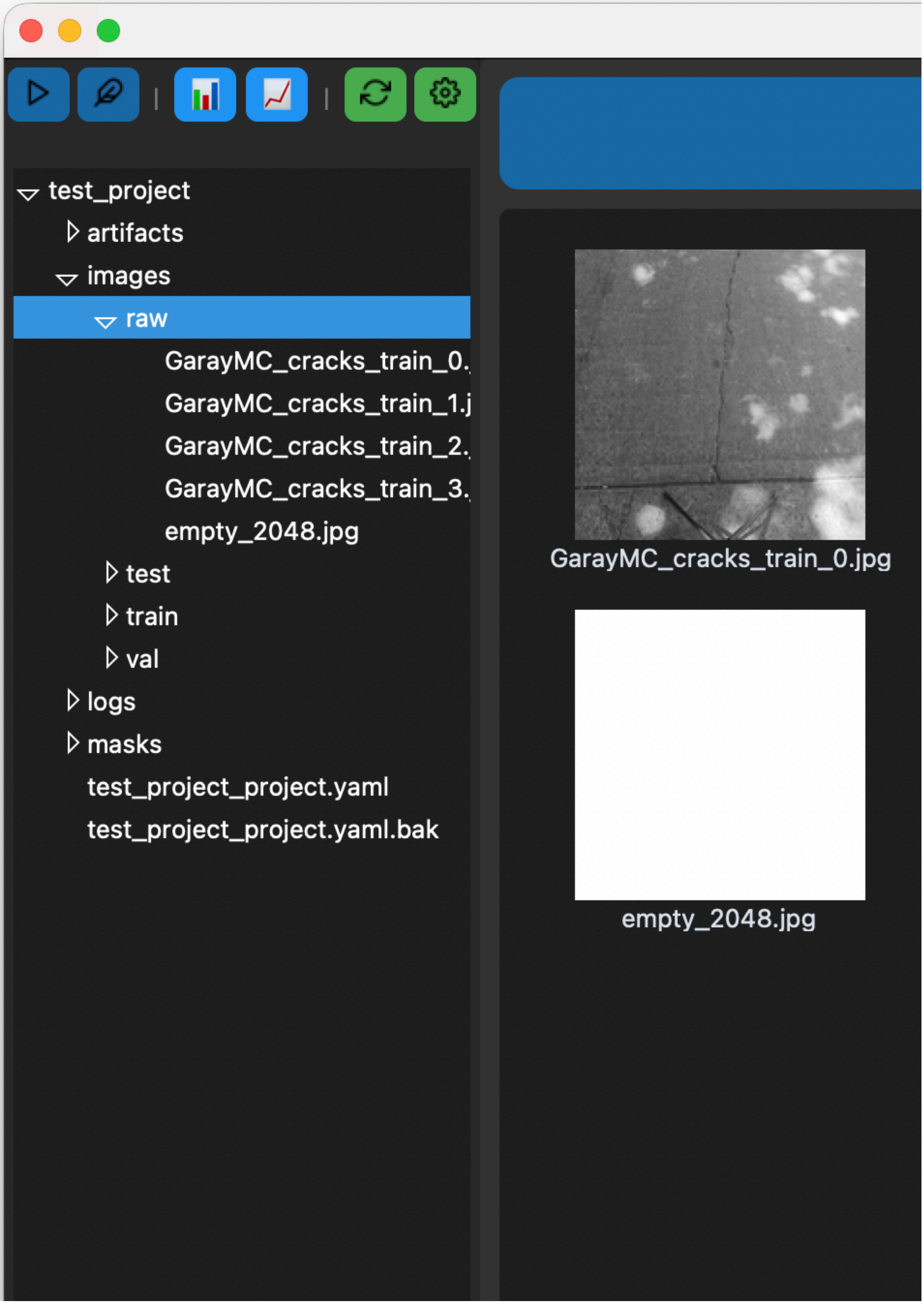
1.
2.
3.
4.
5.
6.

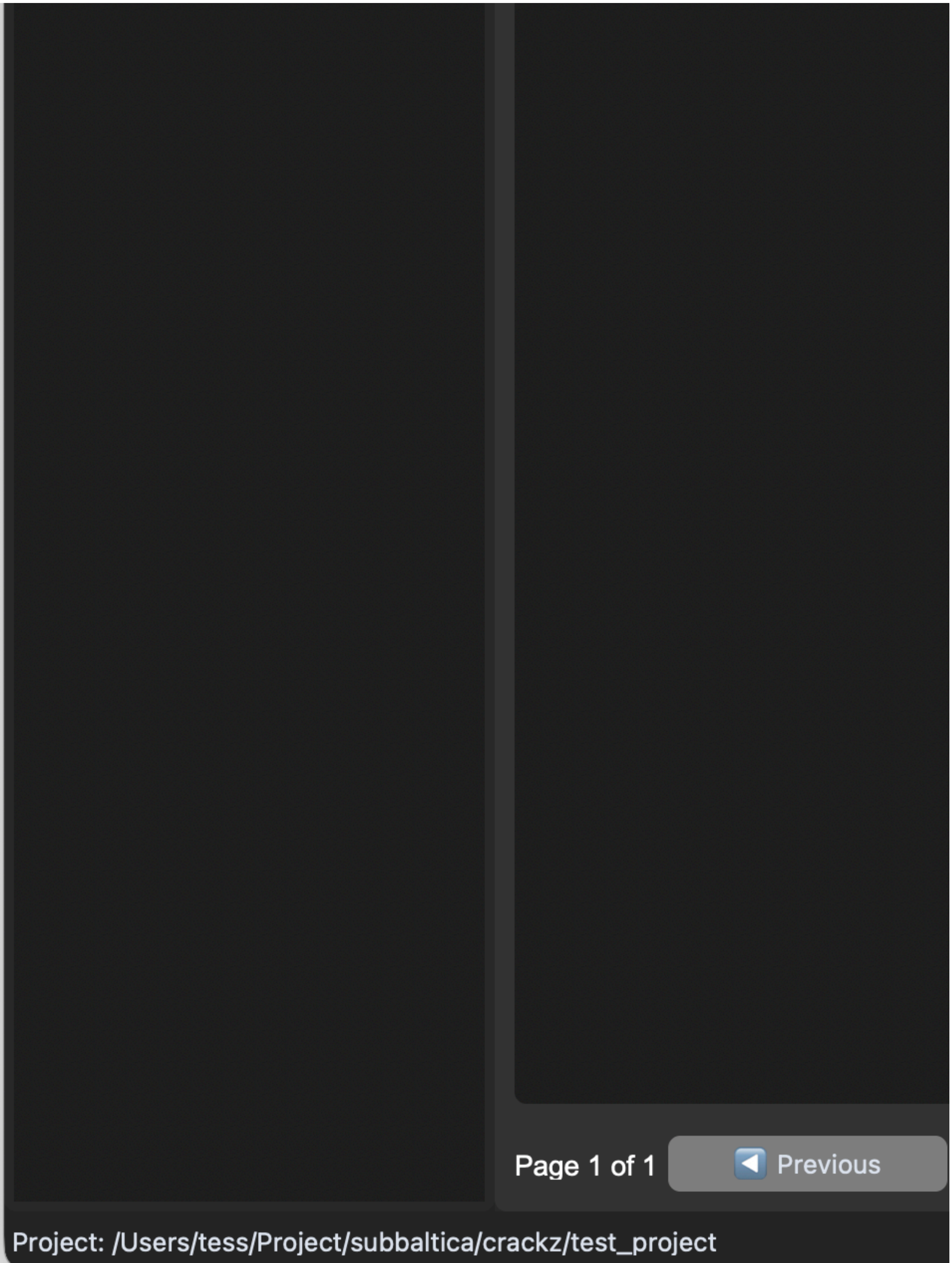

Import Images with Split
Import Images with Automatic Split
Select a source directory with images and masks.
They will be automatically split into train/val/test sets based on the ratios below.
Source Directory:
Select a directory containing images/ and optionally masks/ subdirectories.
Masks will be automatically matched to images by filename.
No directory selected
Select Source Directory...
Split Ratios (auto-adjusting)
Train:
Validation:
Test:
Total: 1.00 ✓
Random Seed (Optional)
Set a seed for reproducible splits


Import
Canc


Import Images with Split
Split Ratios (auto-adjusting)
Train:
Validation:
Test:
Total: 1.00 ✓
Random Seed (Optional)
Set a seed for reproducible splits
Use random seed:
Image Processing Settings
Image Size (width x height):
512
x
512
pixels
Image Quality:


Import
Canc


test_project
artifacts
images
raw
GarayMC_cracks_train_0.
GarayMC_cracks_train_1.j
GarayMC_cracks_train_2.
GarayMC_cracks_train_3.
empty_2048.jpg
test
train
val
logs
masks
test_project_project.yaml
test_project_project.yaml.bak
GarayMC_cracks_train_0.jpg
empty_2048.jpg


Page 1 of 1
Previous
Project: /Users/tess/Project/subbaltica/crackz/test_project

`images/`

`images/`

`images/`

`masks/`

		`B`	
		`E`	
		`Ctrl+Z` `Cmd+Z`	
		`Ctrl+Y` `Cmd+Shift+Z`	
		`Ctrl+S` `Cmd+S`	
		`Esc`	

```
project/
├── images/
│   └── train/
│       └── crack_001.jpg        # Original image
└── masks/
    └── train/
        └── crack_001.png        # Binary mask (auto-created)
```

1.
2.
3.
4.

1.
2.
3.
4.
5.

-
-
-
-
-
-
-

	`A`	
	`R`	
	`E`	

	`Right Arrow`	
	`Left Arrow`	

`masks/`

-
-
-

`A`

`R` `E`

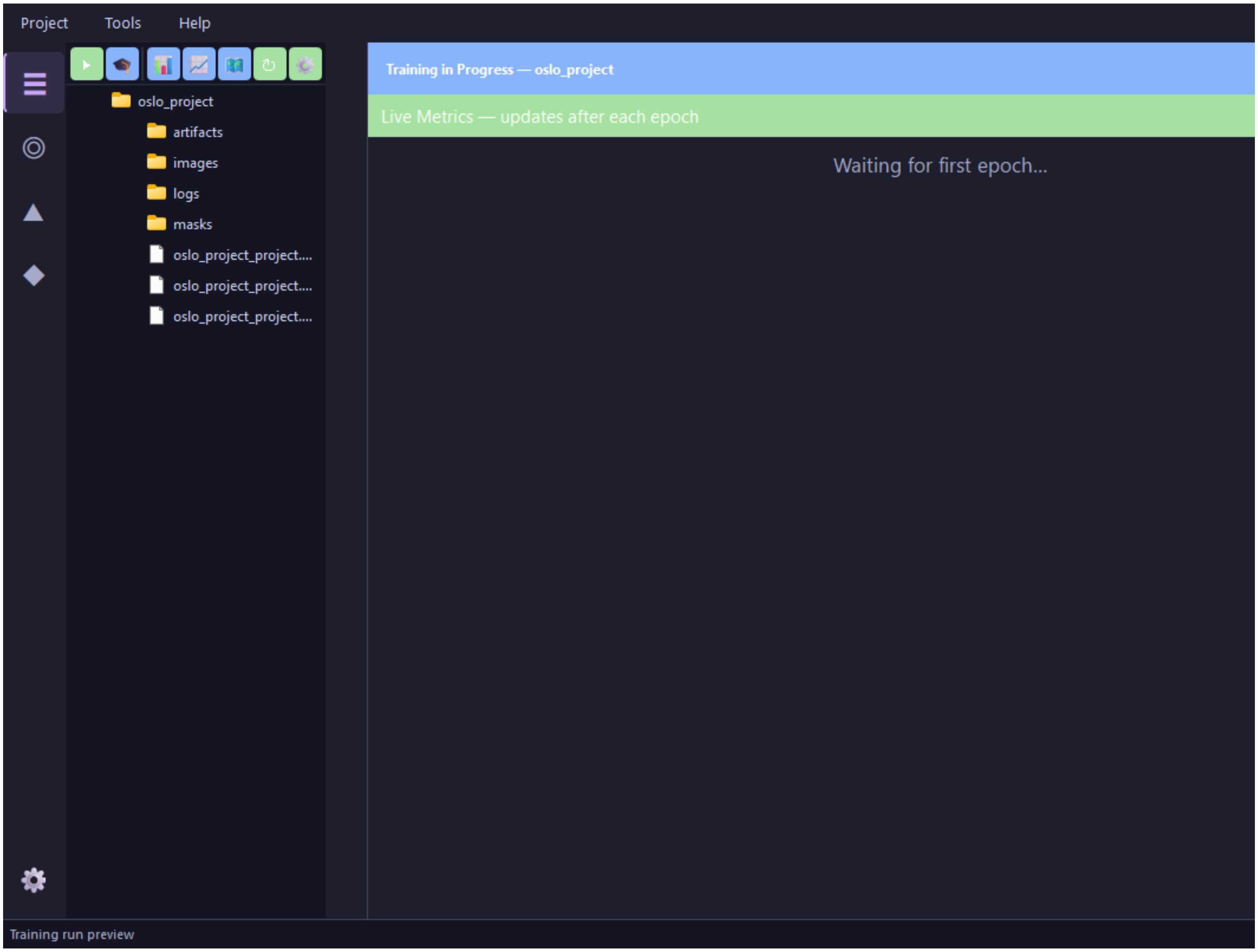
`raw/`

`raw/`

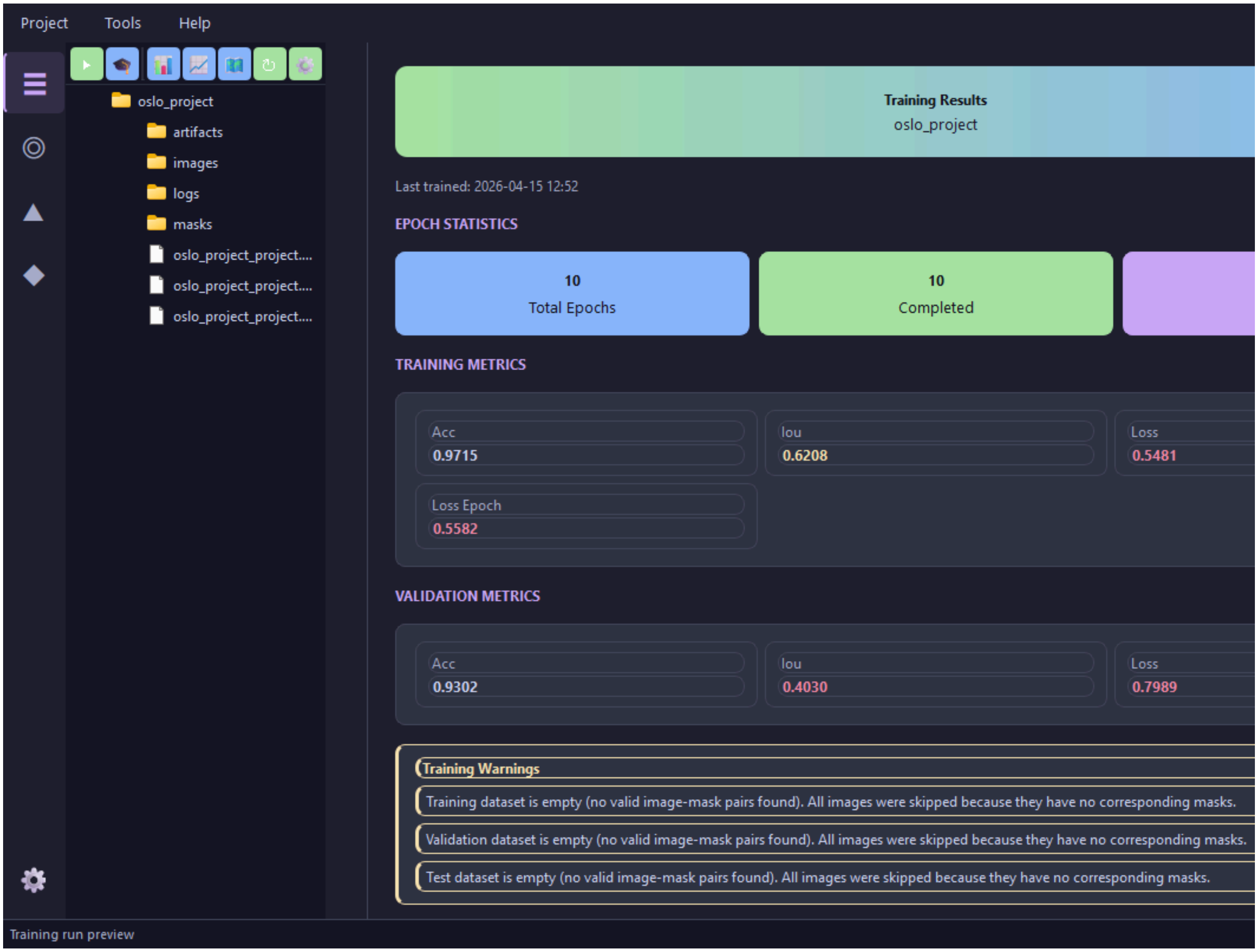
```
# Move annotated images from raw to train split
crackz move-images --project ./my-project --source raw --target train --pattern "crack_*.jpg"
```

`images/raw/`
`train` `val` `test`

1.

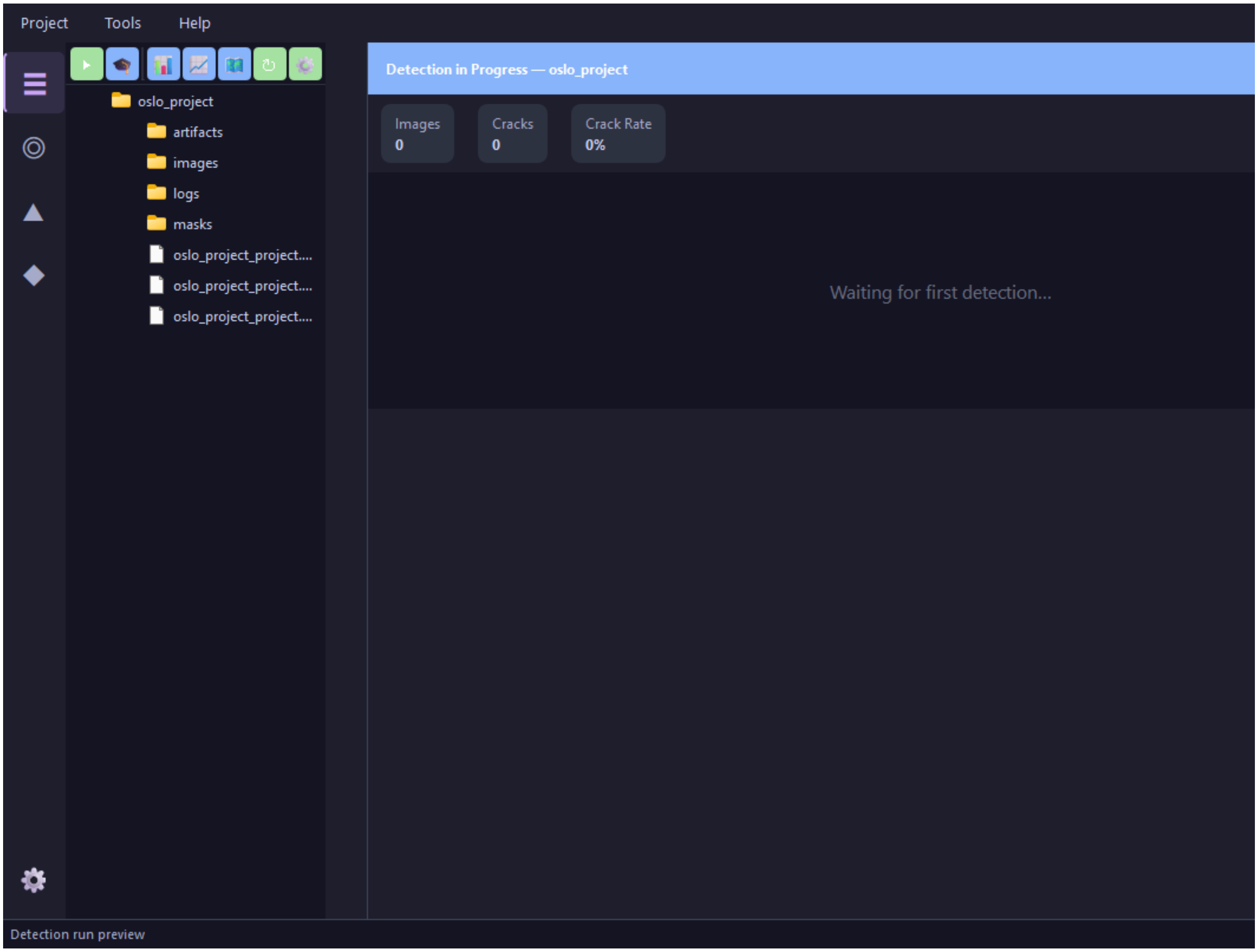


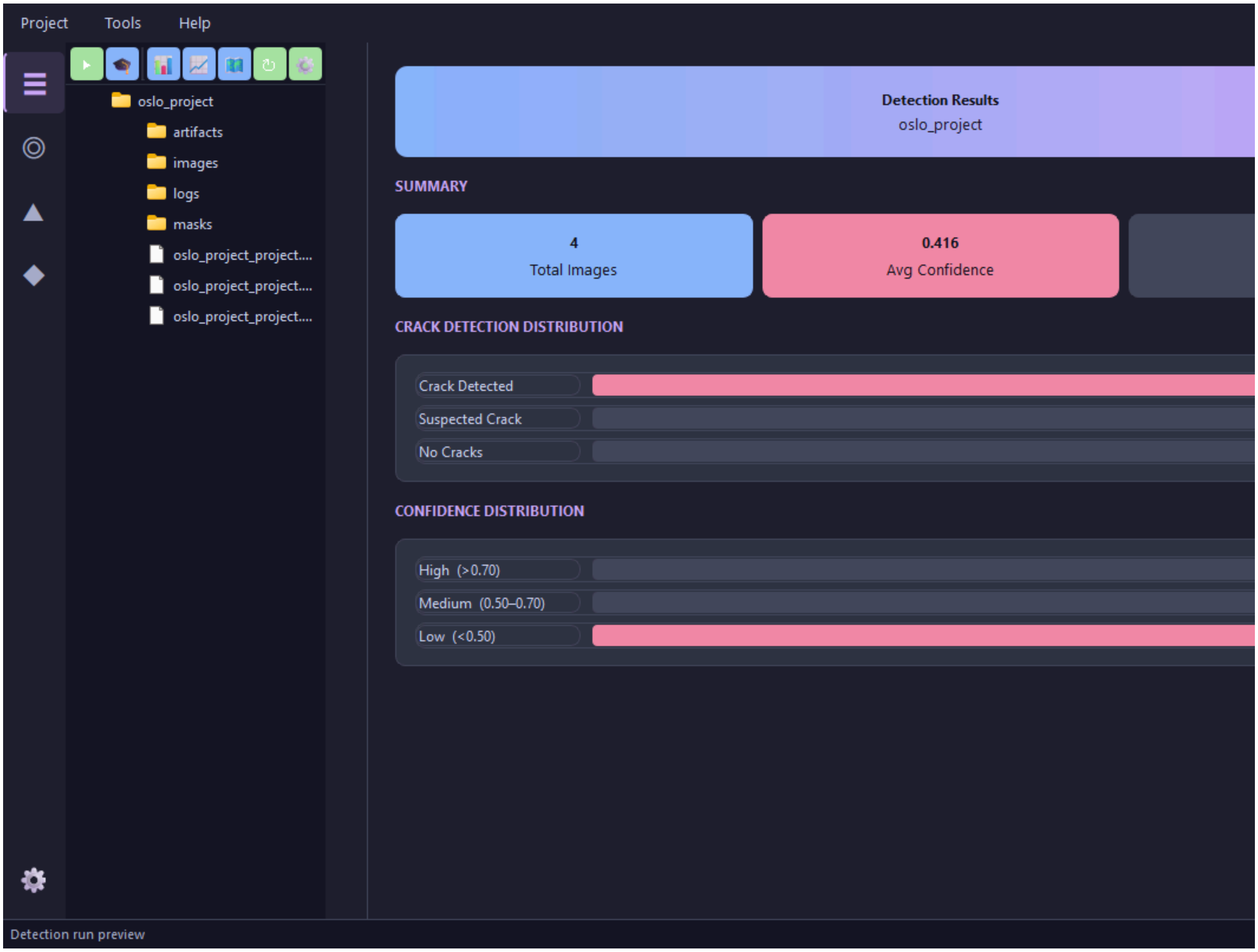
Project
Tools
Help
oslo_project
artifacts
images
logs
masks
oslo_project_project....
oslo_project_project....
oslo_project_project....
Training in Progress — oslo_project
Live Metrics — updates after each epoch
Waiting for first epoch...
Training run preview

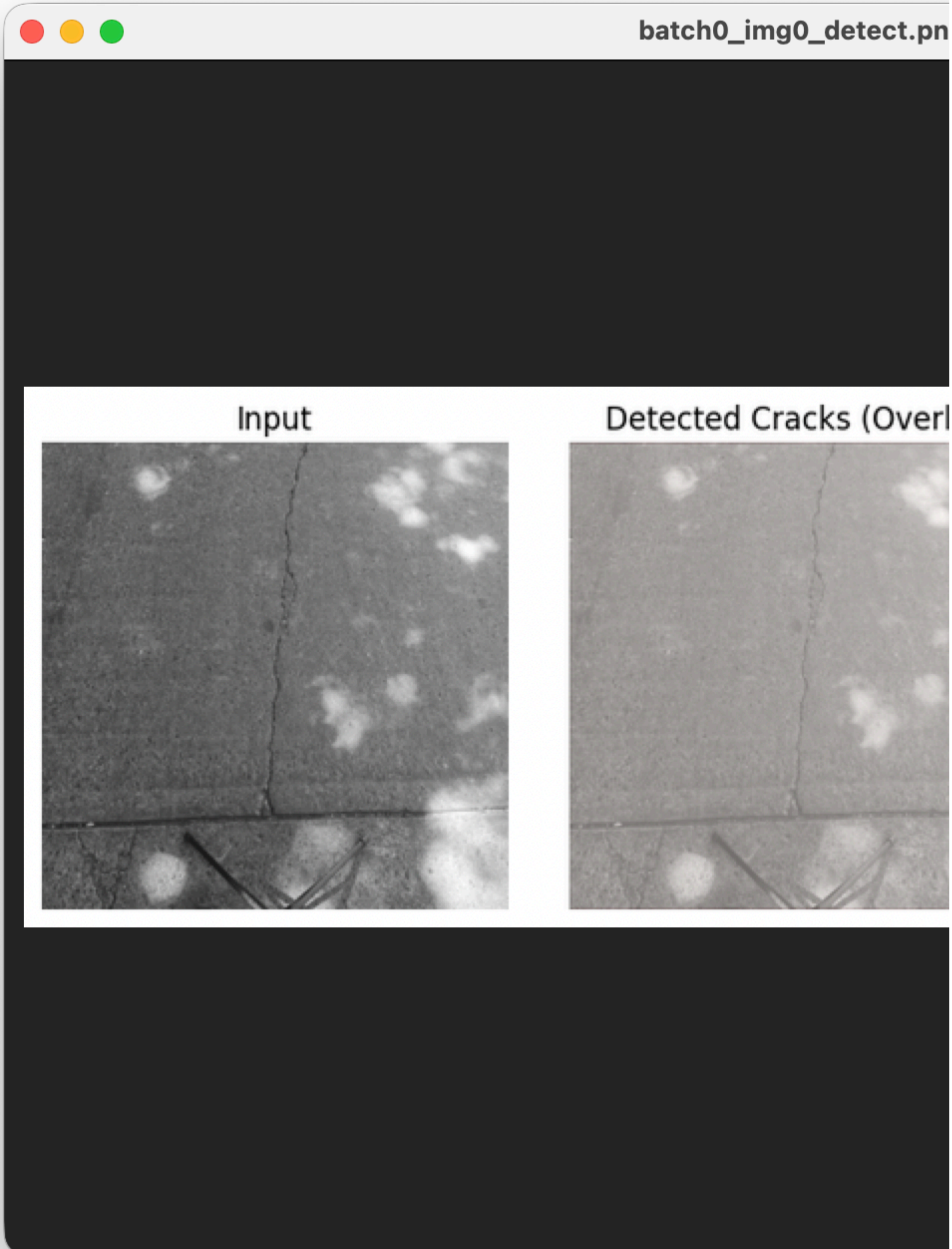

Project
Tools
Help
oslo_project
artifacts
images
logs
masks
oslo_project_project....
oslo_project_project....
oslo_project_project....
Training Results
oslo_project
Last trained: 2026-04-15 12:52
EPOCH STATISTICS
10
Total Epochs
10
Completed
TRAINING METRICS
Acc
0.9715
Iou
0.6208
Loss
0.5481
Loss Epoch
0.5582
VALIDATION METRICS
Acc
0.9302
Iou
0.4030
Loss
0.7989
Training Warnings
Training dataset is empty (no valid image-mask pairs found). All images were skipped because they have no corresponding masks.
Validation dataset is empty (no valid image-mask pairs found). All images were skipped because they have no corresponding masks.
Test dataset is empty (no valid image-mask pairs found). All images were skipped because they have no corresponding masks.
Training run preview

---

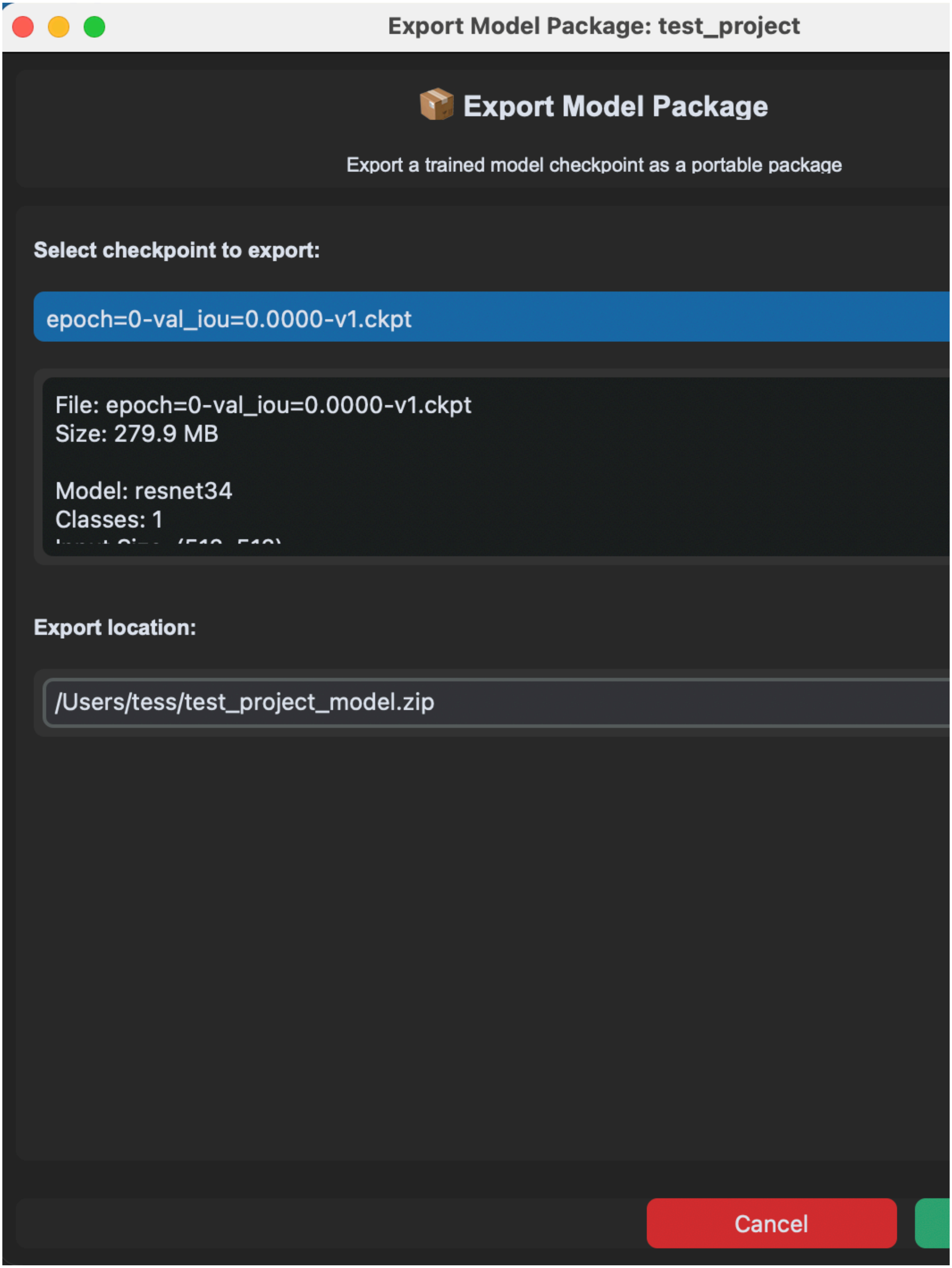
1. `raw`
2.
3.
4.


Project
Tools
Help
oslo_project
artifacts
images
logs
masks
oslo_project_project....
oslo_project_project....
oslo_project_project....
Detection in Progress — oslo_project
Images
0
Cracks
0
Crack Rate
0%
Waiting for first detection...
Detection run preview

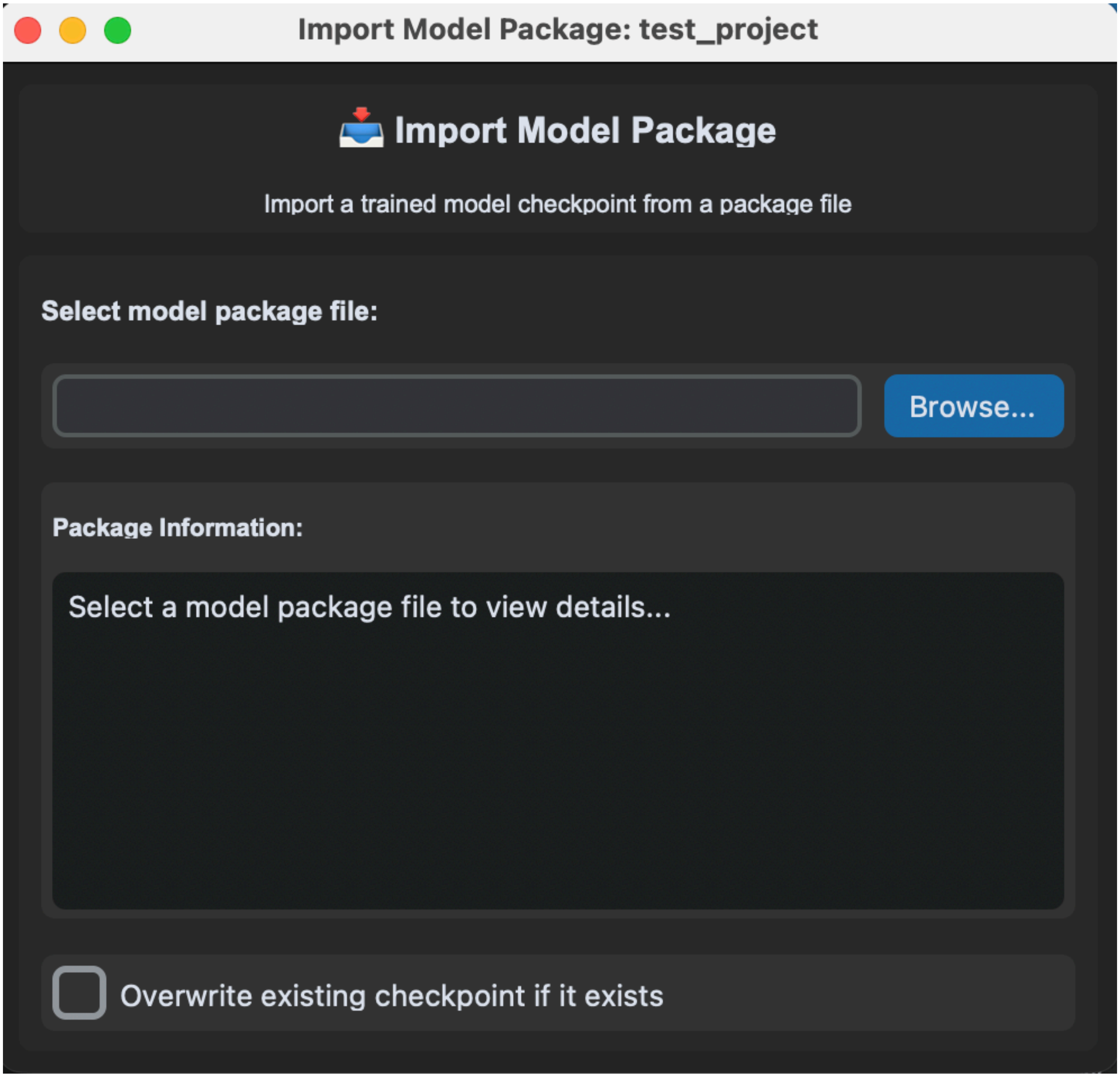

Project
Tools
Help
oslo_project
artifacts
images
logs
masks
oslo_project_project....
oslo_project_project....
oslo_project_project....
Detection Results
oslo_project
SUMMARY
4
Total Images
0.416
Avg Confidence
CRACK DETECTION DISTRIBUTION
Crack Detected
Suspected Crack
No Cracks
CONFIDENCE DISTRIBUTION
High (>0.70)
Medium (0.50–0.70)
Low (<0.50)
Detection run preview


batch0_img0_detect.pn
Input
Detected Cracks (Overl

1.
2.
3.
4.

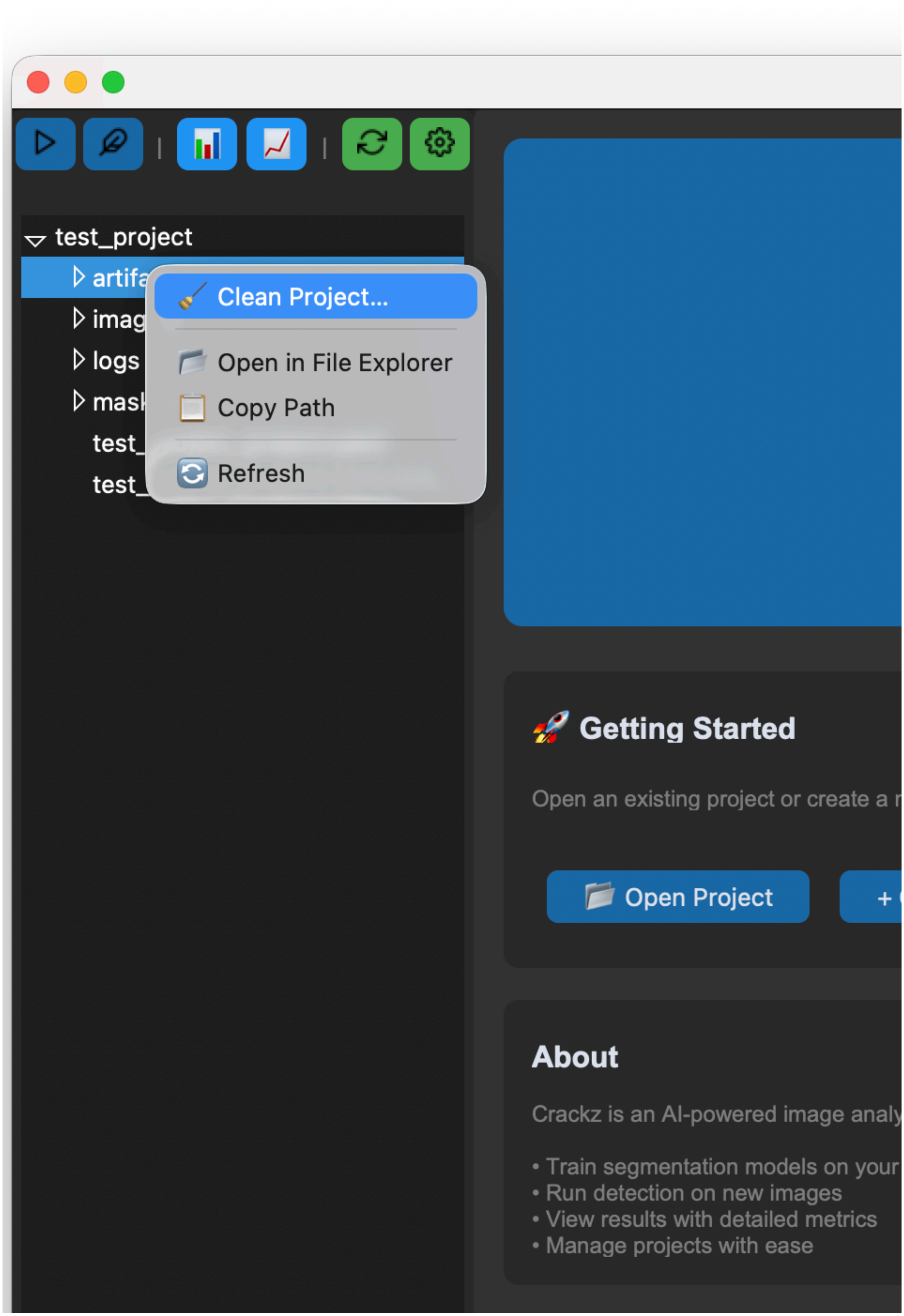

Export Model Package: test_project
Export Model Package
Export a trained model checkpoint as a portable package
Select checkpoint to export:
epoch=0-val_iou=0.0000-v1.ckpt
File: epoch=0-val_iou=0.0000-v1.ckpt
Size: 279.9 MB
Model: resnet34
Classes: 1
Export location:
/Users/tess/test_project_model.zip
Cancel

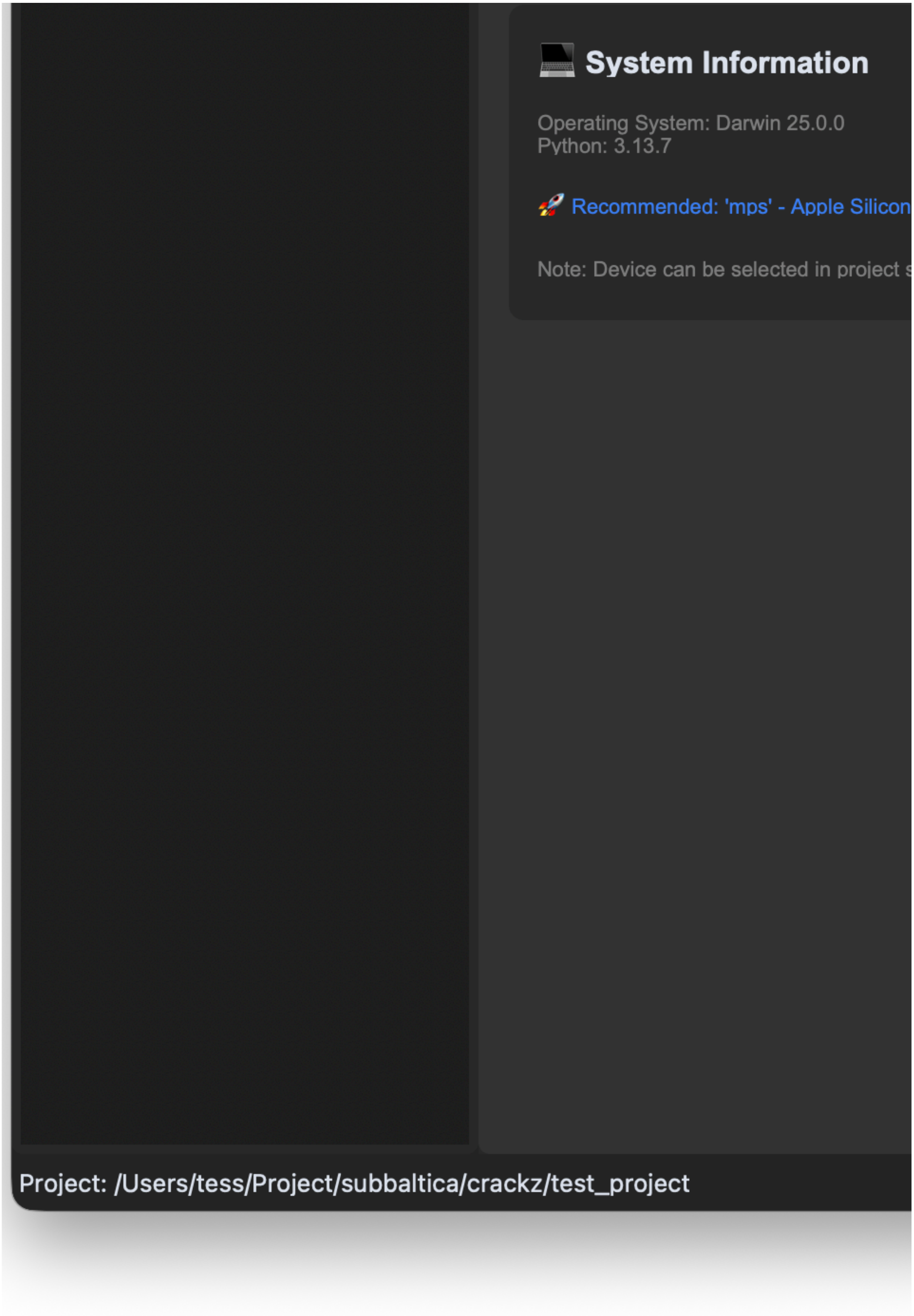
1.
2. `.ckpt`
3.
4.

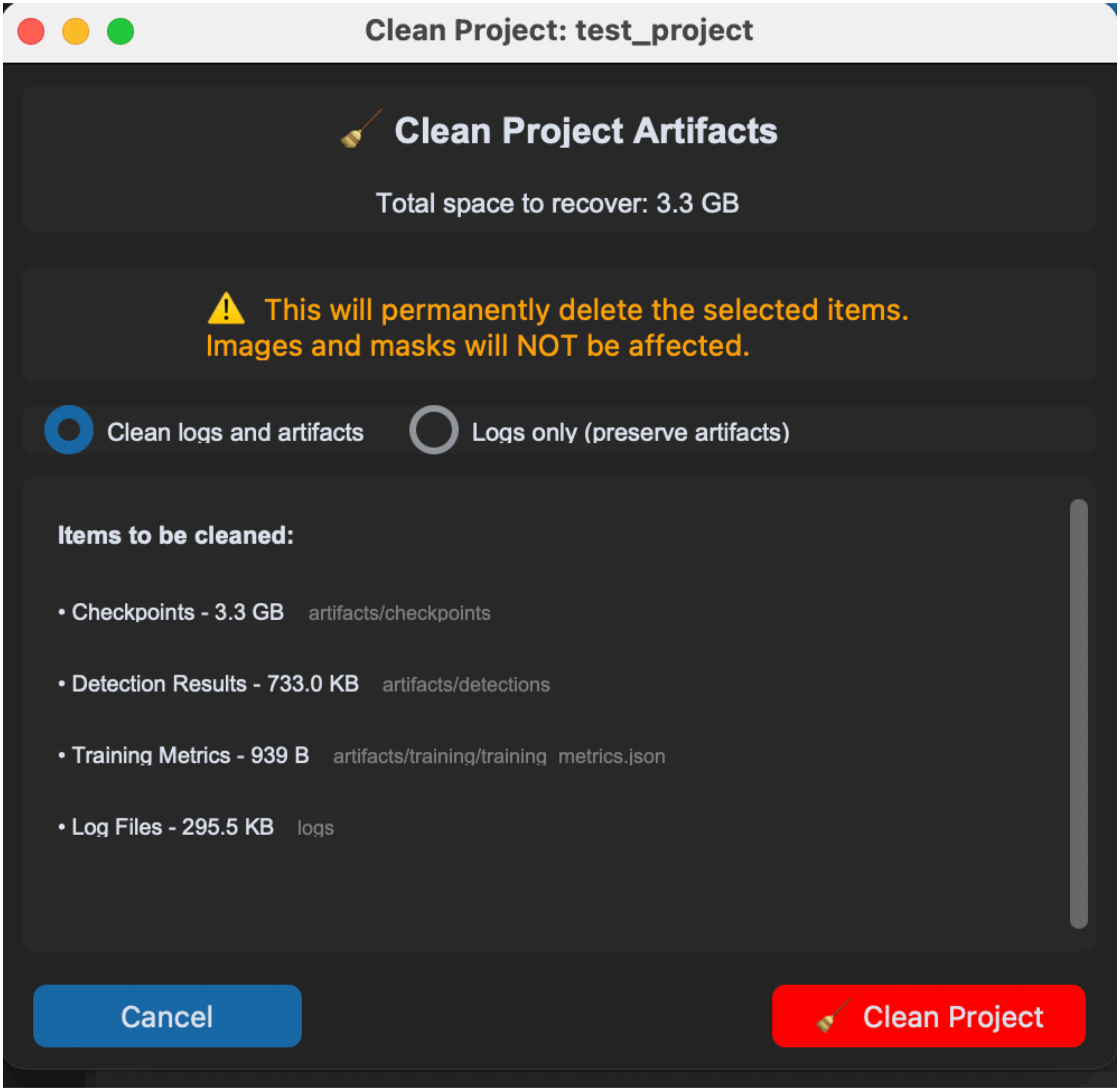


1.
2.
3.
4.

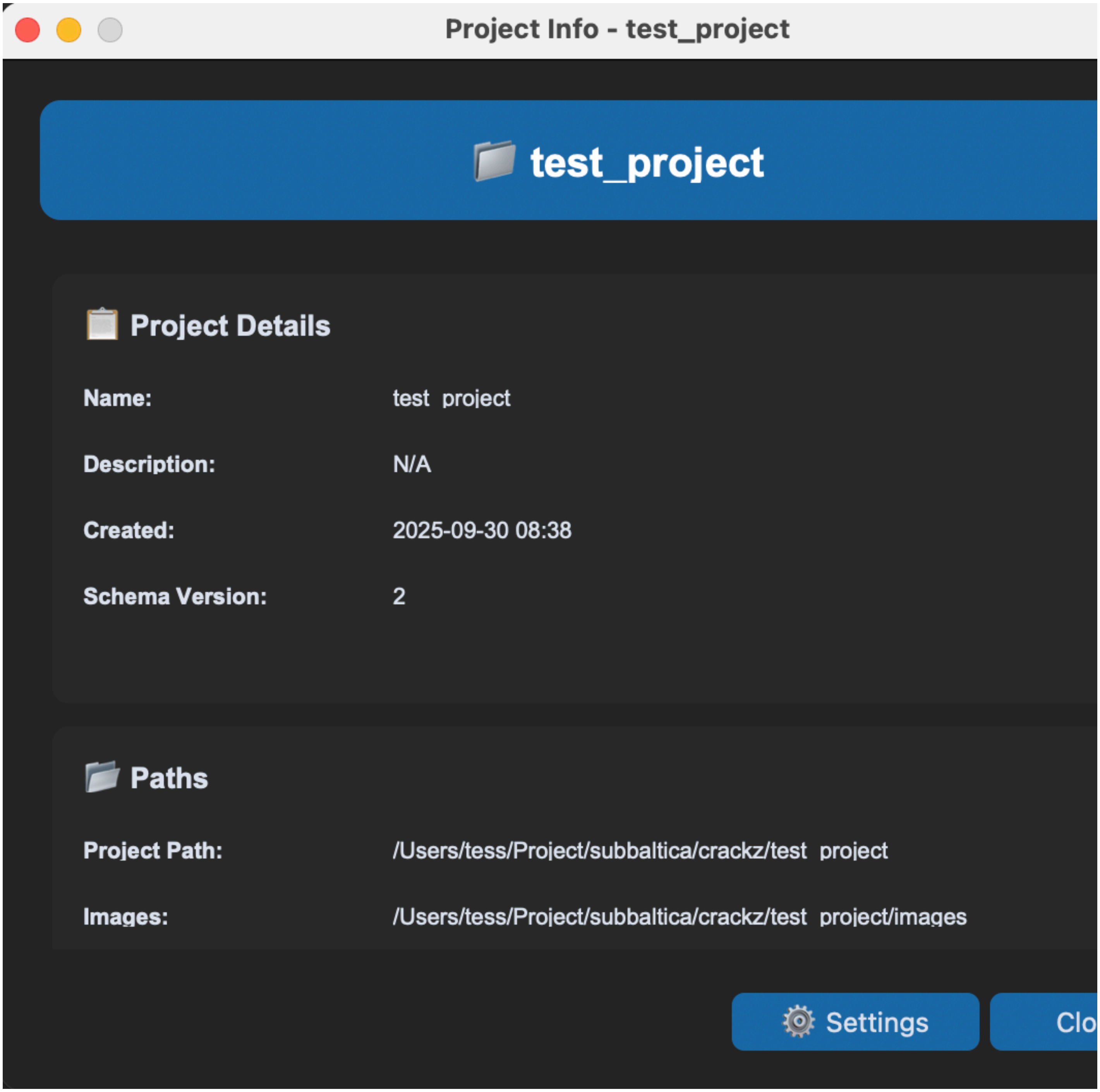
5.
6.

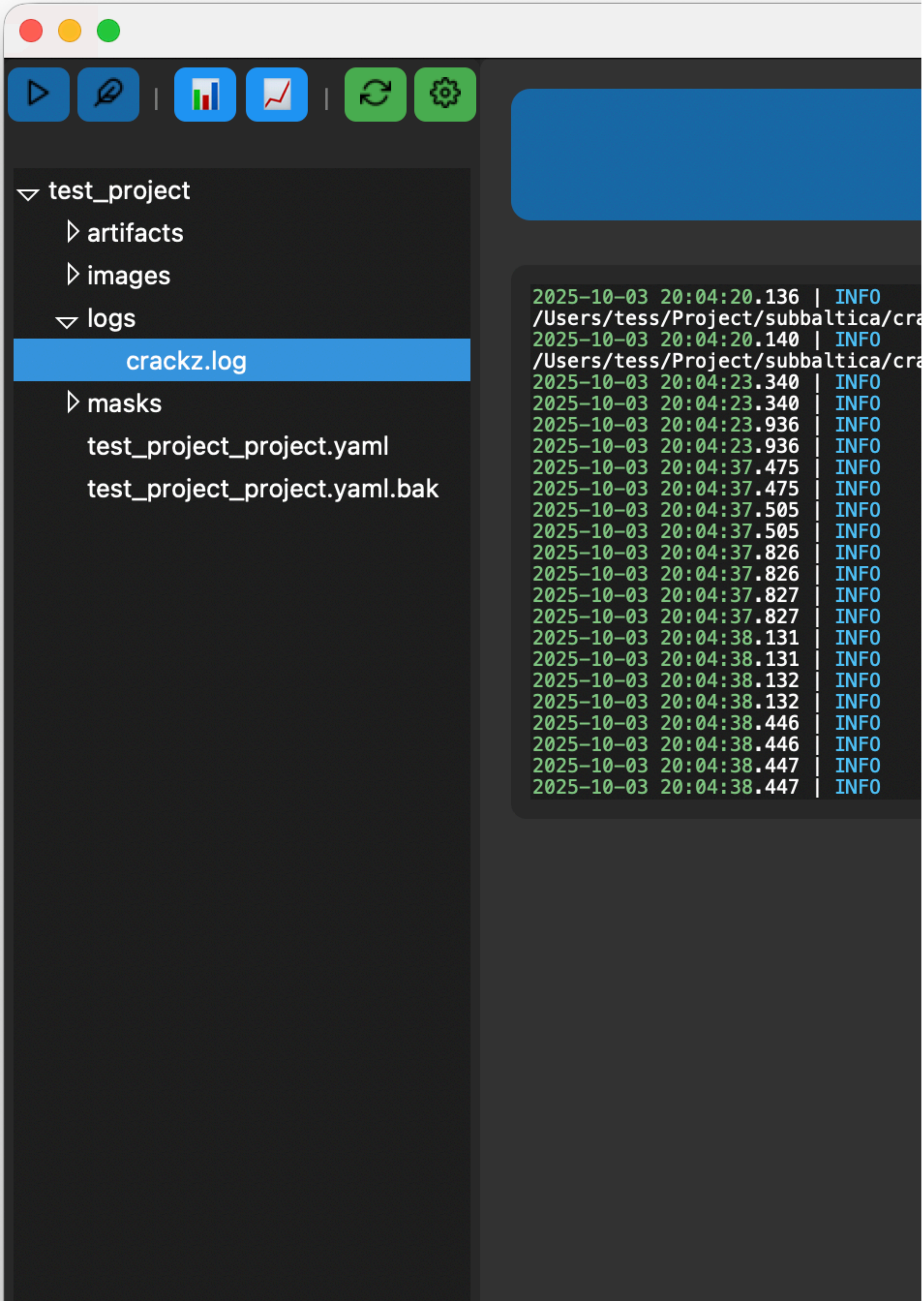

test_project
artifa
imag
logs
mask
test_
test_
Clean Project...
Open in File Explorer
Copy Path
Refresh
Getting Started
Open an existing project or create a r
Open Project
About
Crackz is an AI-powered image analy
• Train segmentation models on your
• Run detection on new images
• View results with detailed metrics
• Manage projects with ease

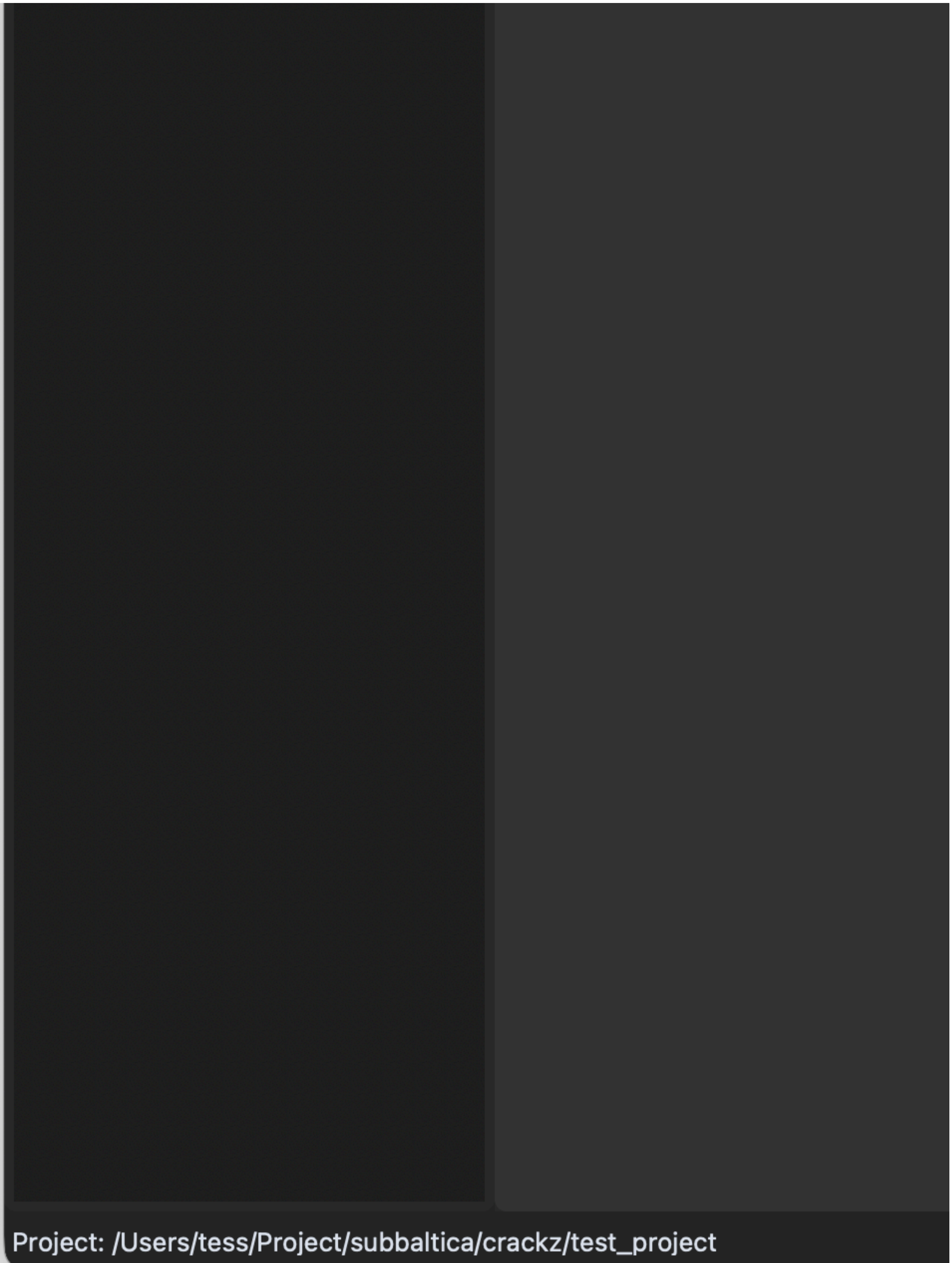

System Information
Operating System: Darwin 25.0.0
Python: 3.13.7
Recommended: 'mps' - Apple Silicon
Note: Device can be selected in project s
Project: /Users/tess/Project/subbaltica/crackz/test_project


Clean Project: test_project
Clean Project Artifacts
Total space to recover: 3.3 GB
This will permanently delete the selected items.
Images and masks will NOT be affected.
Clean logs and artifacts
Logs only (preserve artifacts)
Items to be cleaned:
• Checkpoints - 3.3 GB
artifacts/checkpoints
• Detection Results - 733.0 KB
artifacts/detections
• Training Metrics - 939 B
artifacts/training/training_metrics.json
• Log Files - 295.5 KB
logs
Cancel
Clean Project

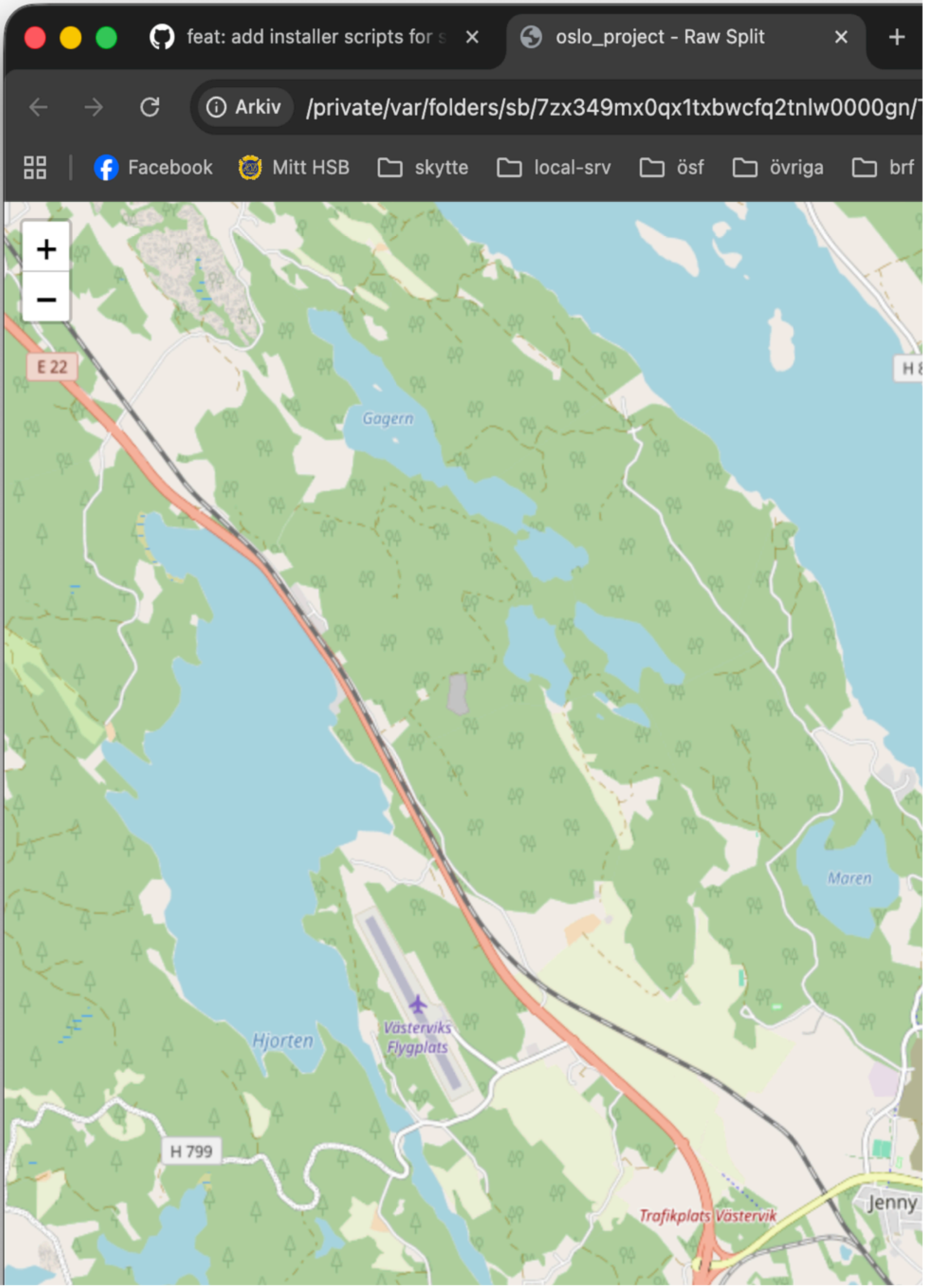
1.
2.


Project Info - test_project
test_project
Project Details
Name:
test_project
Description:
N/A
Created:
2025-09-30 08:38
Schema Version:
2
Paths
Project Path:
/Users/tess/Project/subbaltica/crackz/test_project
Images:
/Users/tess/Project/subbaltica/crackz/test_project/images
Settings
Clo


test_project
artifacts
images
logs
crackz.log
masks
test_project_project.yaml
test_project_project.yaml.bak
2025-10-03 20:04:20.136 | INFO
/Users/tess/Project/subbaltica/cra
2025-10-03 20:04:20.140 | INFO
/Users/tess/Project/subbaltica/cra
2025-10-03 20:04:23.340 | INFO
2025-10-03 20:04:23.340 | INFO
2025-10-03 20:04:23.936 | INFO
2025-10-03 20:04:23.936 | INFO
2025-10-03 20:04:37.475 | INFO
2025-10-03 20:04:37.475 | INFO
2025-10-03 20:04:37.505 | INFO
2025-10-03 20:04:37.505 | INFO
2025-10-03 20:04:37.826 | INFO
2025-10-03 20:04:37.826 | INFO
2025-10-03 20:04:37.827 | INFO
2025-10-03 20:04:37.827 | INFO
2025-10-03 20:04:38.131 | INFO
2025-10-03 20:04:38.131 | INFO
2025-10-03 20:04:38.132 | INFO
2025-10-03 20:04:38.132 | INFO
2025-10-03 20:04:38.446 | INFO
2025-10-03 20:04:38.446 | INFO
2025-10-03 20:04:38.447 | INFO
2025-10-03 20:04:38.447 | INFO


Project: /Users/tess/Project/subbaltica/crackz/test_project

1.
2.
3.


feat: add installer scripts for s
oslo_project - Raw Split
Arkiv
/private/var/folders/sb/7zx349mx0qx1txbwcfq2tnlw0000gn/
Facebook
Mitt HSB
skytte
local-srv
ösf
övriga
brf
E 22
Gagern
Hjorten
Västerviks Flygplats
Maren
H 799
Trafikplats Västervik
Jenny

H 797
Trafikplats Västervik
Mörtvikeskogen
E 22
95
H 807
Verkeback
Gunnebo
53 m
Karlby
42 m
Långmalmen

1.
2.
3.
4.
5.
6.
7.

-
-
-
-
-

§ ~

`§`
`~`

`§` `~` `Cmd/Ctrl+/`

`assets/gui/chat_history_panel.png`

`assets/gui/chat_prompt_templates_panel.png`

`~/.crackz/chat_history.db`

```
# Check installation
python -c "from PySide6.QtWidgets import QApplication; print('PySide6 OK')"

# Launch with debug output
crackz-gui --debug
```

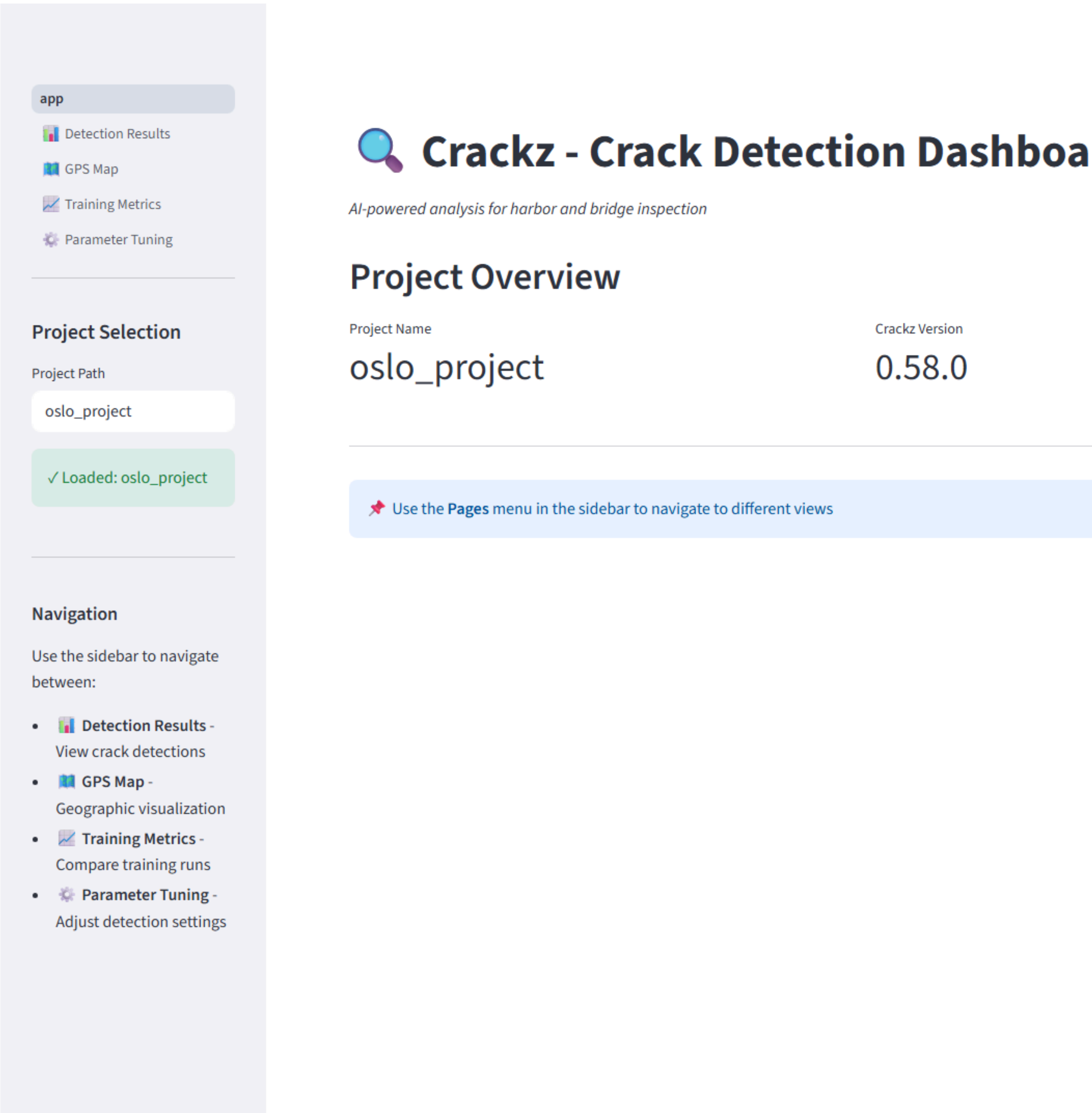
`CRACKZ_LOG_DIR/gui_app.log` <project>/logs/gui_operations.log

```
# Create and configure in GUI
crackz-gui

# Batch process via CLI
crackz detect run --project my-project --batch-size 16
```

```
# Review results in GUI
crackz-gui
```

Dashboard

Streamlit Dashboard



app

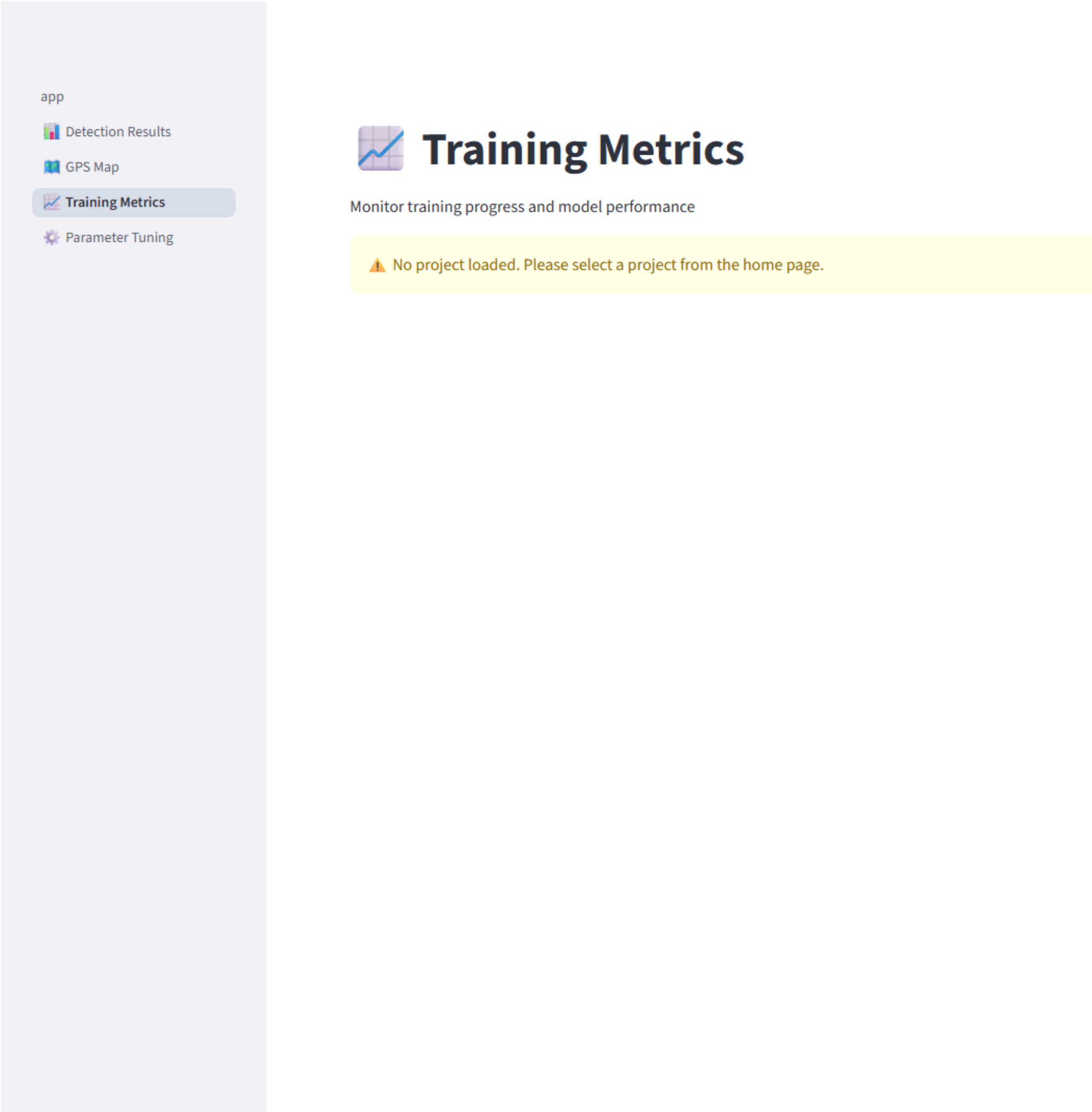
- 📊 Detection Results
- 🗺️ GPS Map
- 📈 Training Metrics
- ⚙️ Parameter Tuning

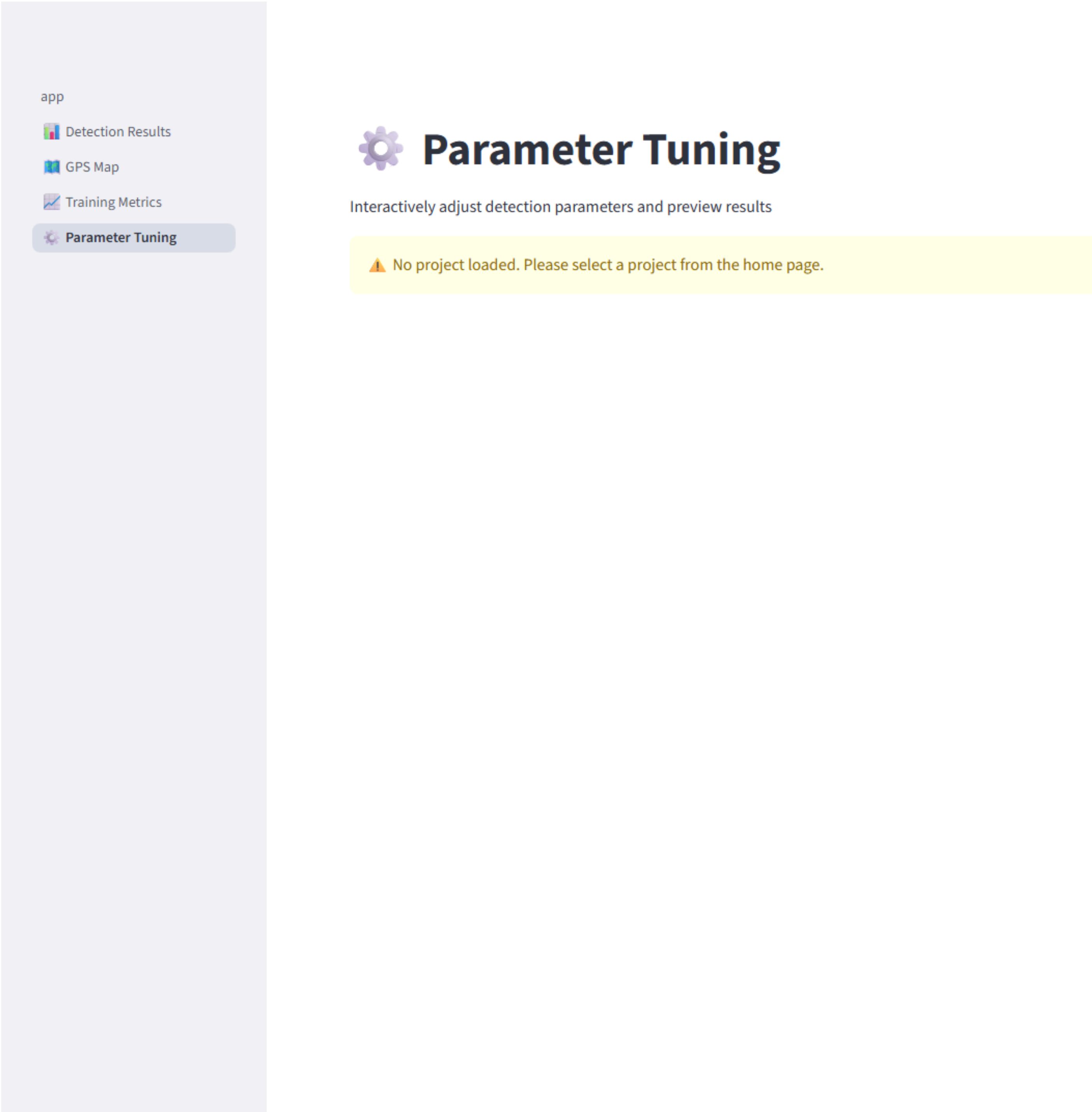
📊 Detection Results

View crack detection results with visualizations and statistics

⚠️ No project loaded. Please select a project from the home page.


app
Detection Results
GPS Map
Training Metrics
Parameter Tuning
GPS Map Visualization
View crack detections on an interactive map
No project loaded. Please select a project from the home page.


app
Detection Results
GPS Map
Training Metrics
Parameter Tuning
Training Metrics
Monitor training progress and model performance
No project loaded. Please select a project from the home page.


app
Detection Results
GPS Map
Training Metrics
Parameter Tuning
Parameter Tuning
Interactively adjust detection parameters and preview results
No project loaded. Please select a project from the home page.

```
# Launch with a specific project
crackz dashboard --project my-project

# Launch with custom port
crackz dashboard --project my-project --port 8502
```

1. `crackz-gui`
2.
3.
4.
5.
6.

```
streamlit run core/src/crackz/dashboard/app.py -- --project my-project
```

```
from crackz.dashboard import launch_dashboard

launch_dashboard(project_name="my-project", port=8501)
```

`crackz geo extract-gps --project <name>`

`artifacts/training/training_metrics.json`

`crackz train --project <name>`

```
core/src/crackz/dashboard/
├── __init__.py          # Launch function
├── app.py              # Main dashboard app
├── pages/              # Dashboard pages
│   ├── 1_□_Detection_Results.py
│   ├── 2_□_GPS_Map.py
│   ├── 3_□_Training_Metrics.py
│   └── 4_⚙_Parameter_Tuning.py
└── utils/              # Utility modules
    ├── data_loader.py  # Data loading functions
    ├── image_utils.py  # Image processing utilities
    ├── map_utils.py    # GPS mapping utilities
    └── viz_utils.py    # Visualization functions
```

	`artifacts/detections/visualizations/`
	`artifacts/detections/detection_results.json`
	`artifacts/training/training_metrics.json`
	`data/{test,val,train,raw}/`

```
crackz dashboard --project my-project --port 8080
```

`.streamlit/config.toml`

```
[server]
port = 8501
headless = true

[theme]
primaryColor = "#FF4B4B"
backgroundColor = "#FFFFFF"
secondaryBackgroundColor = "#F0F2F6"
```

```
textColor = "#262730"
font = "sans serif"
```

- `streamlit>=1.30.0`
- `streamlit-folium>=0.15.0`
- `folium>=0.15.0`
- `plotly>=5.18.0`
- `pandas>=2.0.0`

1.
   ```
   crackz detect --project my-project
   ```
2.
3.

1.
   ```
   crackz geo extract-gps --project my-project
   ```
2.
3.

1.
2.
3.

1.
2.
3.

`uv sync` `pip install streamlit` `--port 8502` `crackz detect --project <name>` `artifacts/detections/visualizations/`

`crackz geo extract-gps`

`crackz train --project <name>` `artifacts/training/training_metrics.json`

1.
2.
3.
4.

```
# 1. Initialize project
crackz init my-project

# 2. Import images
crackz import-images --source ./photos --project my-project

# 3. Train model
crackz train --project my-project

# 4. Run detection
crackz detect --project my-project

# 5. Launch dashboard to view results
crackz dashboard --project my-project
```

```
from crackz.dashboard import launch_dashboard

# Launch dashboard programmatically
launch_dashboard(
    project_name="my-project",  # Optional: project to load
    port=8501                   # Optional: server port
)
```

```
# Show help
crackz dashboard --help

# Launch with project
crackz dashboard --project my-project

# Launch with custom port
crackz dashboard --project my-project --port 8080

# Launch without initial project
crackz dashboard
```

```
# 1. Create and setup project
crackz init harbor-inspection
crackz import-images --source ./drone-photos --project harbor-inspection

# 2. Train model
crackz train --project harbor-inspection --epochs 50

# 3. View training progress
crackz dashboard --project harbor-inspection
# Navigate to "Training Metrics" page to review

# 4. Run detection on test set
crackz detect --project harbor-inspection --split test

# 5. Analyze results
crackz dashboard --project harbor-inspection
# Navigate to "Detection Results" page
# Navigate to "GPS Map" page to see locations

# 6. Tune parameters if needed
# Use "Parameter Tuning" page to optimize thresholds

# 7. Re-run with optimized parameters
crackz detect --project harbor-inspection --threshold 0.6

# 8. Generate final report
crackz report generate --project harbor-inspection --format pdf
```

1.
2.
3. `dashboard`

AI Chat Assistant

AI Chat Assistant

```
# Install keyring support
uv add keyring

# Or with pip
pip install keyring
```

```
from crackz_gui.state.secure_storage import (
    store_api_key,
    retrieve_api_key,
    is_secure_storage_available,
)
```

```
# Check if keyring is available
if is_secure_storage_available():
    # Store API key securely
    store_api_key("your-api-key-here")

    # Retrieve API key
    api_key = retrieve_api_key()

    # Delete API key
    from crackz_gui.state.secure_storage import delete_api_key
    delete_api_key()
else:
    print("Keyring not available - falling back to plaintext storage")
```

1.
2. `§`
3. `~`
4. `Ctrl+/` `Cmd+/`
5.
6.
7.
8.
    -
    -
9.
    -
    - `sk-ant-...` `sk-or-...`
10.
11.

1.
2.
3.
4.

- `§`
- `~`
- `Ctrl+/` `Cmd+/`

```
You: My detection is finding too many false positives. How can I reduce them?

AI: Based on your project, I recommend:
1. Increase confidence_threshold from 0.25 to 0.4
2. Adjust iou_threshold to 0.5 for stricter overlap filtering
3. Consider retraining with more negative examples

Would you like me to explain how to adjust these in your project?
```

```
You: How many epochs should I train for?

AI: For your dataset with 150 training images:
- Start with 50 epochs as a baseline
- Monitor validation loss - it should decrease steadily
- If validation loss plateaus before 50 epochs, you can stop early
- If still improving at 50, try 75-100 epochs

Your current batch_size of 8 is good for this dataset size.
```

```
You: Analyze my project configuration

AI: Here's what I found in your oslo_project:
□ Good splits: 70/20/10 train/val/test distribution
⚠  Low image count (150 total) - consider data augmentation
□ Reasonable batch_size (8) for dataset
□ Suggestion: Enable data augmentation in training config
□ Suggestion: Consider collecting more diverse training data
```

```
You: How do I export detection results to CSV?

AI: To export detection results:
1. Run detection: `crackz detect run <project>`
2. Results are saved in project/artifacts/detections/
3. Use export command: `crackz export --format csv <project>`

Or in GUI:
1. Click "Run Detection" button
2. Results appear in Detection Results panel
3. Click "Export" → "CSV Format"
```

`claude-sonnet-4-5-20250929` `claude-haiku-4-5-20251001` `claude-opus-4-1-20250805`

`§` `~` `Ctrl+/`

1.
2.
3.
4.
5.
6.
7.

`chat_window.py`

`ai_chat.py`

`chat_settings_dialog.py`

1. `ai_chat.py` `_get_context()`

```
def _get_context(self) -> str:
    if self.app_state is None or self.app_state.project is None:
```

```
        return "No project loaded"

    project = self.app_state.project

    # Add your new context here
    return f"""
    Current project: {project.name}

    Your new context:
    - Additional info: {project.additional_info}
    """
```

2.

```
uv run task tests
```

Configuration

Configuration Reference

`crackz init`

```
CLI flag > ENV var > Project YAML > Code default
```

`crackz init --name demo-project` `demo-project/_project.yaml`
`<name>_project.yaml`

```yaml
project:
  name: demo-project
  description: Demo project
  created: 2025-01-01T12:00:00+00:00
paths:
  project_path: demo-project
  image_path: demo-project/images
  image_mask_path: demo-project/masks
  artifacts_path: demo-project/artifacts
logging:
  level: INFO
  log_dir: logs
ai:
  model:
    encoder: resnet34
    encoder_weights: imagenet
    device: cpu
    in_channels: 3
    num_classes: 1
    input_size: [512, 512]
    threshold: 0.5
    loss_name: combo
    dice_weight: 0.7
    focal_weight: 0.3
  training:
    batch_size: 8
    num_workers: 4
    epochs: 10
    learning_rate: 0.001
    shuffle: true
  detection:
    threshold: 0.5
    learning_rate: 0.001
    batch_size: 8
    shuffle: true
    output_dir_name: detections
data:
  mask_threshold: 50
schema_version: 2
```

`ProjectConfig`

```python
from pathlib import Path
from crackz.config.project_config import ProjectConfig

pc = ProjectConfig(
    name="demo-project",
    project_path=Path("demo-project"),
    image_path=Path("demo-project/images"),
    image_mask_path=Path("demo-project/masks"),
    artifacts_path=Path("demo-project/artifacts"),
)
pc.save(Path("project_config.yaml"))

# Later
loaded = ProjectConfig.load(Path("project_config.yaml"))
project = loaded.to_project()
```

`Project`

```
from crackz.config.project_config import ProjectConfig
pc = ProjectConfig.from_project(project)
pc.save(Path("backup_config.yaml"))
```

	`ModelConfig`	
	`TrainingConfig`	
	`DetectionConfig`	

`segmentation-models-pytorch`

`efficientnet-b0`				
`efficientnet-b3`				
`efficientnet-b7`				
`resnet18`				
`resnet34`				
`resnet50`				
`resnet101`				
`densenet121`				
`mobilenet_v2`				
`vgg16`				

`imagenet` `ssl` `swsl` `None` `null`

Binary Segmentation (Default)

```
ai:
  model:
    encoder: resnet34
    num_classes: 1          # Single output channel
    threshold: 0.5          # Probability threshold for crack/no-crack
```

Multi-Class Segmentation

```
ai:
  model:
    encoder: efficientnet-b0
    num_classes: 3          # E.g., background=0, crack=1, corrosion=2, spalling=3
    threshold: 0.5          # Applied per-class
```

Grayscale Input

```
ai:
  model:
    encoder: mobilenet_v2
    in_channels: 1          # Grayscale images
```

```yaml
    normalization:
      mean: [0.5]
      std: [0.5]
```

Training from Scratch (No Pretrained Weights)

```yaml
ai:
  model:
    encoder: resnet34
    encoder_weights: null   # Random initialization
```

`TrainingConfig`

`int` `int | str`

`tensorboard`
`artifacts/runtime/tensorboard/` `False`
`getattr(project.ai.training, "tensorboard", False)`

```python
# Safe access pattern inside runtime code
use_tb = getattr(project.ai.training, "tensorboard", False)
if use_tb:
    ... # create logger
```

`getattr(..., default)`

`tests/unit/crackz/ai/test_detection.py::test_tensorboard_logger_disabled_by_default`
`False` `model_fields`

`TrainingConfig`

`min_confidence` `medium_confidence` `high_confidence`

```yaml
ai:
 detection:
   min_confidence: 0.005     # Save detections with ≥0.5% crack area
   medium_confidence: 0.02   # "Probable Crack" at ≥2% crack area
   high_confidence: 0.05     # "Crack Detected" at ≥5% crack area
   min_crack_area_px: 100    # Suppress detections smaller than 100 pixels
```

`data`

`DataConfig`

`mask_threshold`

```
data:
  mask_threshold: 50  # Default - balanced filtering
```

`crackz import-images --masks_src <path>` `project.import_images()` `scripts/import_masks.py`

`crackz migrate` `mask_threshold: 50`

```
crackz export-config --project demo-project --out tuned_config.yaml
```

```
from crackz.config.project_config import ProjectConfig
pcfg = ProjectConfig.from_project(project)
pcfg.save(Path("tuned_config.yaml"))
```

```
CRACKZ_LOG_DIR=run_logs crackz detect run --project demo-project
```

```
$env:CRACKZ_LOG_DIR='run_logs'; crackz detect run --project demo-project
```

`ai.model.device` `resolve_device_and_accelerator` `ai.model.device: cuda` `python -c "import torch; print(torch.cuda.is_available())"`

`tensorboard: true`

```
ai:
  training:
    tensorboard: true
    epochs: 10
    batch_size: 8
```

```
from crackz import Project

project = Project.load(Path("demo-project"))
project.ai.training.tensorboard = True
project.save()
```

```
<artifacts_path>/runtime/tensorboard/
```

```
tensorboard --logdir demo-project/artifacts/runtime/tensorboard
```

```
docker run --rm -v $(pwd)/demo-project:/workspace -p 6006:6006 crackz-cli \
  tensorboard --logdir /workspace/artifacts/runtime/tensorboard
```

	`crackz detect run --project p --cfg cfg.yaml`
	`training.epochs` `batch_size`
	`detection.batch_size`

`<artifacts_path>/models/` `best.ckpt` `last.ckpt`
`epoch={N}-val_iou={score}.ckpt` `epoch=9-val_iou=0.8234.ckpt`

`<artifacts_path>/checkpoints/`

```
ai:
  training:
    save_top_k: 1          # Keep N best checkpoints (by val_iou), default: 1
    save_last: true        # Always save last.ckpt, default: true
    auto_prune_checkpoints: false  # Auto-prune after training, default: false
    checkpoint_keep: 3     # Checkpoints to keep when auto-pruning, default: 3
```

1. `save_top_k: 1, save_last: true`
2. `save_top_k: 3, auto_prune_checkpoints: true, checkpoint_keep: 3`
3. `save_top_k: 1, save_last: false, auto_prune_checkpoints: true, checkpoint_keep: 1`

```
crackz detect prune-checkpoints --project demo-project --keep 3
```

```
docker run --gpus all --rm -v %cd%\project:/workspace crackz-cli:gpu \
  detect prune-checkpoints --project /workspace/demo-project --keep 2
```

```
from crackz.ai.checkpoints import prune_checkpoints
removed = prune_checkpoints(project, keep=2)
```

`best.ckpt` `003-0.8234.ckpt` `last.ckpt`

```
<artifacts_path>/detections/detection_results.json
```

```
from crackz.ai.results import load_detection_results
from crackz.ai.eval import basic_summary
results = load_detection_results(project)
print(basic_summary(results))
```

`crack_severity`

`seed`

```
# project_config.yaml (unified example)
name: demo-project
description: Example project
project_path: demo-project
image_path: demo-project/images
image_mask_path: demo-project/masks
artifacts_path: demo-project/artifacts
logging:
  level: INFO
  log_dir: logs
model:
  encoder: resnet34
  device: cpu
  input_size: [512, 512]
training:
  epochs: 5
  batch_size: 4
  learning_rate: 0.0005
  tensorboard: false  # Set to true to enable TensorBoard logging
detection:
  threshold: 0.55
  batch_size: 4
```

`schema_version`

```
crackz migrate --project ./my_project
```

`schema_version` `schema_version: 1` `tensorboard: false` `.bak`

```
crackz migrate --cfg ./my_project/my_project_project.yaml
```

```
crackz migrate --project ./my_project --dry-run
```

```
crackz migrate --project ./my_project --no-backup
```

`up-to-date` `noop`

```
for d in projects/*; do
  [ -d "$d" ] && crackz migrate --project "$d" --no-backup;
done
```

`ProjectConfig`

Training Configuration

TrainingConfig Contract

`TrainingConfig` `crackz.project.config` `ai.training`

`batch_size`			
`num_workers`			
`epochs`			
`learning_rate`			
`shuffle`			
`loss`			
`metrics`			
`seed`			
`precision`			
`devices`			
`strategy`			
`tensorboard`			`artifacts/training/tensorboard`

`tests/unit/crackz/project/test_training_config_schema.py`
`TrainingConfig`

1.
2.
3.
4. `CURRENT_SCHEMA_VERSION`

1.
2.
3.
4.

`--epochs` `--batch-size`

`tensorboard=True` `<project>/artifacts/training/tensorboard`

`seed`

Data Splits & Import

Data Splits and Import Guide

1.
2.
3.
4.
5.

`images/raw/`

`raw/`

```
crackz import-images --project my-project --src ./photos --type raw --size 512 512
```

`images/train/` `masks/train/`

`img001.jpg` `img001.png`

`images/val/` `masks/val/`

`images/test/` `masks/test/`

`--type raw`

```
crackz import-images \
 --project my-project \
 --src ./labeled-data/images \
 --src-masks ./labeled-data/masks \
```

```
  --split-ratios 0.7 0.15 0.15 \
  --size 512 512
```

```
crackz import-images \
  --project my-project \
  --src ./labeled-data/images \
  --src-masks ./labeled-data/masks \
  --split-ratios 0.7 0.15 0.15 \
  --seed 42 \
  --size 512 512
```

```
# Small dataset: larger validation set
crackz import-images \
  --project my-project \
  --src ./small-dataset/images \
  --src-masks ./small-dataset/masks \
  --split-ratios 0.6 0.25 0.15 \
  --size 512 512

# Large dataset: more for training
crackz import-images \
  --project my-project \
  --src ./large-dataset/images \
  --src-masks ./large-dataset/masks \
  --split-ratios 0.8 0.1 0.1 \
  --size 512 512
```

1.
2.
3.
4.
5.
6.
7.
8.
9.
10.
11.
12.
13.
14.
15.
    -
    -
16.
17.
18.
19.
20.
21.
22.

23.

24.
25.
26.

-
-
-
-

```
crackz init --name harbor-inspection
cd harbor-inspection
```

```
crackz import-images \
  --project . \
  --src ~/datasets/harbor-cracks/images \
  --src-masks ~/datasets/harbor-cracks/masks \
  --split-ratios 0.7 0.15 0.15 \
  --seed 42 \
  --size 512 512
```

`images/train/` `images/val/` `images/test/`
`masks/`

```
# Count images
ls images/train/ | wc -l   # Should show 140
ls images/val/ | wc -l     # Should show 30
ls images/test/ | wc -l    # Should show 30

# Count masks (should match images)
ls masks/train/ | wc -l    # Should show 140
ls masks/val/ | wc -l      # Should show 30
ls masks/test/ | wc -l     # Should show 30
```

```
crackz detect train --epochs 50 --batch-size 8
```

`images/train/` `masks/train/`
`images/val/` `masks/val/`

`artifacts/metrics.json` `artifacts/loss_curve.png` `checkpoints/best.ckpt`

```
crackz detect run --split test
```

`images/test/`
`masks/test/`

`detections/`

```
# Import new unlabeled images
crackz import-images \
  --project . \
  --src ~/new-harbor-photos \
  --type raw \
  --size 512 512

# Run detection
crackz detect run --split raw
```

`detections/`

1.

```
--seed 42  # Same split every time
```

2.

```
cp -r original-data backup/
```

3.

```
# Check if splits are representative
ls images/train/ | wc -l
ls images/val/ | wc -l
```

4.

```
# For 50 images: 0.6, 0.25, 0.15 ensures val gets ~13 images
--split-ratios 0.6 0.25 0.15
```

5.

```
# In project documentation
split_info:
  seed: 42
  ratios: [0.7, 0.15, 0.15]
  date: 2025-10-19
```

1.
2.
3.
4.
5.
6.

-
-
-
-
-
-
-

```
# Wrong: 0.7 + 0.2 + 0.2 = 1.1 ×
# Right: 0.7 + 0.15 + 0.15 = 1.0 
--split-ratios 0.7 0.15 0.15
```

`images/val/`

```
# If you have 10 images and use 0.7/0.15/0.15:
# train=7, val=1, test=2 (works)

# If you have 5 images and use 0.8/0.1/0.1:
# train=4, val=0, test=1 (val is empty!)

# Solution: Use larger val ratio or more images
--split-ratios 0.6 0.25 0.15  # Ensures val gets at least 1
```

```
# Image: images/crack_001.jpg
# Mask must be: masks/crack_001.png (same stem)

# Check your filenames:
ls images/ > image_list.txt
ls masks/ > mask_list.txt
# Compare the stems (filename without extension)
```

```
# Add --seed for reproducible splits
--seed 42
```

```
# Increase validation ratio
--split-ratios 0.65 0.25 0.1  # Gives more to validation
```

```
# Import with smaller size
--size 256 256  # Instead of 512 512

# Or reduce batch size during training
crackz detect train --batch-size 2  # Instead of 8
```

```
# Separate images by class first
mkdir temp/crack temp/no-crack

# Move files to class folders
# ... organize manually or with script ...

# Import each class separately with same ratio
crackz import-images --src temp/crack --split-ratios 0.7 0.15 0.15 --seed 42
crackz import-images --src temp/no-crack --split-ratios 0.7 0.15 0.15 --seed 42
```

```
# Import full dataset to train
crackz import-images --src data --type train --size 512 512

# Manually create folds by moving files between train/val
# Train 5 models with different train/val splits
# Average results for robust performance estimate
```

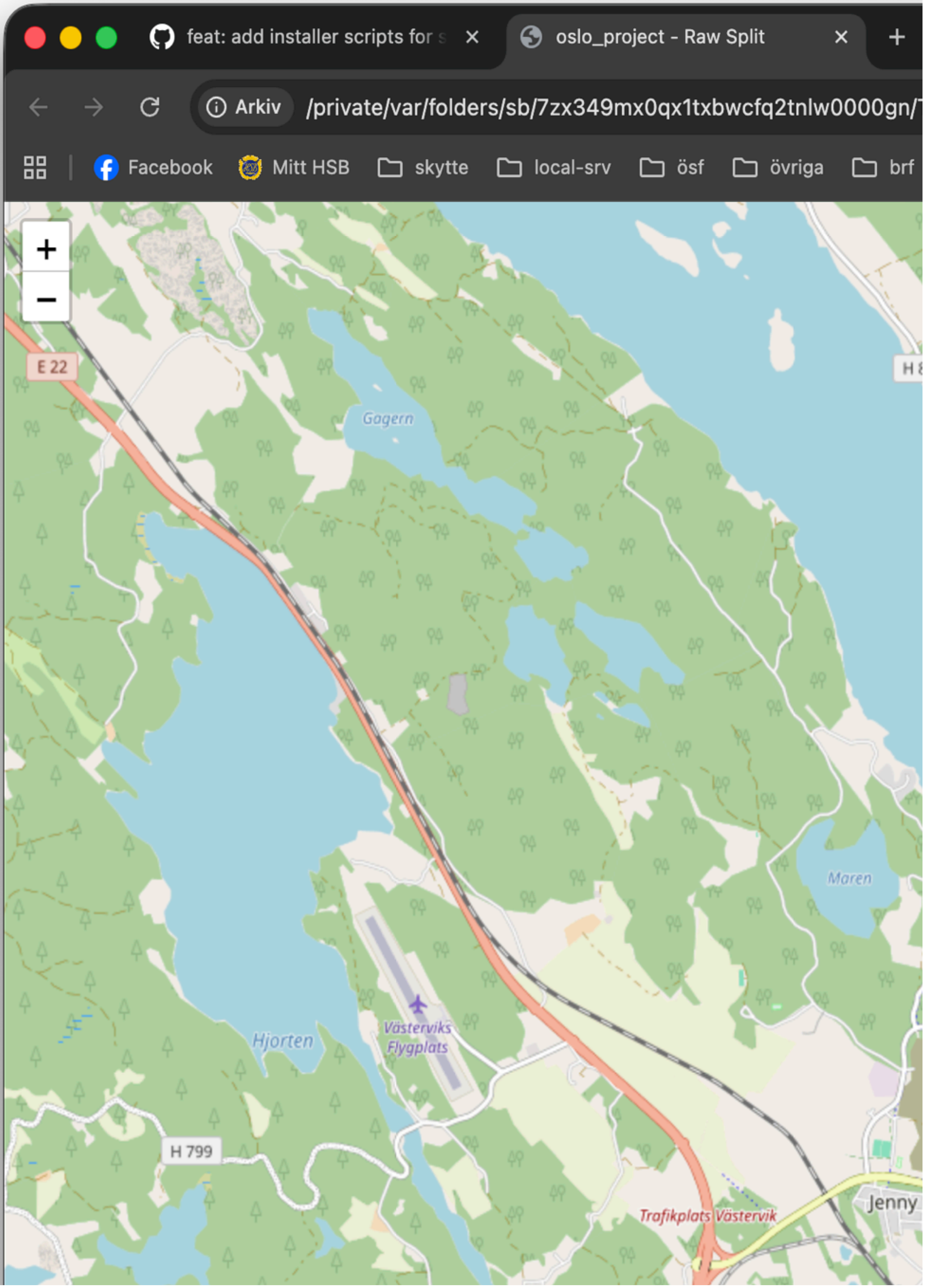
```
# Import new batch with same seed and ratios
crackz import-images \
  --src new-data \
  --split-ratios 0.7 0.15 0.15 \
  --seed 42 \
  --size 512 512

# WARNING: This adds to existing splits
# Consider fresh re-import of all data for clean splits
```

	`images/raw/`				
	`images/train/`				
	`images/val/`				
	`images/test/`				

```
# Import for training (with split)
crackz import-images --src data/images --src-masks data/masks \
  --split-ratios 0.7 0.15 0.15 --seed 42 --size 512 512

# Import for detection (no split)
crackz import-images --src photos --type raw --size 512 512

# Train on train split, validate on val split
crackz detect train --epochs 50 --batch-size 8

# Test on test split
crackz detect run --split test

# Detect on raw split
crackz detect run --split raw
```

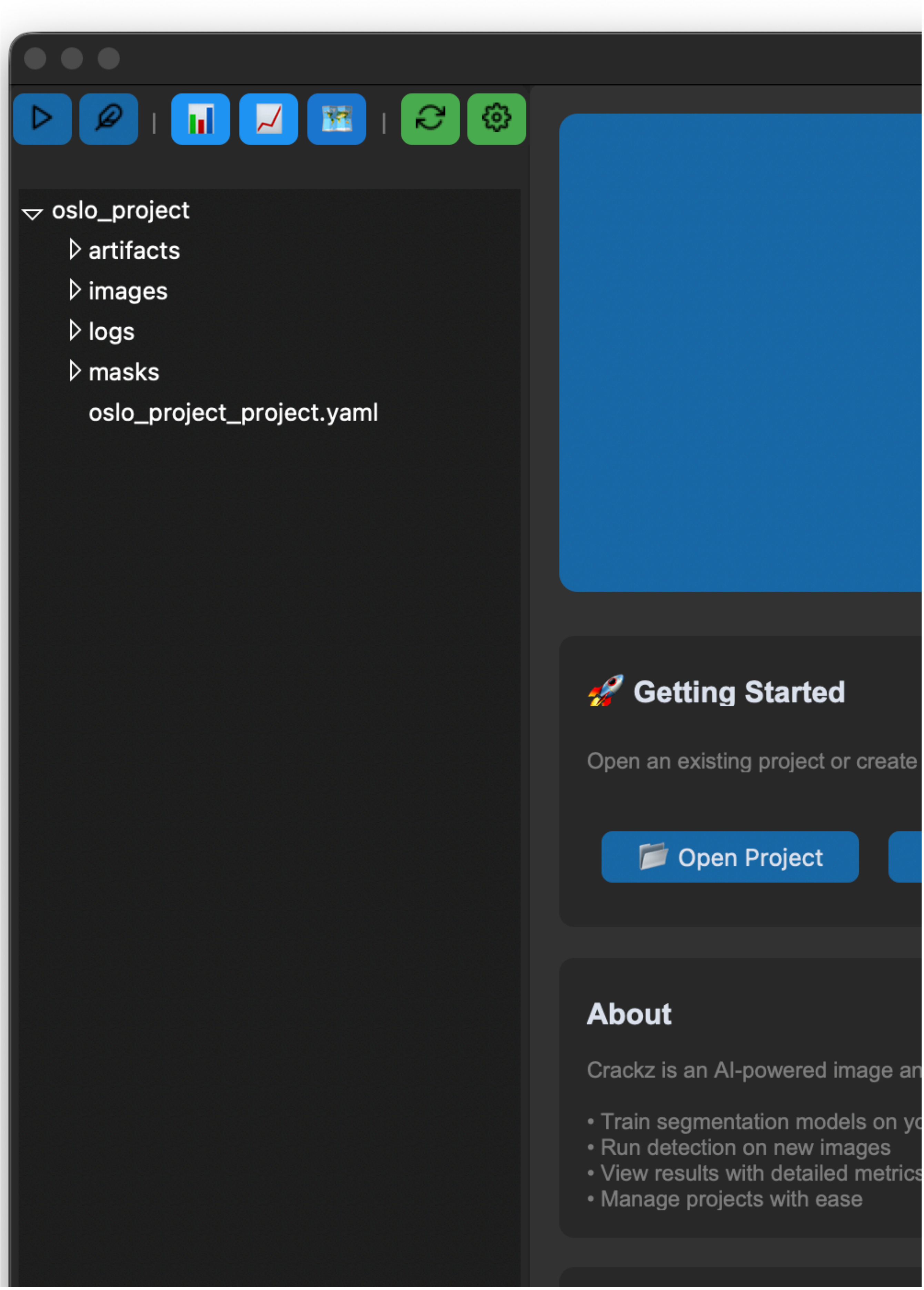
Map Viewer

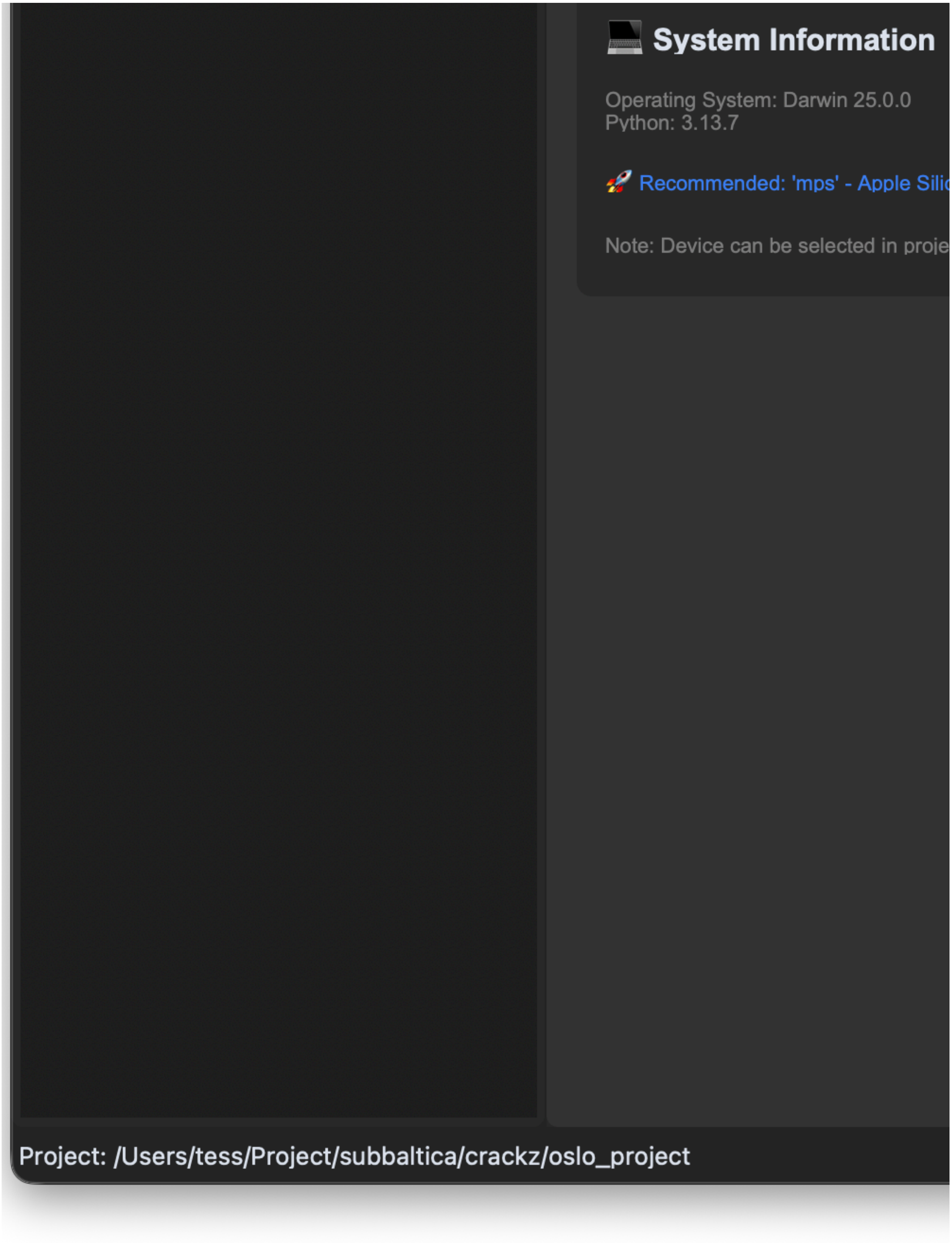
Map Viewer Feature


feat: add installer scripts for s
oslo_project - Raw Split
Arkiv
/private/var/folders/sb/7zx349mx0qx1txbwcfq2tnlw0000gn/
Facebook
Mitt HSB
skytte
local-srv
ösf
övriga
brf
E 22
Gagern
Maren
Hjorten
Västerviks Flygplats
H 799
Trafikplats Västervik
Jenny


H 797
Trafikplats Västervik
Mörtvikeskogen
E 22
H 807
95
Verkeback
Gunnebo
53 m
Karlby
42 m
Långmalmen

-
-
-
-


oslo_project
artifacts
images
logs
masks
oslo_project_project.yaml
Getting Started
Open an existing project or create
Open Project
About
Crackz is an AI-powered image an
• Train segmentation models on yo
• Run detection on new images
• View results with detailed metrics
• Manage projects with ease



Interactive map view opened in web browser with geotagged images displayed as markers

- `gui/src/crackz_gui/components/map_viewer.py`
- `webbrowser.open()`

```
from pathlib import Path
from crackz_gui.qt.views.map_viewer_view import MapViewerDialog
from crackz.project.project import load_project

# Load a project
project = load_project(Path("path/to/project"))

# Note: MapViewerDialog requires a parent GUI app instance and is typically
# used within the Crackz GUI application. For command-line access to GPS data, use:
from crackz.project.geoposition import get_project_geopositions

# Get GPS data for all images in the project
gps_data = get_project_geopositions(project)
for image_path, geo_pos in gps_data.items():
    print(f"{image_path}: {geo_pos.latitude}, {geo_pos.longitude}")
```

`tests/test_data/` `oslo_project/`

- `tests/test_data/images/`
- `oslo_project/images/`

`scripts/add_test_gps_data.py`

```
python scripts/add_test_gps_data.py
```

1.
2. `logs/crackz.log`
3.

- `crackz.project.geoposition.get_project_geopositions()`
- `python scripts/add_test_gps_data.py`

- `gui/src/crackz_gui/components/map_viewer.py`
- `gui/src/crackz_gui/components/browser/toolbar.py`

- `core/src/crackz/project/geoposition.py`
- `scripts/add_test_gps_data.py`

Report Generation

Report Generation

-
-
-
-
-

```
crackz report generate --project demo-project
```

`demo-project_report.docx`

```
crackz report generate --project demo-project --output inspection_report.docx
```

```
crackz report generate --cfg project_config.yaml --output report.docx
```

-
-
-
-

-
-
-

```
ai:
  detection:
    min_confidence: 0.005      # 0.5% - Possible Crack threshold
    medium_confidence: 0.02    # 2% - Probable Crack threshold
    high_confidence: 0.05      # 5% - Crack Detected threshold
```

`detections/`

```
crackz report generate --project harbor_inspection_2024 \
  --output "Harbor_Inspection_Report_2024-Q4.docx"
```

```
# Monthly reports
crackz report generate --project monthly_monitoring \
  --output "reports/2024-11_monitoring.docx"
```

```
crackz report generate --project compliance_check \
  --output "compliance/inspection_$(date +%Y%m%d).docx"
```

```
crackz report generate --project qa_validation \
  --output "internal/qa_review.docx"
```

- `crackz detect run`

```
crackz detect run --project demo-project
```

```
# Check for visualization images (new structure)
ls -la demo-project/artifacts/detections/visualizations/

# Or legacy location (older projects)
ls -la demo-project/artifacts/detections/*.png
```

```
crackz detect run --project demo-project
```

`artifacts/detections/visualizations/`

```
data:
  size: [512, 512]  # Use smaller images
```

`--output`

```
crackz report generate --project harbor_main \
  --output "Harbor_Main_Inspection_2024-11-05.docx"
```

```
for project in project1 project2 project3; do
  crackz report generate --project "$project" \
    --output "reports/${project}_$(date +%Y%m%d).docx"
done
```

```
crackz report generate --project demo-project  # Creates demo-project/demo-project_report.docx
```

```
# Add to project notes
project:
  notes: "Report generated 2024-11-05 with default thresholds"
```

Docker

Docker Usage

```
# Pull from registry
docker pull ghcr.io/anaxiatech-ab/crackz:latest
docker pull ghcr.io/anaxiatech-ab/crackz:slim
docker pull ghcr.io/anaxiatech-ab/crackz:gpu

# Run from registry
docker run --rm ghcr.io/anaxiatech-ab/crackz:latest --help
```

`latest`	`Dockerfile`		
`slim`	`Dockerfile.slim`		
`gpu`	`Dockerfile.gpu`		

```
# Full
docker build -t crackz-cli .
# Slim
docker build -f Dockerfile.slim -t crackz-cli:slim .
# GPU (CUDA 12.8)
docker build -f Dockerfile.gpu -t crackz-cli:gpu .
```

```
docker build --build-arg PYTHON_VERSION=3.13 -t crackz-cli .
```

1. `ghcr.io/astral-sh/uv:bookworm-slim` `uv build --wheel`
2.
3. `python:3.13-slim`
4. `nvidia/cuda:12.8.0-runtime-ubuntu22.04`
5.
6. `--build-arg TORCH_INDEX_URL=...`

- `uv build`
-
-
-

- `nvidia/cuda:12.8.0-runtime-ubuntu22.04`
-
- `git` `libglib2.0-0` `libjpeg-turbo8` `libgomp1` `ca-certificates`
-
-

```
# Default: CUDA 12.8 (recommended, matches PyTorch cu128)
docker build -f Dockerfile.gpu -t crackz-cli:gpu .

# Override for CUDA 12.1
docker build -f Dockerfile.gpu \
  --build-arg TORCH_INDEX_URL=https://download.pytorch.org/whl/cu121 \
  -t crackz-cli:gpu-cu121 .

# Override for CUDA 12.4 (if needed for compatibility)
docker build -f Dockerfile.gpu \
  --build-arg TORCH_INDEX_URL=https://download.pytorch.org/whl/cu124 \
  -t crackz-cli:gpu-cu124 .
```

`TORCH_INDEX_URL`

```
#18 [runtime 7/9] RUN /app/.venv/bin/python3 -c 'import torch; print("Torch:", torch.__version__, "CUDA available:", torch.cuda.is_available())'
#18 0.699 Torch: 2.8.0+cpu CUDA available: False
```

`CUDA available: False`

`linux/amd64`

`/home/app`

```
# Help
docker run --rm crackz-cli --help

# Init project (creates /home/app/demo-project inside container)
mkdir -p project
docker run --rm -v "$(pwd)/project:/home/app" crackz-cli init --name demo-project

# Import images (mount samples to /data)
mkdir -p samples
# (populate samples/ with images and optional masks/ first)
docker run --rm \
  -v "$(pwd)/project:/home/app" \
  -v "$(pwd)/samples:/data" \
  crackz-cli import-images --project /home/app/demo-project --src /data --type raw --size 512 512

# Detection
docker run --rm -v "$(pwd)/project:/home/app" crackz-cli detect run --project /home/app/demo-project
```

```
docker run --gpus all --rm -v "$(pwd)/project:/home/app" crackz-cli:gpu detect run --project /home/app/demo-project
```

`/home/app`		`project/`
`/data`		`samples/`

`/app` `/home/app`

`CRACKZ_LOG_DIR`		`logs`
`PYTHONUNBUFFERED`		`1`

`/home/app/central_logs`

```
docker run --rm -e CRACKZ_LOG_DIR=central_logs -v "$(pwd)/project:/home/app" \
  crackz-cli detect run --project /home/app/demo-project
```

`/home/app`

```
make build        # full image
make build-slim   # slim image
make build-gpu    # gpu image (CUDA 12.8)
make init NAME=demo-project              # creates project under /home/app
make import PROJECT=demo-project DATA_DIR=./samples
make detect PROJECT=demo-project
make detect-gpu PROJECT=demo-project     # GPU detection
make train-gpu PROJECT=demo-project      # GPU training
```

`act`

```
# Install act (macOS)
brew install act

# Test GPU build with make targets
make act-gpu-dryrun          # Dry-run to validate workflow
make act-gpu                 # Actually run the GPU build locally
make act-docker-build        # Test all Docker builds (full, slim, gpu)
```

	`latest`
	`slim`
	`gpu`
	`gpu` `TORCH_INDEX_URL`

		`CRACKZ_LOG_DIR`
		`--gpus all`
		`TORCH_INDEX_URL`

Azure Batch

Azure Batch Deployment

1.
2.
3.
4. `az login`

```
./scripts/azure/azure_batch.sh \
  --batch-account mycrackzaccount \
  --storage-account mycrackzstorage
```

`mycrackzaccount` `mycrackzstorage`

```
# Get storage account key
STORAGE_KEY=$(az storage account keys list \
  --account-name mycrackzstorage \
  --resource-group subbaltica-crackz-rg \
  --query "[0].value" -o tsv)

# Upload project directory
az storage blob upload-batch \
  --account-name mycrackzstorage \
  --account-key "$STORAGE_KEY" \
  --destination crackz-data \
  --source ./my-project
```

```
# Create job
az batch job create \
  --id crackz-detection-job \
  --pool-id crackz-pool

# Add tasks to the job
az batch task create \
  --job-id crackz-detection-job \
  --task-id detect-task-1 \
  --command-line "docker run crackz-cli:latest detect run --project /mnt/batch/tasks/fsmounts/crackz-data/my-project"
```

`--batch-account`		
`--resource-group`		`subbaltica-crackz-rg`
`--location`		`swedencentral`
`--pool-id`		`crackz-pool`
`--vm-size`		`Standard_D2s_v3`
`--node-count`		`2`
`--image`		`crackz-cli:latest`
`--storage-account`		
`--container`		`crackz-data`

`Standard_D2s_v3`				
`Standard_D4s_v3`				
`Standard_D8s_v3`				
`Standard_NC6s_v3`				
`Standard_NC4as_T4_v3`				

```
./scripts/azure/azure_batch.sh \
  --batch-account crackz-batch \
  --pool-id high-performance-pool \
  --vm-size Standard_D8s_v3 \
  --node-count 10 \
  --storage-account crackzstore
```

```
./scripts/azure/azure_batch.sh \
  --batch-account crackz-batch-gpu \
  --pool-id gpu-pool \
  --vm-size Standard_NC4as_T4_v3 \
  --node-count 2 \
  --image crackz-cli:gpu
```

```
# Enable auto-scaling
az batch pool autoscale enable \
  --pool-id crackz-pool \
  --account-name mycrackzaccount \
  --auto-scale-formula '
    startingNumberOfVMs = 2;
    maxNumberofVMs = 10;
    pendingTaskSamplePercent = $PendingTasks.GetSamplePercent(180 * TimeInterval_Second);
    pendingTaskSamples = pendingTaskSamplePercent < 70 ? startingNumberOfVMs : avg($PendingTasks.GetSample(180 * TimeInterval_Second));
    $TargetDedicatedNodes = min(maxNumberofVMs, pendingTaskSamples);
  '
```

```
#!/bin/bash
# Create job
az batch job create --id multi-project-job --pool-id crackz-pool

# Submit tasks for each project
for project in project1 project2 project3; do
  az batch task create \
    --job-id multi-project-job \
    --task-id "detect-$project" \
    --command-line "docker run crackz-cli:latest detect run --project /data/$project"
done
```

```
# Training task
az batch task create \
  --job-id pipeline-job \
  --task-id train \
  --command-line "docker run crackz-cli:latest detect train --project /data/my-project --epochs 50"

# Detection task (depends on training)
az batch task create \
  --job-id pipeline-job \
  --task-id detect \
  --depends-on train \
  --command-line "docker run crackz-cli:latest detect run --project /data/my-project"
```

```
az batch task create \
  --job-id output-job \
  --task-id detect-with-output \
  --command-line "docker run -v /mnt/output:/output crackz-cli:latest detect run --project /data/project" \
  --resource-files '[{"httpUrl":"https://mystorage.blob.core.windows.net/input/data.zip","filePath":"data.zip"}]' \
  --output-files '[{"filePattern":"/mnt/output/**/*","destination":{"container":{"containerUrl":"https://mystorage.blob.core.windows.net/results"}},"uploadOptions":{"uploadCo
```

```
az batch pool show \
  --pool-id crackz-pool \
  --account-name mycrackzaccount \
  --query "{State:allocationState, DedicatedNodes:currentDedicatedNodes}"
```

```
az batch job list \
  --account-name mycrackzaccount \
  -o table
```

```
az batch task list \
  --job-id crackz-detection-job \
  --account-name mycrackzaccount \
  -o table
```

```
az batch task file download \
  --job-id crackz-detection-job \
  --task-id detect-task-1 \
  --file-path stdout.txt \
  --destination ./task-stdout.txt \
  --account-name mycrackzaccount
```

```
# After pool creation, modify to use low-priority nodes
az batch pool resize \
```

```
 --pool-id crackz-pool \
 --account-name mycrackzaccount \
 --target-dedicated-nodes 0 \
 --target-low-priority-nodes 4
```

```
./scripts/azure/azure_batch.sh \
 --batch-account mycrackzaccount \
 --pool-id crackz-pool \
 --destroy
```

	`az batch location quotas show --location <region>`
	`az batch task file download`
	`az batch account login`

```
# Get task details
az batch task show \
  --job-id crackz-detection-job \
  --task-id detect-task-1 \
  --account-name mycrackzaccount

# Download stderr
az batch task file download \
  --job-id crackz-detection-job \
  --task-id detect-task-1 \
  --file-path stderr.txt \
  --destination ./task-stderr.txt \
  --account-name mycrackzaccount
```

```
# Check current quotas
az batch location quotas show --location swedencentral

# Submit quota increase request through Azure Portal
# Support > New support request > Issue type: Service and subscription limits
```

```
name: Azure Batch Processing
on:
  push:
    branches: [main]

jobs:
  batch-process:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5

      - name: Azure Login
        uses: azure/login@v1
        with:
          creds: ${{ secrets.AZURE_CREDENTIALS }}

      - name: Create Batch Pool
        run: |
          ./scripts/azure/azure_batch.sh \
            --batch-account ${{ secrets.BATCH_ACCOUNT }} \
            --storage-account ${{ secrets.STORAGE_ACCOUNT }}

      - name: Submit Batch Job
        run: |
```

```
          az batch job create --id ci-job-${{ github.run_number }} --pool-id crackz-pool
          az batch task create \
            --job-id ci-job-${{ github.run_number }} \
            --task-id detect \
            --command-line "docker run crackz-cli:latest detect run --project /data/project"
```

1.
2.
3.
4.
5.

`taskSlotsPerNode`

```
# Modify pool to allow multiple tasks per node
az batch pool set \
  --pool-id crackz-pool \
  --account-name mycrackzaccount \
  --json-file pool-config.json
```

`pool-config.json`

```
{
  "taskSlotsPerNode": 4
}
```

```
ai:
  detection:
    batch_size: 16  # Adjust based on VM size
```

Azure VM with CUDA

Azure Windows VM with CUDA for Crackz

`azure_vm_cuda.sh`

```
# Deploy default GPU VM
./scripts/azure/azure_vm_cuda.sh

# Deploy with custom configuration
./scripts/azure/azure_vm_cuda.sh \
  --name ml-workstation \
  --size Standard_NC12s_v3 \
  --location westus2

# Connect via RDP and verify
# RDP to the public IP, then run:
nvidia-smi
cd C:\Crackz && .venv\Scripts\Activate.ps1
python -m crackz --version
```

`Standard_NC6s_v3`						
`Standard_NC12s_v3`						
`Standard_NC24s_v3`						

`Standard_NC6s_v2`						
`Standard_NV6`						

```
# Default deployment (V100 GPU, East US)
./scripts/azure/azure_vm_cuda.sh

# Custom name and location
./scripts/azure/azure_vm_cuda.sh --name gpu-crackz --location westeurope

# Specific GPU configuration
./scripts/azure/azure_vm_cuda.sh --size Standard_NC12s_v3 --location northeurope
```

`--name NAME`		`crackz-gpu-vm`
`--location REGION`		`eastus`
`--size SIZE`		`Standard_NC6s_v3`
`--user USERNAME`		`crackzuser`
`--password PASSWORD`		`CrackzGpu2024!`
`--mode MODE`		`standard`
`--data-disk-size SIZE`		`1024`
`--storage-type TYPE`		`Premium_LRS`
`--no-data-disk`		
`--no-cuda`		
`--no-crackz`		
`--enable-nested-virt`		
`--destroy`		
`--help`		

```
# Production inference server
./scripts/azure/azure_vm_cuda.sh \
  --name crackz-prod \
  --size Standard_NC12s_v3 \
  --location westus2 \
  --mode production

# Development environment with large storage
./scripts/azure/azure_vm_cuda.sh \
  --name crackz-dev \
  --size Standard_NV6 \
  --mode development \
  --data-disk-size 2048 \
  --storage-type Premium_LRS

# Budget setup without data disk
./scripts/azure/azure_vm_cuda.sh \
  --name crackz-simple \
  --size Standard_NV6 \
  --no-data-disk

# Manual setup (no auto-install)
./scripts/azure/azure_vm_cuda.sh \
  --name crackz-manual \
  --size Standard_NC6s_v3 \
  --no-cuda \
  --no-crackz

# Docker development (with nested virtualization)
./scripts/azure/azure_vm_cuda.sh \
  --name crackz-docker \
  --size Standard_NC6s_v3 \
  --enable-nested-virt

# Clean up resources
./scripts/azure/azure_vm_cuda.sh --name crackz-gpu-vm --destroy
```

`--data-disk-size SIZE`			
`--storage-type TYPE`			
`--no-data-disk`			

```
C:\ (OS Disk - 127 GB)
├── Windows/
├── Program Files/
├── Users/crackzuser/Desktop/Crackz/   # Crackz installation
└── ...

D:\ (Data Disk - configurable size)
├── Projects/                          # ML project workspace
│   ├── datasets/                     # Training datasets
│   ├── models/                       # Model checkpoints
│   ├── outputs/                      # Detection results
```

```
│   └── exports/                    # Exported models
└── temp/                           # Temporary processing
```


```
# Production: High-performance storage
./scripts/azure/azure_vm_cuda.sh \
  --name ml-prod \
  --data-disk-size 2048 \
  --storage-type Premium_LRS

# Development: Cost-effective storage
./scripts/azure/azure_vm_cuda.sh \
  --name ml-dev \
  --data-disk-size 512 \
  --storage-type Standard_LRS

# Simple setup: No separate disk
./scripts/azure/azure_vm_cuda.sh \
  --name ml-simple \
  --no-data-disk
```

1.
2.
3.
4.
5.
6.
7.
8. `C:\Crackz\.venv`
9.
10.
11.
12.
13.
14.
15.
16.

-
-
-
-
-
-
-

```
# 1. Connect via RDP to the VM
# 2. Double-click "File Upload Server" desktop shortcut
# 3. Open browser and go to: http://localhost:8080
# 4. Or access remotely: http://<VM-PUBLIC-IP>:8080
```

```
# From your local computer:
# Windows: \\<VM-PUBLIC-IP>\CrackzUploads
# Mac/Linux: smb://<VM-PUBLIC-IP>/CrackzUploads

# Credentials: crackzuser / CrackzGpu2024!
```

```
# Using any SFTP client (WinSCP, FileZilla, etc.)
sftp crackzuser@<VM-PUBLIC-IP>
# Upload to: /d/Projects/uploads/images/
```

```
D:\Projects\uploads\
├── images/          # Individual images for analysis
└── batch/           # Batch processing folders
```

```
# Use Windows built-in Remote Desktop
mstsc /v:<PUBLIC_IP>

# Or download RDP file
az vm show -d -g crackz-gpu-rg -n crackz-gpu-vm --query publicIps -o tsv
```

```
# Check GPU status
nvidia-smi

# Test CUDA availability
python -c "import torch; print(f'CUDA available: {torch.cuda.is_available()}')"
python -c "import torch; print(f'GPU count: {torch.cuda.device_count()}')"
```

```
# Activate virtual environment
cd C:\Crackz
.\.venv\Scripts\Activate.ps1

# Check Crackz version
python -m crackz --version

# Test GPU detection
python -m crackz gui  # Should detect CUDA device
```

```
# Activate virtual environment
cd C:\Crackz
.\.venv\Scripts\Activate.ps1

# Option 1: Direct analysis on uploaded images
python -m crackz detect run --images "D:\Projects\uploads\images" --device cuda

# Option 2: Create project and import uploaded images
python -m crackz init harbor-inspection
python -m crackz import-images "D:\Projects\uploads\images" harbor-inspection/images/raw
python -m crackz detect run harbor-inspection --device cuda

# Option 3: Batch processing multiple folders
python -m crackz detect run --images "D:\Projects\uploads\batch\*" --device cuda

# View results
explorer "harbor-inspection\artifacts\detection_results"
```

```
# Create test project
python -m crackz init test-project

# Import sample images
python -m crackz import-images C:\path\to\images test-project/images/raw

# Run GPU-accelerated detection
python -m crackz detect run test-project --device cuda
```

-
-
-
- `az vm deallocate`

-
-
-
-

-
-
-
-
-
-
-

`CrackzGpu2024!`

```
# Manually install CUDA
# Download from: https://developer.nvidia.com/cuda-11-8-0-download-archive
# Choose Windows x86_64 local installer
```

```
# Manual installation
cd C:\Crackz
.\.venv\Scripts\Activate.ps1
uv pip install --find-links https://download.pytorch.org/whl/cu118 torch torchvision
# Download wheel from GitHub releases and install
```

```
# Check drivers
nvidia-smi

# Verify CUDA installation
nvcc --version

# Check PyTorch CUDA
python -c "import torch; print(torch.version.cuda)"
```

- `nvidia-smi -l 1`

```
# Windows Event Logs
eventvwr.msc

# CUDA installation logs
Get-Content C:\ProgramData\NVIDIA Corporation\CUDA Installer\log.txt

# Crackz logs
Get-Content $env:TEMP\crackz-*.log
```


```
az vm list-sizes --location eastus --query "[?contains(name, 'NC')]"
```


```
# Use spot instances (60-90% discount)
az vm create --priority Spot --max-price -1 ...

# Auto-shutdown schedule
az vm auto-shutdown --name crackz-gpu-vm --resource-group crackz-gpu-rg --time 1800

# Deallocate when not in use
az vm deallocate --name crackz-gpu-vm --resource-group crackz-gpu-rg
```

`%TEMP%\crackz-*.log`

VM vs Batch Comparison

Azure Deployment Comparison


```
# Create Windows 11 VM
./scripts/azure/azure_vm.sh \
  --name crackz-dev \
  --size Standard_D4s_v3

# Connect via RDP and use GUI
# IP address displayed after creation
```

```
# Create batch pool with 10 nodes
./scripts/azure/azure_batch.sh \
  --batch-account mybatchaccount \
  --storage-account mystorage \
  --node-count 10 \
  --vm-size Standard_D4s_v3

# Submit 100 detection jobs
for i in {1..100}; do
  az batch task create \
    --job-id detection-job \
    --task-id "task-$i" \
    --command-line "docker run crackz-cli:latest detect run --project /data/project-$i"
done
```

```
# Phase 1: Development on VM
./scripts/azure/azure_vm.sh --name crackz-dev --size Standard_D4s_v3
# RDP in, use GUI to develop and test
# Export config: crackz export-config --project demo --out optimized.yaml

# Phase 2: Production on Batch
./scripts/azure/azure_batch.sh \
  --batch-account prod-batch \
  --pool-id production-pool \
```

```
  --node-count 20 \
  --vm-size Standard_D8s_v3

# Upload optimized config and data to storage
# Submit batch jobs with optimized parameters
```

	`azure_vm.sh`	`azure_batch.sh`

1.
2.
3.
4.

1.
2.
3.
4.
5.

1.
2.
3.
4.

```
# 1. Develop on VM
./scripts/azure/azure_vm.sh --name dev-vm

# 2. Test configuration
crackz detect train --project my-project --epochs 50
crackz detect run --project my-project

# 3. Export configuration
crackz export-config --project my-project --out batch-config.yaml

# 4. Upload to Azure Storage
az storage blob upload-batch \
  --account-name mystorage \
  --destination crackz-data \
  --source my-project

# 5. Create Batch pool
./scripts/azure/azure_batch.sh \
  --batch-account mybatch \
  --storage-account mystorage \
  --node-count 10

# 6. Submit batch jobs
az batch job create --id production-job --pool-id crackz-pool
# ... add tasks
```

- `scripts/azure/azure_vm.sh --help`

`--size Standard_NC*`

`az batch task file download`

Advanced Performance

Advanced Performance & Reproducibility

`ai.training` `ai.detection` `ProjectConfig`

	`precision`	`ai.training`	
	`devices`	`ai.training`	
	`strategy`	`ai.training`	`ddp`
	`seed`	`ai.training` `ai.detection`	

`16` `16-mixed` `bf16-mixed`
`32`

```
ai:
  training:
    precision: 16-mixed
    batch_size: 16
```

```
crackz detect train --project demo-project
```

`devices` `strategy` `ddp`

```
ai:
  training:
    devices: 2
    strategy: ddp
    batch_size: 8
    precision: bf16-mixed
```

```
crackz detect train --project demo-project
```

```
docker run --gpus all --rm -v %cd%\project:/home/app crackz-cli:gpu \
  detect train --project /home/app/demo-project
```

`ddp`		
`ddp_spawn`		
`auto`		

`seed`

```
ai:
  training:
```

```
    seed: 1234
  detection:
    seed: 1234
```

`PYTHONHASHSEED` `random`

```
import torch
torch.use_deterministic_algorithms(True)
```

```
project:
  name: demo-project
  description: Perf tuned
paths:
  project_path: demo-project
  image_path: demo-project/images
  image_mask_path: demo-project/masks
  artifacts_path: demo-project/artifacts
logging:
  level: INFO
  log_dir: logs
ai:
  model:
    encoder: resnet34
    device: cuda
    input_size: [512, 512]
    threshold: 0.5
  training:
    batch_size: 16
    epochs: 20
    learning_rate: 0.0005
    precision: 16-mixed
    devices: 2
    strategy: ddp
    seed: 4242
  detection:
    batch_size: 8
    seed: 4242
```

`crackz export-config`

```
crackz export-config --project demo-project --out tuned_config.yaml
```

`tuned_config.yaml`

`ai.detection.seed`

`ddp_spawn`

	`torch.use_deterministic_algorithms(True)`

Migration Guide

Project Schema Migration

`schema_version`
`ai.training.tensorboard`

`schema_version`
`CURRENT_SCHEMA_VERSION` `crackz.project.migration`

`schema_version`

	`schema_version` `ai.training.tensorboard`	
	`schema_version: 1` `ai.training.tensorboard: false`	
	`data` `mask_threshold: 1`	
	`project.crackz_version`	

```
schema_version: 3
project:
  name: my_project
  description: My crack detection project
  created: '2025-01-15T10:30:00Z'
  crackz_version: '0.50.0'  # NEW: Tracks Crackz app version
```

`project.crackz_version`
`checkpoints/` `models/` `runtime/`
`detections/visualizations/`

`checkpoints_path` `checkpoints/`
`models/` `runtime/`
`crackz_version`

`data`

```
data:
  mask_threshold: 1
schema_version: 2
```

`data.mask_threshold`

`detect train` `detect run`

1.
2. `Project schema v0 < v1. Migrate now? [y/N]:`
3. `.bak`
4.
5. `CRACKZ_AUTO_MIGRATE=1`
6.

```
crackz migrate --project <path-to-project-dir>
# or
crackz migrate --cfg <path-to-project-yaml>
```

`--dry-run` `--no-backup` `<file>.bak` `--migrate-files`

```
# Using project directory
crackz migrate --project my_project --dry-run
crackz migrate --project my_project

# Using direct YAML file path
crackz migrate --cfg my_project/my_project_project.yaml
```

```
# Preview what files would be moved
crackz migrate --project my_project --migrate-files --dry-run

# Migrate both schema AND files
crackz migrate --project my_project --migrate-files
```

`checkpoints/` `models/` `training/tensorboard/` `runtime/tensorboard/` `training/lightning_logs/` `runtime/lightning_logs/` `detections/*.png` `detections/visualizations/`

`{filename}.v{version}.{timestamp}.bak`

```
myproject_project.v2.20250115_143022.bak
```

`v2` `20250115_143022`

`myproject_project.v2.20250115_143022.bak` `.v2.20250115_143022.bak`

`CRACKZ_AUTO_MIGRATE=1`	

```
from pathlib import Path
from crackz.project.migration import migrate_project_file

result = migrate_project_file(Path("my_project"))  # or path/to_yaml
print(result)
# Example result:
# {
#   'status': 'migrated',
#   'from_version': 2,
#   'to_version': 3,
#   'steps': ['2->3: updated to artifacts structure v3 + added crackz_version tracking'],
#   'path': 'my_project/my_project_project.yaml',
#   'backup_path': 'my_project/my_project_project.v2.20250115_143022.bak',
```

```
#   'dry_run': False
# }
```

`status` `from_version` `to_version` `steps` `path` `backup_path` `dry_run`

1.
2.

`.bak`

`.bak`

Shell Completion

Shell Completion

`crackz`

-
-
-
-

```
crackz --show-completion bash
crackz --show-completion zsh
crackz --show-completion fish
crackz --show-completion powershell
```

`source` `Invoke-Expression`

```
crackz --install-completion SHELL
```

`SHELL` `bash` `zsh` `fish` `powershell`

```
crackz det<TAB>
```

```
crackz detect
```

```
crackz --install-completion bash
# New shell OR
source ~/.bash_completion
```

`bash-completion` `--show-completion`

```
crackz --install-completion zsh
# Then restart shell or:
autoload -U compinit && compinit
```

`$fpath` `fpath`

```
crackz --install-completion fish
# Typically installs into ~/.config/fish/completions/
# Reload:
exec fish
```

```
pwsh -File scripts/setup_completion.ps1
# Or force re-install:
pwsh -File scripts/setup_completion.ps1 -Force
```

`crackz` `crackz --install-completion powershell`

```
crackz --show-completion powershell | Out-String | Invoke-Expression
```

`. $PROFILE`

```
crackz --install-completion powershell
# If the profile wasn't updated, add manually:
Add-Content $PROFILE "crackz --show-completion powershell | Out-String | Invoke-Expression"
. $PROFILE
```

	`crackz --show-completion bash > /tmp/crackz.sh && source /tmp/crackz.sh`
	`source <(crackz --show-completion zsh)`
	`crackz --show-completion fish | source`
	`crackz --show-completion powershell | Out-String | Invoke-Expression`

		`. $PROFILE`
		`uv run crackz --install-completion ...` `uv run crackz --show-completion ...`
	`--` `Force`	`-Force`
		`autoload -U compinit && compinit`
`_completion_loader`		`--show-completion`

`~/.bashrc` `~/.zshrc` `$PROFILE`
`~/.config/fish/completions/`

```
crackz --show-completion bash > .crackz_comp.sh
source .crackz_comp.sh
# run tests that exercise completion
```

-
-

Project Scope

Project Scope

1.
2.
3.
4.
5.

-
-
-
-
-

1.
2.
3.
4.
5.
6.
7.
8.
9.
10.
11.
12.

1.
2.
3.

-

3.
4.
5.

1.
2.
3.
4.
5.

1.
2.
3.
4.
5.

1.
2.
3.
4.
5.
6.

•
•
•
•
•
•

•
•
•
•
•

•
•
•
•
•

•
•
•
•
•

Current Tasks

Current Tasks & Backlog

- `- [x] Task description`

- `crackz.app` `crackz-core` `crackz` `crackz-gui` `crackz-gui` `crackz-gui` `crackz-gui`
- `crackz.app.active_learning`

- `docs/superpowers/plans/2026-04-21-multiclass-defect-segmentation.md` `crackz report export --format docx`

-
-

-
-
-
-
-

-
-
-
-

1.
2.
3.
4.
5.

`docs: update task list`

Architecture

System Architecture

```
crackz/                          # Workspace root
├── pyproject.toml               # Workspace config, dev deps, tool settings (no [project])
├── core/                        # crackz-core package
│   ├── pyproject.toml           # Package metadata and dependencies
│   ├── src/crackz/              # Core library (AI, CLI, config, logging, project)
│   └── tests/                   # Core unit + integration tests
├── gui/                         # crackz-gui package (depends on crackz-core)
│   ├── pyproject.toml           # Package metadata, dependency on crackz-core
│   ├── src/crackz_gui/          # PySide6 Qt GUI application
│   └── tests/                   # GUI unit + integration tests
└── tests/                       # Root-level tests (cross-package, legacy)
```

`core/` `gui/` `core/` `uv sync --dev` `from crackz.ai import ...` `from crackz_gui.qt import ...`

```
┌──────────────────────────────────────────────────────────┐
│  Layer 4 — Presentation / Entry Points                   │
│  crackz.cli · crackz_gui · crackz.dashboard              │
├──────────────────────────────────────────────────────────┤
│  Layer 3 — Application Façade                            │
│  crackz.app                                              │
├──────────────────────────────────────────────────────────┤
│  Layer 2 — Domain / Business Logic                       │
│  crackz.ai · crackz.project · crackz.report              │
├──────────────────────────────────────────────────────────┤
│  Layer 1 — Infrastructure / Support                      │
│  crackz.config · crackz.log · crackz.tracing · crackz.util │
└──────────────────────────────────────────────────────────┘
```

```
graph TD
  subgraph L4["Layer 4 — Presentation"]
    CLI["crackz.cli"]
    GUI["crackz_gui"]
    DASH["crackz.dashboard"]
  end
  subgraph L3["Layer 3 — Application Façade"]
    APP["crackz.app"]
  end
  subgraph L2["Layer 2 — Domain"]
    AI["crackz.ai"]
    PROJECT["crackz.project"]
    REPORT["crackz.report"]
  end
  subgraph L1["Layer 1 — Infrastructure"]
    CONFIG["crackz.config"]
    LOG["crackz.log"]
    TRACING["crackz.tracing"]
    UTIL["crackz.util"]
  end

  CLI --> APP
  GUI --> APP
  DASH --> APP
  APP --> AI
  APP --> PROJECT
  APP --> REPORT
  AI --> CONFIG
  AI --> LOG
  AI --> UTIL
  PROJECT --> CONFIG
  PROJECT --> LOG
  PROJECT --> UTIL
  REPORT --> CONFIG
  REPORT --> LOG
  CONFIG --> LOG
  CONFIG --> UTIL
```

	`crackz.cli`	`crackz_gui` `crackz.dashboard`	
	`crackz_gui`	`crackz.dashboard`	`crackz.cli`
	`crackz.dashboard`		`crackz.cli` `crackz_gui`
	`crackz.app`		
	`crackz.ai`		
	`crackz.project`		
	`crackz.report`		
	`crackz.config`	`crackz.project` `crackz.log` `crackz.util`	`crackz.ai` `crackz.report`
	`crackz.log`		
	`crackz.tracing`	`crackz.log`	
	`crackz.util`	`crackz.log`	

`.importlinter` `scripts/check_architecture.py` `uv run task pre_commit`

`pydantic-settings` `raw/ train/ val/ test/` `CrackzDataset` `Dataset`

`loguru` `artifacts/models/` `artifacts/runtime/`

1.
2.
3.
4. `crackz detect train`
5.
6.
7.
8. `artifacts/models/`
9. `crackz detect run`
10.

`segmentation-models-pytorch`

`project.yaml`

```yaml
ai:
  model:
    encoder: resnet34        # or efficientnet-b0, mobilenet_v2, etc.
    encoder_weights: imagenet # or None for random initialization
```

```
C4Container
  title Crackz — Container View (Implemented)
  Person(user, "User")
  System_Boundary(crackz, "Crackz Local System") {
    Container(cli, "CLI", "Typer", "Command parsing & dispatch")
    Container(gui, "GUI", "PySide6 (Qt 6)", "Desktop inspection & invocation")
    Container(core, "Core Library", "Python Packages", "project, ai, config, logging")
    Container(fs, "Local Filesystem", "images/ + artifacts/", "Images, masks, checkpoints, metrics")
  }
  Rel(user, cli, "Runs commands")
  Rel(user, gui, "Interacts (point & click)")
  Rel(cli, core, "Invokes APIs")
  Rel(gui, core, "Invokes APIs")
  Rel(core, fs, "read/write images, masks, checkpoints, metrics")
```

```
C4Context
  title Crackz — System Context (Local)
  Person(user, "User", "Engineer / Operator")
  System(local, "Crackz Application", "CLI + GUI + Core Library")
  System_Ext(osfs, "Host Filesystem", "Images + Artifacts")
  Rel(user, local, "Invoke CLI / Launch GUI")
  Rel(local, osfs, "read / write")
```

```
C4Component
  title Core Library Components (Implemented)
  Container(core, "core", "Python")
  Component(project, "Project & Paths", "pydantic", "Project metadata & path helpers")
  Component(config, "Configuration", "pydantic-settings", "Merge defaults / env / YAML")
  Component(importer, "Image Importer", "Pillow", "Resize, quality, organize splits & masks")
  Component(dataset, "Dataset Layer", "Torch Dataset", "CrackzDataset + DataLoader factory")
  Component(train, "Training Orchestrator", "PyTorch Lightning", "Fit/validate/test + callbacks")
  Component(detect, "Detection Pipeline", "PyTorch", "Batch inference over dataset splits (raw/test)")
  Component(metrics, "Metrics & Checkpoints", "Lightning + JSON", "Save val/test metrics & best.ckpt/last.ckpt")
  Component(perf, "Performance Utilities", "Seeding", "Seed + trainer overrides")
  Component(logging, "Logging", "loguru", "Structured log sink & setup")
  Component(evaluation, "Evaluation Framework", "Metrics", "IoU, precision, recall, F1, accuracy")
  Component(reporter, "Report Generator", "python-docx", "Branded Word reports")
  Component(viz, "Visualization", "Matplotlib", "Detection overlays (3-panel)")
  Component(pipeline_orch, "Pipeline Orchestrator", "Workflow", "Train→detect + cancellation")
  Component(device_mgmt, "Device Management", "Abstraction", "CPU/GPU/MPS selection")
  Component(severity, "Severity Classifier", "Categorization", "3-tier crack severity")
  Component(suggester, "Mask Suggester", "Semi-automation", "Model-assisted annotation")
  Component(perf_est, "Performance Estimator", "RuntimeTracker", "Time estimates")
  Component(async_cb, "Async Callbacks", "Janus", "Sync/async bridging")
  Component(tracing_mod, "Tracing", "OpenTelemetry", "AI API observability")
  Component(ai_chat_svc, "AI Chat", "Anthropic", "User assistance")
  Rel(importer, project, "Uses path layout")
  Rel(dataset, project, "Resolves split directories")
  Rel(train, dataset, "Consumes (train/val/test) loaders")
  Rel(detect, dataset, "Consumes detection loader (raw or test)")
```

```
 Rel(train, metrics, "Saves checkpoints + metrics")
 Rel(detect, metrics, "Writes detection results JSON/PNGs")
 Rel(project, config, "Initializes from effective settings")
 Rel(train, perf, "Reads overrides / seed")
 Rel(detect, perf, "Sets seed (optional)")
 Rel(dataset, logging, "Reports warnings (missing masks / size mismatch)")
 Rel(detect, evaluation, "Evaluates performance")
 Rel(evaluation, metrics, "Calculates metrics")
 Rel(detect, viz, "Visualizes results")
 Rel(detect, reporter, "Generates reports")
 Rel(train, pipeline_orch, "Orchestrated by")
 Rel(detect, pipeline_orch, "Orchestrated by")
 Rel(train, device_mgmt, "Selects device")
 Rel(detect, device_mgmt, "Selects device")
 Rel(detect, severity, "Categorizes cracks")
 Rel(suggester, detect, "Uses model")
 Rel(train, perf_est, "Tracks runtime")
 Rel(detect, perf_est, "Tracks runtime")
 Rel(train, async_cb, "Reports progress")
 Rel(detect, async_cb, "Reports progress")
 Rel(ai_chat_svc, tracing_mod, "Optionally traced")
```

```
flowchart LR
 A["Test Dataset"] --> B["Evaluator"]
 B --> C["Metrics Calculator"]
 C --> D["Evaluation Results"]
 D --> E["JSON/CSV Reporter"]
 D --> F["Report Generator"]
 F --> G["Branded Word Document"]
 I["Detection Results"] --> J["Visualization"]
 J --> K["Detection Overlays"]
 I --> F
```

```
flowchart TB
 A["Training Thread (sync)"] --> B["CallbackDispatcher.sync_q"]
 B --> C["Janus Queue"]
 C --> D["CallbackDispatcher.async_q"]
 D --> E["GUI Event Loop (async)"]
 E --> F["UI Update"]
```

```
flowchart LR
 A["Import Command / GUI Action"] --> B["Image Importer"]
 B --> C["Split Directories (images/*, masks/*)"]
 C --> D["Train Command"]
 D --> E["Training Orchestrator<br/>(Trainer.fit)"]
 E --> F["Checkpoints &amp; Metrics<br/>(best.ckpt / last.ckpt / JSON)"]
 F --> G["Detect Command"]
 G --> H["Inference Loop<br/>(sigmoid → threshold)"]
 H --> I["Annotated Output<br/>(PNGs + detection_results.json)"]
```

```
sequenceDiagram
 participant User
 participant CLI as CLI / GUI
 participant Project
 participant Import as ImageImporter
 participant FS as Filesystem
 User->>CLI: import-images --src ./samples --type train
 CLI->>Project: load_project(path|yaml)
 CLI->>Import: iterate source images
 Import->>FS: resize + normalize + write split file
 Import->>FS: write mask (if provided)
 Import-->>CLI: progress callback
 CLI-->>User: summary (count, skipped)
```

```
sequenceDiagram
 participant User
 participant CLI as CLI / GUI
 participant Project
 participant DS as CrackzDataset
 participant Trainer as Lightning Trainer
 participant Model
 participant FS as Filesystem
 User->>CLI: detect train --project P
 CLI->>Project: load_project()
 CLI->>DS: build train/val/test datasets
 DS-->>CLI: DataLoaders
 CLI->>Trainer: fit(model, loaders)
 Trainer->>FS: save best.ckpt / last.ckpt
 Trainer-->>CLI: metrics (val/test)
 CLI-->>User: training summary
 User->>CLI: detect run --project P
 CLI->>Project: load_project()
 CLI->>Model: load checkpoint (best or last)
 CLI->>DS: build raw dataset (or test)
 DS-->>CLI: DataLoader
 CLI->>Model: forward (batched)
 Model-->>CLI: probabilities
 CLI->>CLI: threshold -> mask
 CLI->>FS: write annotated PNG + JSON metrics
 CLI-->>User: detection summary
```

```
flowchart TD
 Defaults[Built-in Defaults] --> Merge
 YAML[Project YAML] --> Merge
 Env[Environment] --> Merge
 Merge --> Effective[Effective Settings]
```

```
flowchart LR
 User([User]) --> CLI["CLI (Typer)"]
 User --> GUI["GUI (PySide6)"]
 CLI --> CORE["Core Library"]
 GUI --> CORE
 CORE --> FS["Filesystem<br/>images / artifacts/models / artifacts/runtime"]
 CORE -.optional.-> GPU[("GPU")]

 subgraph Optional_Docker [Optional Docker]
   IMG["Container Image<br/>(Python 3.13 + deps)"]
 end

 %% Show that container can host CLI/GUI
 IMG --> CLI
 IMG --> GUI
```

	`crackz.project.project`	
	`pydantic-settings`	
	`project.import_images`	
	`crackz.ai.data`	
	`crackz.ai.detection.train`	
	`crackz.ai.detection.detect`	
	`crackz.ai.model`	
	`crackz.ai.checkpoints`	
	`crackz.ai.device`	
	`crackz.util`	
	`crackz.ai.performance`	
	`crackz.log.crackz_logger`	
	`crackz_gui.*`	
	`crackz.ai.evaluation`	
	`crackz.report.generator`	
	`crackz.ai.visualization`	

	`crackz.ai.pipeline`
	`crackz.ai.pipeline.cancellation`
	`crackz.ai.detection.severity`
	`crackz.ai.mask_suggester`
	`crackz.ai.performance.runtime_estimation`
	`crackz.ai.core.async_callbacks`
	`crackz.tracing`
	`crackz_gui.services.ai_chat`	

`train/` `val/` `test/` `raw/`

-
-
-
-
-
-
-
-
-
-
- `core/` `gui/`

Last updated: synchronized with repository implementation on commit timeline of current editing session.

Architecture Guidelines

Architecture Guidelines

```
┌──────────────────────────────────────────┐
│  User Interfaces (CLI, GUI)              │  ← Entry points
├──────────────────────────────────────────┤
│  Business Logic (Commands, Operations)   │  ← Use cases
├──────────────────────────────────────────┤
│  Core Domain (Project, AI, Config)       │  ← Domain models
├──────────────────────────────────────────┤
│  Infrastructure (Logging, Storage)       │  ← Technical concerns
└──────────────────────────────────────────┘
```

```
# ✅ GOOD: Clear separation
# core/src/crackz/cli/detection_cli.py - CLI commands only
# core/src/crackz/ai/detection.py - Detection logic only
# core/src/crackz/project/structure.py - Project layout only

# × BAD: Mixed concerns
# core/src/crackz/everything.py - CLI, detection, project management all in one file
```

```
# □ GOOD: Depend on protocol
from typing import Protocol

class ModelProtocol(Protocol):
    def predict(self, image: torch.Tensor) -> torch.Tensor: ...

def run_detection(model: ModelProtocol, image: torch.Tensor) -> dict:
    """Works with any model implementing the protocol."""
    return {"mask": model.predict(image)}

# × BAD: Depend on concrete class
def run_detection(model: SegmentationModel, image: torch.Tensor) -> dict:
    """Tightly coupled to SegmentationModel."""
    return {"mask": model.predict(image)}
```

```
# □ GOOD: Single responsibility
class ProjectLoader:
    """Loads project configuration from disk."""
    def load(self, path: Path) -> ProjectConfig: ...

class ProjectValidator:
    """Validates project structure and configuration."""
    def validate(self, project: ProjectConfig) -> list[str]: ...

# × BAD: Multiple responsibilities
class ProjectManager:
    """Loads, validates, trains models, exports results."""
    def load(self, path: Path) -> ProjectConfig: ...
    def validate(self, project: ProjectConfig) -> list[str]: ...
    def train_model(self, project: ProjectConfig) -> None: ...
    def export_results(self, project: ProjectConfig) -> None: ...
```

```
# □ GOOD: Extensible via configuration
def create_model(config: ModelConfig) -> nn.Module:
    """Factory method - add new encoders via config."""
    if config.encoder in SUPPORTED_ENCODERS:
        return create_segmentation_model(config)
    raise ValueError(f"Unsupported encoder: {config.encoder}")

# × BAD: Requires code changes for new models
def create_model(encoder_name: str) -> nn.Module:
    if encoder_name == "resnet34":
        return ResNet34Model()
    elif encoder_name == "efficientnet":
        return EfficientNetModel()
    # Must modify this function for each new encoder!
```

```
# □ GOOD: Explicit parameters
def train_model(
    project_path: Path,
    epochs: int,
    batch_size: int,
    learning_rate: float,
) -> dict[str, float]:
    """All parameters explicit."""
    pass
```

```
# × BAD: Implicit global state
GLOBAL_CONFIG = {}  # Modified elsewhere

def train_model():
    """What parameters? Where do they come from?"""
    epochs = GLOBAL_CONFIG["epochs"]  # Implicit dependency
```

```
core/src/crackz/          # Core library (crackz-core package)
├── ai/                   # AI/ML pipeline (domain logic)
│   ├── data.py           # Dataset, data loading
│   ├── models.py         # Model definitions
│   ├── training.py       # Training pipeline
│   └── detection.py      # Inference pipeline
├── cli/                  # CLI interface (entry points)
│   ├── options.py        # Shared CLI options
│   ├── exceptions.py     # CLI error handling
│   ├── project_cli.py
│   └── detection_cli.py
├── config/               # Configuration (domain models)
│   ├── project.py        # ProjectConfig
│   ├── ai.py             # AIConfig
│   └── logging.py        # LogConfig
├── log/                  # Logging (infrastructure)
│   └── setup.py          # Loguru configuration
└── project/              # Project management (domain logic)
    └── structure.py      # Directory layout

gui/src/crackz_gui/       # GUI package (crackz-gui, depends on crackz-core)
├── qt/                   # PySide6 Qt application
│   ├── app_state.py      # QtAppState (reactive state)
│   └── async_bridge.py   # TaskSignals, run_in_background
├── services/             # External service integrations
├── state/                # State management
└── util/                 # GUI utilities
```

```
CLI (core)  ──→  Business Logic  ──→  Domain Models  ──→  Infrastructure
GUI (gui)   ──→  (ai/, project/)     (config/)           (log/)

Examples:
- core: cli/detection_cli.py → ai/detection.py → config/ai.py → log/setup.py □
- gui:  crackz_gui/qt/panels/train.py → crackz.ai.training → crackz.config □
- gui:  crackz-gui depends on crackz-core (declared in gui/pyproject.toml) □
```

```
Domain ──X→ GUI/CLI     # Domain shouldn't know about UI
Infrastructure ──X→ Domain  # Infrastructure shouldn't depend on business logic
GUI ──X→ CLI            # GUI shouldn't import CLI modules

Examples:
- ai/detection.py → cli/options.py ×  # Domain shouldn't import from CLI
- log/setup.py → config/project.py ×  # Infrastructure shouldn't import domain
- crackz_gui → crackz.cli ×  # GUI should use core APIs, not CLI
```

```
# × BAD: Domain imports from CLI
from crackz.cli.options import PROJECT_PATH_OPTION
def train_model(project_path: Path = PROJECT_PATH_OPTION): ...

# □ GOOD: CLI calls domain with explicit parameters
from crackz.cli.options import PROJECT_PATH_OPTION
from crackz.ai.training import train_model

def train_cli(project_path: Path = PROJECT_PATH_OPTION):
    train_model(project_path)  # Domain function has no CLI dependency
```

`core/src/crackz/ai/models.py`

```
def create_segmentation_model(config: ModelConfig) -> nn.Module:
    """Factory for creating segmentation models."""
    import segmentation_models_pytorch as smp

    return smp.Unet(
        encoder_name=config.encoder,
        encoder_weights=config.encoder_weights,
        in_channels=config.in_channels,
        classes=config.num_classes,
    )
```

`core/src/crackz/ai/detection.py`

```
class DetectionStrategy(Protocol):
    def detect(self, images: list[Path]) -> dict[Path, np.ndarray]: ...

class SingleImageDetection:
    def detect(self, images: list[Path]) -> dict[Path, np.ndarray]:
        """Process one at a time."""
        return {img: self._detect_one(img) for img in images}

class BatchDetection:
    def detect(self, images: list[Path]) -> dict[Path, np.ndarray]:
        """Process in batches for efficiency."""
        results = {}
        for batch in chunk_list(images, self.batch_size):
            results.update(self._detect_batch(batch))
        return results
```

`core/src/crackz/config/project.py`

```
class ProjectConfig(BaseModel):
    """Pydantic acts as builder with validation."""
    name: str
    project_path: Path
    training: TrainingConfig = Field(default_factory=TrainingConfig)
    logging: LogConfig = Field(default_factory=LogConfig)

    @classmethod
    def load(cls, path: Path) -> ProjectConfig:
        """Build from YAML with defaults."""
        with open(path) as f:
            data = yaml.safe_load(f)
        return cls(**data)  # Validation + defaults applied
```

`core/src/crackz/cli/*.py`

```
@app.command(name="train")
def train_command(
    project_path: Path = PROJECT_PATH_OPTION,
    epochs: int | None = EPOCHS_OPTION,
) -> None:
    """Each CLI command is a self-contained operation."""
    try:
        # Load configuration
        config = ProjectConfig.load(project_path / f"{project_path.name}_project.yaml")

        # Override with CLI args
        if epochs:
            config.training.epochs = epochs

        # Execute
        train_model(config)
```

```
        typer.secho("□ Training completed", fg=typer.colors.GREEN)
    except Exception as e:
        cli_failure(e, "Training")
```

`core/src/crackz/project/structure.py`

```
class ProjectRepository:
    """Abstracts project storage."""

    def load(self, path: Path) -> ProjectConfig:
        """Load project from filesystem."""
        return ProjectConfig.load(path / f"{path.name}_project.yaml")

    def save(self, project: ProjectConfig) -> None:
        """Save project to filesystem."""
        project.save(project.project_path / f"{project.name}_project.yaml")

    def exists(self, path: Path) -> bool:
        """Check if project exists."""
        return (path / f"{path.name}_project.yaml").exists()
```

`core/src/crackz/ai/core/async_callbacks.py`

`RuntimeError: no running event loop`

`sync_q` `async_q`

```
from crackz.ai.core.async_callbacks import CallbackDispatcher

# In training setup (main thread)
dispatcher = CallbackDispatcher(maxsize=100)

# In PyTorch Lightning callback (training thread)
sync_reporter = dispatcher.create_sync_progress_reporter()
sync_reporter(0.5, "Training epoch 5/10")  # Thread-safe put

# In GUI (async event loop)
async for progress, message in dispatcher.iter_progress():
    update_progress_bar(progress, message)  # Async get
```

`AsyncProgressReporter` `AsyncMetricsReporter`

`core/src/crackz/tracing/`

`--extra tracing`

```
# In core/src/crackz/tracing/setup.py
from crackz.tracing import setup_tracing, is_tracing_enabled

# Automatic instrumentation on setup
if is_tracing_enabled():
    setup_tracing()  # Instruments Anthropic client automatically

# The instrumentation happens automatically via AnthropicInstrumentor
# No manual client instrumentation needed
```

```
# Install tracing dependencies
uv sync --extra tracing

# Enable at runtime
export CRACKZ_TRACING_ENABLED=1
```

`core/src/crackz/ai/evaluation/reporter.py` `core/src/crackz/report/generator.py`

```
from crackz.ai.evaluation.reporter import export_json, export_csv
from crackz.report.generator import ReportGenerator

# Aggregation (format-agnostic)
results = {
    "accuracy": 0.95,
    "precision": 0.92,
    "recall": 0.89,
    "f1_score": 0.90,
    "iou": 0.85
}

# Export to JSON (lightweight)
export_json(
    results,
    output_path=project_path / "evaluation_results.json"
)

# Export to CSV (analysis-friendly)
export_csv(
    results,
    output_path=project_path / "evaluation_results.csv"
)

# Export to DOCX (rich formatting)
generator = ReportGenerator(
    project_path=project_path,
    template_path=template_path
)
generator.generate_report(
    detection_results=results,
    output_path=project_path / "report.docx"
)
```

`core/src/crackz/ai/device/device.py`

`torch.device("cuda")`

```
from crackz.ai.device.device import AIDevice

# Auto-select best available device
device = AIDevice.default()  # Returns CUDA if available, else MPS, else CPU

# Check availability
available = AIDevice.available_devices()  # [AIDevice.CPU, AIDevice.CUDA]

# Test device works
success, message, details = AIDevice.test_device("cuda")
if success:
    print(f"CUDA available: {details['device_name']}")

# Use in model
model = SegmentationModel()
model.to(device.value)  # device.value is "cuda", "mps", "cpu", etc.
```

`core/src/crackz/ai/detection/severity.py`

```python
from crackz.ai.detection.severity import categorize_crack_severity

# Default thresholds: 5% (high), 2% (medium)
severity = categorize_crack_severity(confidence=0.08)  # "Crack Detected"
severity = categorize_crack_severity(confidence=0.03)  # "Suspected Crack"
severity = categorize_crack_severity(confidence=0.01)  # "No Crack"

# Custom thresholds via config
severity = categorize_crack_severity(
    confidence=0.04,
    high_threshold=0.06,    # Raise "Crack Detected" threshold
    medium_threshold=0.03    # Raise "Suspected" threshold
)  # "Suspected Crack" (was "Crack Detected" with defaults)

# Override with has_crack flag (confidence + area thresholds met)
severity = categorize_crack_severity(confidence=0.01, has_crack=True)
# "Crack Detected" (flag indicates combined criteria met)
```

```yaml
# project.yaml
ai:
  detection:
    high_confidence_threshold: 0.05   # 5% crack area
    medium_confidence_threshold: 0.02  # 2% crack area
    min_confidence: 0.005             # 0.5% minimum to save
```

```python
# core/src/crackz/cli/options.py
PROJECT_PATH_OPTION = typer.Option(
    None,
    "--project",
    help="Path to project directory",
)

FORCE_OPTION = typer.Option(
    False,
    "--force",
    help="Force operation without confirmation",
)

# core/src/crackz/cli/some_command.py
from crackz.cli.options import PROJECT_PATH_OPTION, FORCE_OPTION

def my_command(
    project_path: Path | None = PROJECT_PATH_OPTION,
    force: bool = FORCE_OPTION,
) -> None:
    pass
```

`print()` `logger` `typer.echo()`

```
from crackz.log import logger
import typer

# User-facing success/error messages
typer.secho("✓ Detection completed", fg=typer.colors.GREEN)
typer.secho("× Training failed", fg=typer.colors.RED)

# Technical/operational logs
logger.info("Loading {} images from {}", len(images), path)
logger.debug("Model config: {}", config.model_dump())
logger.error("Failed to load checkpoint: {}", error, exc_info=True)
```

`typer.secho()` `logger.*`

`print()` `typer.echo()`

```
from crackz.cli.exceptions import cli_failure

def train_command():
    try:
        # Business logic
        result = train_model(project_path)
        typer.secho("✓ Training completed", fg=typer.colors.GREEN)
    except FileNotFoundError as e:
        cli_failure(e, "File access")
    except RuntimeError as e:
        cli_failure(e, "Training execution")
    except Exception as e:
        cli_failure(e, "Unexpected error")
```

`cli_failure` `logger.error()` `typer.secho()`

```
class AIConfig(BaseModel):
    model: ModelConfig = Field(default_factory=ModelConfig)
    training: TrainingConfig = Field(default_factory=TrainingConfig)

# Usage
config = AIConfig()  # Loads from defaults only
config_from_yaml = AIConfig(**yaml_data)  # Loads from YAML + defaults

# CLI override (doesn't persist to YAML)
if cli_epochs:
    config.training.epochs = cli_epochs  # Runtime only
```

```
logger.info("message")
    ↓
┌─────────────────┐
```

```
│  Loguru Router  │
└────────────────┘
         │
         ├──→ Central Log: /tmp/crackz_logs/app.log (all operations)
         └──→ Project Log: <project>/logs/operations.log (project-specific)
```

```
from loguru import logger

# Configure at startup
logger.add(
    central_log_path,
    format="{time} {level} {message}",
    level="INFO",
)

# Add project-specific handler when project opens
logger.add(
    project_log_path,
    format="{time} {level} {message}",
    level="DEBUG",
    filter=lambda record: record["extra"].get("project") == project_name,
)

# Usage
logger.bind(project=project.name).info("Training started")  # Goes to both logs
```

`frozen=False` `Field(description=...)` `schema_version`

```
class ProjectConfig(BaseModel):
    """Project configuration with validation and versioning."""

    schema_version: int = Field(
        default=2,
        ge=1,
        description="Config schema version for migration support",
    )

    name: str = Field(
        ...,  # Required
        min_length=1,
        max_length=100,
        pattern=r"^[a-zA-Z0-9_-]+$",
        description="Project name (alphanumeric, underscore, hyphen)",
    )

    root: Path = Field(
        ...,  # Required
        description="Project root directory",
    )

    ai: AIConfig = Field(
        default_factory=AIConfig,
        description="AI/ML configuration",
    )

    @field_validator("root")
    @classmethod
    def root_must_exist(cls, v: Path) -> Path:
        """Validate project root exists."""
        if not v.exists():
            raise ValueError(f"Project root does not exist: {v}")
        return v

    @model_validator(mode="after")
    def name_matches_root(self) -> Self:
        """Validate name matches root directory."""
        if self.root.name != self.name:
            raise ValueError(f"Name '{self.name}' doesn't match directory '{self.root.name}'")
        return self
```

`schema_version: int` `crackz migrate <project>`

```
def migrate_v1_to_v2(config_data: dict) -> dict:
    """Migrate project config from v1 to v2."""
    if config_data.get("schema_version", 1) == 1:
        # v2 added 'ai.detection.batch_size'
        if "ai" not in config_data:
            config_data["ai"] = {}
        if "detection" not in config_data["ai"]:
            config_data["ai"]["detection"] = {}
        if "batch_size" not in config_data["ai"]["detection"]:
            config_data["ai"]["detection"]["batch_size"] = 8

        config_data["schema_version"] = 2

    return config_data
```

1.
2.
3.
4.
5.
6.
7.
8.
9.
10.
11.
12.

```
class CrackzError(Exception):
    """Base exception for all Crackz errors."""
    pass

class ConfigurationError(CrackzError):
    """Invalid configuration."""
    pass

class ProjectError(CrackzError):
    """Project-related error."""
    pass

class ModelError(CrackzError):
    """Model loading/execution error."""
    pass
```

```
def command():
    try:
        # Business logic
        result = operation()
        typer.secho("✓ Success", fg=typer.colors.GREEN)

    except FileNotFoundError as e:
        # User error: file doesn't exist
        logger.error("File not found: {}", e)
        typer.secho(f"× File not found: {e}", fg=typer.colors.RED)
        raise typer.Exit(1)

    except ConfigurationError as e:
        # User error: invalid config
        logger.error("Configuration error: {}", e)
        typer.secho(f"× Configuration error: {e}", fg=typer.colors.RED)
        raise typer.Exit(1)

    except MemoryError:
        # System error: out of memory
        logger.error("Out of memory during operation")
        typer.secho("× Out of memory. Try reducing batch size.", fg=typer.colors.RED)
        raise typer.Exit(1)

    except Exception as e:
        # Programmer error: unexpected
        logger.exception("Unexpected error: {}", e)
```

```
        typer.secho("× Unexpected error. Check logs for details.", fg=typer.colors.RED)
        raise typer.Exit(1)
```

```
        /\
       /  \        E2E (Integration)         ← Few, slow, expensive
      /────\       Tests: 5% (25 tests)
     /      \
    /────────\     Integration Tests        ← Some, medium speed
   /          \    Tests: 15% (100 tests)
  /────────────\
 /______________\  Unit Tests               ← Many, fast, cheap
                   Tests: 80% (570 tests)
```

```
core/tests/
├── unit/                   # Fast, isolated, no I/O
│   └── crackz/
│       ├── ai/
│       │   ├── test_models.py      # Model creation, logic
│       │   ├── test_training.py    # Training loop logic
│       │   └── test_detection.py   # Detection logic
│       ├── cli/
│       │   ├── test_options.py     # CLI option validation
│       │   └── test_commands.py    # Command parsing
│       └── config/
│           └── test_project.py     # Config validation
└── integration/            # Slower, system-level, I/O allowed
    ├── crackz/
    │   ├── test_cli_workflows.py   # End-to-end CLI
    │   └── test_training_pipeline.py # Full training
    └── docs/
        └── test_docs_build.py      # Documentation build

gui/tests/
├── unit/                   # GUI unit tests
└── integration/            # GUI integration tests
```

```python
def test_model_creation():
    """Test model factory with different configs."""
    # Arrange
    config = ModelConfig(encoder="resnet34", num_classes=1)

    # Act
    model = create_segmentation_model(config)

    # Assert
    assert isinstance(model, nn.Module)
    assert hasattr(model, "encoder")
    assert hasattr(model, "decoder")
```

```python
@pytest.mark.integration
def test_full_training_workflow(tmp_path):
    """Test complete training workflow."""
    # Arrange
    project = create_test_project(tmp_path, num_images=10)

    # Act
    result = train_model(project, epochs=1)

    # Assert
    assert result["final_loss"] < result["initial_loss"]
    assert (project.project_path / "artifacts" / "models" / "checkpoint.ckpt").exists()
```

```python
@pytest.mark.benchmark
def test_detection_performance(benchmark, sample_images):
    """Ensure detection meets performance targets."""
    model = load_model(TEST_CHECKPOINT)

    result = benchmark(detect_cracks, model, sample_images)

    # Verify performance target: <5s per image on CPU
    assert result.mean < 5.0
```

`core/tests/conftest.py`

```python
@pytest.fixture
def tiny_model():
    """Fast model for unit tests."""
    return create_segmentation_model(ModelConfig(
        encoder="resnet18",
        encoder_weights=None,  # No pre-trained weights
        num_classes=1,
    ))

@pytest.fixture
def sample_project(tmp_path):
    """Sample project with test data."""
    project_dir = tmp_path / "test_project"
    initialize_project(project_dir, name="test_project")

    # Add minimal test data
    (project_dir / "images" / "raw").mkdir(parents=True)
    create_test_image(project_dir / "images" / "raw" / "test.png")

    return project_dir

@pytest.fixture
def mock_model(mocker):
    """Mock model for testing without actual inference."""
    mock = mocker.Mock(spec=nn.Module)
    mock.predict.return_value = torch.zeros(1, 1, 512, 512)
    return mock
```

```python
# × BAD: Load everything into memory
all_images = [Image.open(p) for p in image_paths]  # OOM for large datasets
for img in all_images:
    process(img)

# □ GOOD: Process in batches
for batch_paths in chunk_list(image_paths, batch_size=8):
    batch = [Image.open(p) for p in batch_paths]
    process_batch(batch)
    # batch garbage collected after each iteration
```

```python
def train_with_cleanup():
    try:
        # Training code
        model.train()
        for batch in dataloader:
            loss = model.training_step(batch)
            loss.backward()
            optimizer.step()
    finally:
        # Always clean up GPU memory
        if torch.cuda.is_available():
            torch.cuda.empty_cache()
```

```python
class Project:
    def __init__(self, root: Path):
        self.root = root
        self._config: ProjectConfig | None = None  # Not loaded yet
        self._images: list[Path] | None = None     # Not loaded yet

    @property
    def config(self) -> ProjectConfig:
        """Lazy load configuration."""
        if self._config is None:
            self._config = ProjectConfig.load(
```

```
            self.root / f"{self.root.name}_project.yaml"
        )
    return self._config

@property
def images(self) -> list[Path]:
    """Lazy load image list."""
    if self._images is None:
        self._images = list((self.root / "images").rglob("*.png"))
    return self._images
```

1.
2.
3.
4.
5.

```
from pydantic import BaseModel, Field, validator

class ProjectConfig(BaseModel):
    name: str = Field(
        ...,
        min_length=1,
        max_length=100,
        pattern=r"^[a-zA-Z0-9_-]+$",  # Prevent path traversal
    )

    @validator("name")
    def no_path_traversal(cls, v: str) -> str:
        """Prevent directory traversal attacks."""
        if ".." in v or "/" in v or "\\" in v:
            raise ValueError("Invalid project name: contains path separators")
        return v
```

```
def safe_path_join(base: Path, *parts: str) -> Path:
    """Join paths safely, preventing traversal attacks."""
    result = base
    for part in parts:
        if ".." in part or part.startswith("/") or part.startswith("\\"):
            raise ValueError(f"Unsafe path component: {part}")
        result = result / part

    # Verify result is still under base
    if not result.resolve().is_relative_to(base.resolve()):
        raise ValueError("Path traversal detected")

    return result
```

---

1. `core/src/crackz/cli/my_command.py`
2.

```
from __future__ import annotations

import typer
from crackz.cli.options import PROJECT_PATH_OPTION
from crackz.cli.exceptions import cli_failure
from crackz.log import logger

app = typer.Typer(no_args_is_help=True)

@app.command(name="my-command")
def my_command(
    project_path: Path = PROJECT_PATH_OPTION,
) -> None:
    """Description shown in --help."""
```

```
    try:
        # Business logic
        logger.info("Starting operation")
        result = do_operation(project_path)
        typer.secho("□ Operation completed", fg=typer.colors.GREEN)
    except Exception as e:
        cli_failure(e, "Operation name")
```

3. `core/src/crackz/cli/main.py`
4. `core/tests/unit/crackz/cli/test_my_command.py`
5.

1. `core/src/crackz/config/my_feature.py`

```
from pydantic import BaseModel, Field

class MyFeatureConfig(BaseModel):
    """Configuration for my feature."""

    enabled: bool = Field(
        default=False,
        description="Enable my feature",
    )

    option: str = Field(
        default="default_value",
        description="Feature option",
    )
```

2. `core/src/crackz/config/project.py`

```
class ProjectConfig(BaseModel):
    # ... existing fields ...
    my_feature: MyFeatureConfig = Field(default_factory=MyFeatureConfig)
```

3. `core/src/crackz/project/migrations.py`
4.
5.

1. `core/src/crackz/ai/models/my_model.py`
2.

```
class MyCustomModel(nn.Module):
    def __init__(self, config: ModelConfig):
        super().__init__()
        # Model setup

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # Forward pass
        return x

    def predict(self, x: torch.Tensor) -> torch.Tensor:
        """Inference interface."""
        self.eval()
        with torch.no_grad():
            return self(x)
```

3. `core/src/crackz/ai/models/__init__.py`

```
def create_model(config: ModelConfig) -> nn.Module:
    if config.type == "segmentation":
        return create_segmentation_model(config)
    elif config.type == "my_custom":
        return MyCustomModel(config)
    else:
        raise ValueError(f"Unknown model type: {config.type}")
```

4. `core/src/crackz/config/ai.py`
5.

1. `core/src/crackz/ai/detection/my_strategy.py`
2.

```
class MyDetectionStrategy:
    def __init__(self, model: nn.Module, config: DetectionConfig):
        self.model = model
        self.config = config

    def detect(self, images: list[Path]) -> dict[Path, np.ndarray]:
        """Run detection with my strategy."""
        results = {}
        for image_path in images:
            mask = self._detect_single(image_path)
            results[image_path] = mask
        return results
```

3. `core/src/crackz/ai/detection/__init__.py`

4. `core/src/crackz/config/ai.py`
5.

API Reference

API Reference

```
from pathlib import Path
from crackz.project.project import load_project_from_path
from crackz.ai.detection import detect

project = load_project_from_path(Path("demo-project"))
# Optional: custom progress reporter
progress = lambda p, msg=None: print(f"{p:.0%}", msg or "")

detect(project, progress_reporter=progress)
```

```
from crackz.project.project import load_project_from_path
from crackz.ai.training import train

project = load_project_from_path("demo-project")
train(project)  # writes checkpoints & metrics under project.artifacts/checkpoints
```

```
from pathlib import Path
from crackz.project.project import (
    ProjectMetadata, ProjectPaths, create_project, save_project
)

project = create_project(
    metadata=ProjectMetadata(name="demo-project", description="Example"),
    paths=ProjectPaths(
        project_path=Path("demo-project"),
        image_path=Path("demo-project/images"),
        image_mask_path=Path("demo-project/masks"),
        artifacts_path=Path("demo-project/artifacts"),
    ),
    force=False,
)
save_project(project)
```

```
from crackz.project.project import load_project_from_path
project = load_project_from_path("demo-project")
# Uses same logic as CLI import-images (see project.import_images signature)
project.import_images(
    source_path="./samples",  # directory with raw images
    masks_source_path="./samples/masks",  # optional
    image_type=project.ImagesType.RAW if hasattr(project, 'ImagesType') else None,  # or enum from crackz.ImagesType
    size=(512, 512),
    quality=90,
    overwrite=False,
)
```

`CrackzSettings` `CRACKZ_` `CRACKZ_LOG_LEVEL=DEBUG`

`BaseSettings`

▶ `core/src/crackz/config/crackz_settings.py`

`classmethod`

```
from_conf(conf_file)
```

▶ `core/src/crackz/config/crackz_settings.py`

`CrackzError`

▶ `core/src/crackz/config/exceptions.py`

[`ConfigError`]

▶ `core/src/crackz/config/exceptions.py`

[`ConfigError`]

▶ `core/src/crackz/config/exceptions.py`

`Project`

`project.project` `Project`

`BaseModel`

▶ `core/src/crackz/config/project_config.py`

`classmethod`

```
load(path)
```

▶ `core/src/crackz/config/project_config.py`

```
save(path)
```

▶ `core/src/crackz/config/project_config.py`

```
to_project()
```

`Project` `ProjectConfig`

▶ `core/src/crackz/config/project_config.py`

`classmethod`

```
from_project(project)
```

`ProjectConfig` `Project`

▶ `core/src/crackz/config/project_config.py`

`Project`

`BaseModel`

▶ `core/src/crackz/project/project.py`

`property`

```
name
```

`property`

`root`

property

`config_file`

property

`detections_path`

`Path` `Path`

property

`models_path`

`Path` `Path`

property

`checkpoints_path`

`Path` `Path`

property

`runtime_path`

`Path` `Path`

property

`training_path`

property

`detections_visualizations_path`

`Path` `Path`

property

`detection_results_path`

property

`training_metrics_path`

property

`tensorboard_logs_path`

property

`registry_db_path`

property

`lightning_logs_path`

property

`val_path`

`Path` `Path`

property

`test_path`

`Path` `Path`

property

`train_path`

`Path` `Path`

property

`train_mask_path`

`Path` `Path`

property

`val_mask_path`

`Path` `Path`

property

`test_mask_path`

`Path` `Path`

property

`raw_path`

`Path` `Path`

property

`raw_mask_path`

`Path` `Path`

`property`

```
project_file_path
```

```
map_type_to_dir(image_type)
```

`raw` `train`
`test` `val` `raw`

`Path`
`None`

▶ `core/src/crackz/project/project.py`

```
map_type_to_masks_dir(image_type)
```

`raw` `train`
`test` `val` `raw`

`Path`
`None`

▶ `core/src/crackz/project/project.py`

```
save_project()
```

`model_dump`

`model_dump`

▶ `core/src/crackz/project/project.py`

```
import_image_files(
    files,
    dest_dir,
    size=(512, 512),
    quality=90,
    *,
    overwrite=False,
    progress_cb=None,
)
```

▶ `core/src/crackz/project/project.py`

```
import_images(
    source_path,
    masks_source_path=None,
    image_type=ImagesType.RAW,
```

```
    size=(512, 512),
    quality=90,
    *,
    overwrite=False,
    progress_cb=None,
    split=False,
    train_ratio=0.7,
    val_ratio=0.15,
    test_ratio=0.15,
    random_seed=None,
)
```

-
-
-
-
-

▶ `core/src/crackz/project/project.py`

```
create_project_dir(*, force)
```

`force`

`project_path` `artifacts_path`

▶ `core/src/crackz/project/project.py`

```
clean(*, logs_only=False, keep_best_model=True)
```

▶ `core/src/crackz/project/project.py`

```
create_project_and_save(
    metadata,
    paths,
    ai=default_ai_config,
    logging=default_logging_config,
    *,
    force=False,
)
```

▶ `core/src/crackz/project/project.py`

```
create_project(
    metadata, paths, ai=None, logging=None, *, force=False
)
```

▶ `core/src/crackz/project/project.py`

```
save_project(project)
```

`Project.save_project`

▶ `core/src/crackz/project/project.py`

```
load_project(project_file_path)
```

`Project`

`project_file_path`	`Path`		*required*

[`Project`]	`Project`
[`Project`]	`None`

▶ `core/src/crackz/project/project.py`

```
load_project_from_path(project_path)
```

▶ `core/src/crackz/project/project.py`

```
load_from_project_path_or_cfg(cfg, project_path)
```

▶ `core/src/crackz/project/project.py`

`StrEnum`

`RAW`		
`TRAIN`		
`TEST`		
`VAL`		

▶ `core/src/crackz/project/types.py`

`CrackzError`

▶ `core/src/crackz/project/exceptions.py`

[`ProjectError`]

▶ `core/src/crackz/project/exceptions.py`

[`ProjectError`]

▶ `core/src/crackz/project/exceptions.py`

[`ProjectError`]

▶ `core/src/crackz/project/exceptions.py`

`ProjectError`

▶ `core/src/crackz/project/exceptions.py`

`ProjectError`

▶ `core/src/crackz/project/exceptions.py`

`BaseModel`

▶ `core/src/crackz/ai/detection/results.py`

```
categorize_crack_severity(
    confidence,
    *,
    high_threshold=DEFAULT_HIGH_CONFIDENCE,
    medium_threshold=DEFAULT_MEDIUM_CONFIDENCE,
    has_crack=False,
)
```

`confidence`	`float`		*required*
`high_threshold`	`float`		`DEFAULT_HIGH_CONFIDENCE`
`medium_threshold`	`float`		`DEFAULT_MEDIUM_CONFIDENCE`
`has_crack`	`bool`		`False`

`str`	

```
>>> categorize_crack_severity(0.08)  # 8% crack area
'Crack Detected'
>>> categorize_crack_severity(0.03)  # 3% crack area
'Suspected Crack'
>>> categorize_crack_severity(0.01)  # 1% crack area
'No Crack'
>>> categorize_crack_severity(0.01, has_crack=True)
'Crack Detected'  # has_crack flag indicates both confidence and area thresholds met
```

▶ `core/src/crackz/ai/detection/severity.py`

```
create_tensorboard_logger(project)
```

`project`	[`Project`]		*required*

`TensorBoardLogger | None`	

▶ `core/src/crackz/ai/training/tensorboard.py`

```
train(
   project,
   progress_reporter=None,
   cancellation_token=None,
   metrics_callback=None,
   ckpt_path=None,
)
```

`project`	[`Project`]		*required*
`progress_reporter`	`ProgressReporter | None`		`None`
`cancellation_token`	[`CancellationToken`] `| None`		`None`
`metrics_callback`	`MetricsReporter | None`		`None`
`ckpt_path`	`Path | None`		`None`

`CancellationError`	

▶ `core/src/crackz/ai/training/training_runner.py`

```
detect(
    project,
    progress_reporter=None,
    cancellation_token=None,
    checkpoint_name=None,
    result_callback=None,
    *,
    on_incompatible="original",
)
```

`project`	[`Project`]()		*required*
`progress_reporter`	`ProgressReporter | None`		`None`
`cancellation_token`	[`CancellationToken`]() `| None`		`None`
`checkpoint_name`	`str | None`		`None`
`result_callback`	`Callable[[dict[str, Any]], None] | None`		`None`
`on_incompatible`	`IncompatibleMode`	`"untrained"` `"original"` `"raise"`	`'original'`

`None`	
`None`	`detection_results.json`

`CancellationError`	
`CheckpointIncompatibleError`	*on_incompatible* `"raise"`

▶ `core/src/crackz/ai/detection/detection_runner.py`

```
__getattr__(name)
```

▶ `core/src/crackz/ai/detection/__init__.py`

▶ `core/src/crackz/ai/data/dataset.py`

```
apply(im)
```

▶ `core/src/crackz/ai/data/dataset.py`

`Dataset[Sample]`

▶ `core/src/crackz/ai/data/dataset.py`

```
load_image_with_crackz_image(image_path)
```

`image_path`	`Path`		*required*

`Image`	

[`ImageLoadingError`]()	

▶ `core/src/crackz/ai/data/dataset.py`

```
CrackzDetectionDataset(
    image_dir, transform=None, normalization_config=None
)
```

`image_dir`	`Path | str`		*required*
`transform`	`ImageTransform | None`		`None`
`normalization_config`	`NormalizationConfig | None`		`None`

[`CrackzDataset`]()	

▶ `core/src/crackz/ai/data/dataset.py`

```
create_training_datasets(
    train_path,
    train_mask_path,
    val_path,
    val_mask_path,
    test_path,
    test_mask_path,
)
```

`train_path`	`Path`		*required*
`train_mask_path`	`Path`		*required*
`val_path`	`Path`		*required*
`val_mask_path`	`Path`		*required*
`test_path`	`Path`		*required*
`test_mask_path`	`Path`		*required*

`tuple[`[`CrackzDataset`]()`, `[`CrackzDataset`]()`, `[`CrackzDataset`]()`]`	

▼

▶ `core/src/crackz/ai/data/dataset.py`

```
create_dataset(
    project,
    transforms,
    mask_transforms,
    image_dir=None,
    image_masks_dir=None,
)
```

`project`	[`Project`]()		*required*
`transforms`	`ImageTransform | None`		*required*
`mask_transforms`	`MaskTransform | None`		*required*
`image_dir`	`Path | None`		`None`
`image_masks_dir`	`Path | None`		`None`

[`CrackzDataset`]()	

▶ `core/src/crackz/ai/data/dataset.py`

```
collate(batch)
```

▶ `core/src/crackz/ai/data/dataloader.py`

```
make_crackz_loader(
    dataset, batch_size=32, num_workers=8, *, shuffle=True
)
```

▶ `core/src/crackz/ai/data/dataloader.py`

```
create_training_dataloaders(
    train_ds,
    val_ds,
    test_ds,
    batch_size,
    num_workers,
    *,
    shuffle_train=True,
)
```

`train_ds`	[`CrackzDataset`]()		*required*
`val_ds`	[`CrackzDataset`]()		*required*
`test_ds`	[`CrackzDataset`]()		*required*
`batch_size`	`int`		*required*
`num_workers`	`int`		*required*
`shuffle_train`	`bool`		`True`

`tuple[DataLoader, DataLoader, DataLoader]`	

▶ `core/src/crackz/ai/data/dataloader.py`

`CrackzModel`

`ComboLoss`

`dataclass`

`CrackzModel`

`lr`	`float`	
`backbone`	`str`	
`in_channels`	`int`	
`num_classes`	`int`	
`threshold`	`float`	

▶ `core/src/crackz/ai/model/segmentation/model.py`

`staticmethod`

```
from_project(project, lr)
```

`CrackzConfig` `Project`

▶ `core/src/crackz/ai/model/segmentation/model.py`

`Module`

▶ `core/src/crackz/ai/model/segmentation/model.py`

```
forward(y_pred, y_true)
```

▶ `core/src/crackz/ai/model/segmentation/model.py`

`LightningModule`

▶ `core/src/crackz/ai/model/segmentation/model.py`

```
training_step(batch, _batch_idx)
```

▶ `core/src/crackz/ai/model/segmentation/model.py`

```
validation_step(batch, _batch_idx)
```

▶ `core/src/crackz/ai/model/segmentation/model.py`

```
test_step(batch, _batch_idx)
```

▶ `core/src/crackz/ai/model/segmentation/model.py`

```
on_train_epoch_end()
```

▶ `core/src/crackz/ai/model/segmentation/model.py`

```
on_validation_epoch_end()
```

▶ `core/src/crackz/ai/model/segmentation/model.py`

```
on_test_epoch_end()
```

▶ `core/src/crackz/ai/model/segmentation/model.py`

```
configure_optimizers()
```

▶ `core/src/crackz/ai/model/segmentation/model.py`

```
get_loss_function(config)
```

▶ `core/src/crackz/ai/model/segmentation/model.py`

`Callback`

▶ `core/src/crackz/ai/training/callbacks.py`

```
on_train_start(trainer, pl_module)
```

▶ `core/src/crackz/ai/training/callbacks.py`

```
on_train_batch_end(
    trainer, pl_module, outputs, batch, batch_idx
)
```

▶ `core/src/crackz/ai/training/callbacks.py`

```
on_train_epoch_end(trainer, pl_module)
```

▶ `core/src/crackz/ai/training/callbacks.py`

```
on_validation_epoch_end(trainer, pl_module)
```

▶ `core/src/crackz/ai/training/callbacks.py`

```
on_test_epoch_end(trainer, pl_module)
```

▶ `core/src/crackz/ai/training/callbacks.py`

```
on_fit_end(trainer, pl_module)
```

▶ `core/src/crackz/ai/training/callbacks.py`

`Callback`

▶ `core/src/crackz/ai/training/callbacks.py`

```
__call__(progress, message=None)
```

▶ `core/src/crackz/ai/training/callbacks.py`

```
on_train_start(trainer, pl_module)
```

▶ `core/src/crackz/ai/training/callbacks.py`

```
on_train_epoch_start(trainer, pl_module)
```

▶ `core/src/crackz/ai/training/callbacks.py`

```
on_train_batch_end(
    trainer, pl_module, outputs, batch, batch_idx
)
```

▶ `core/src/crackz/ai/training/callbacks.py`

```
on_train_epoch_end(trainer, pl_module)
```

▶ `core/src/crackz/ai/training/callbacks.py`

```
on_fit_end(trainer, pl_module)
```

▶ `core/src/crackz/ai/training/callbacks.py`

```
on_test_end(trainer, pl_module)
```

▶ `core/src/crackz/ai/training/callbacks.py`

`Callback`

▶ `core/src/crackz/ai/training/callbacks.py`

```
on_validation_epoch_end(trainer, pl_module)
```

▶ `core/src/crackz/ai/training/callbacks.py`

```
make_reporter(ui_callback)
```

▶ `core/src/crackz/ai/training/callbacks.py`

```
create_training_callbacks(
    project,
    val_ds,
    tb_logger,
    progress_reporter,
    runtime_tracker,
    cancellation_token,
)
```

`project`	[`Project`]()		*required*
`val_ds`	`CrackzDataset`		*required*

`tb_logger`	`TensorBoardLogger | None`		*required*
`progress_reporter`	`ProgressReporter | None`		*required*
`runtime_tracker`	`RuntimeTracker`		*required*
`cancellation_token`	`CancellationToken | None`		*required*

`list[Any]`	

▶ `core/src/crackz/ai/training/callbacks.py`

```
get_available_checkpoints(checkpoints_dir)
```

`checkpoints_dir`	`Path`		*required*

`list[Path]`	

▶ `core/src/crackz/ai/model/checkpoints.py`

```
has_trained_model(project)
```

`project`	[`Project`]()		*required*

`bool`	

▶ `core/src/crackz/ai/model/checkpoints.py`

```
select_checkpoint(checkpoints_dir)
```

`checkpoints_dir`	`Path`		*required*

`Path | None`	

▶ `core/src/crackz/ai/model/checkpoints.py`

```
peek_checkpoint_backbone(checkpoint_path)
```

▶ `core/src/crackz/ai/model/checkpoints.py`

```
get_checkpoint_compatibility(project)
```

▶ `core/src/crackz/ai/model/checkpoints.py`

```
load_model_from_checkpoint(
    project,
    lr,
    checkpoint_name=None,
    *,
    on_incompatible="original",
)
```

`project`	[`Project`]()		*required*
`lr`	`float`		*required*
`checkpoint_name`	`str | None`		`None`
`on_incompatible`	`IncompatibleMode`	`"original"` `"untrained"` `"raise"` `CheckpointIncompatibleError`	`'original'`

`CrackzModel`

`CheckpointIncompatibleError` *on_incompatible* `"raise"`

▶ `core/src/crackz/ai/model/checkpoints.py`

`create_checkpoint_callback(project)`

`project`	`Project`		*required*

`ModelCheckpoint`

▶ `core/src/crackz/ai/model/checkpoints.py`

`prune_checkpoints(project, keep)`

`project`	`Project`		*required*
`keep`	`int`		*required*

`list[Path]`

▶ `core/src/crackz/ai/model/checkpoints.py` `plot_image(img_path, out_file, pred_mask)`

▶ `core/src/crackz/ai/visualization/visualization.py`

`save_detection_figure(fig, out_file)`

▶ `core/src/crackz/ai/visualization/visualization.py`

`save_metrics_json(results, output_dir)`

▶ `core/src/crackz/ai/visualization/visualization.py`

`save_training_metrics_json(results, output_dir)`

▶ `core/src/crackz/ai/visualization/visualization.py`

`show_image(ax, index, title, img, alpha=None, cmap='gray')`

▶ `core/src/crackz/ai/visualization/visualization.py`

`to_numpy_img(tensor)`

▶ `core/src/crackz/ai/visualization/visualization.py`

`Enum`

▶ `core/src/crackz/ai/device/device.py`

`staticmethod`

```
available_devices()
```

▶ `core/src/crackz/ai/device/device.py`

`staticmethod`

```
default()
```

▶ `core/src/crackz/ai/device/device.py`

`staticmethod`

```
test_device(device_name)
```

`device_name`	`str`		*required*

`bool`	
`str`	•
`dict[str, str | None]`	•
`tuple[bool, str, dict[str, str | None]]`	•

▶ `core/src/crackz/ai/device/device.py`

```
get_accelerator()
```

▶ `core/src/crackz/ai/device/utils.py`

```
resolve_device_and_accelerator(project)
```

`project`	[Project]()		*required*

`str`

▶ `core/src/crackz/ai/device/utils.py`

[`AIPipelineError`]

[`ModelNotFoundError`]

[`DataLoadingError`]

`CrackzError`

▶ `core/src/crackz/ai/exceptions.py`

`CrackzError`

▶ `core/src/crackz/ai/exceptions.py`

[`AIPipelineError`]

▶ `core/src/crackz/ai/exceptions.py`

`AIPipelineError`

▶ `core/src/crackz/ai/exceptions.py`

`AIPipelineError`

▶ `core/src/crackz/ai/exceptions.py`

`AIPipelineError`

▶ `core/src/crackz/ai/exceptions.py`

`detection_results.json`

▼

`save_metrics_json`

`crackz.ai.eval`

`BaseModel`

▶ `core/src/crackz/ai/detection/results.py`

```
load_detection_results(project_or_path, *, strict=False)
```

▶ `core/src/crackz/ai/detection/results.py`

```
iter_iou(results)
```

▶ `core/src/crackz/ai/detection/results.py`

`DetectionResult`

```
aggregate_iou(results)
```

▶ `core/src/crackz/ai/detection/eval.py`

```
basic_summary(results)
```

▶ `core/src/crackz/ai/detection/eval.py`

```
get_available_checkpoints(checkpoints_dir)
```

`checkpoints_dir`	`Path`		*required*

`list[Path]`	

▶ `core/src/crackz/ai/model/checkpoints.py`

```
has_trained_model(project)
```

`project`	[`Project`]()		*required*

`bool`	

▶ `core/src/crackz/ai/model/checkpoints.py`

```
select_checkpoint(checkpoints_dir)
```

`checkpoints_dir`	`Path`		*required*

`Path | None`	

▶ `core/src/crackz/ai/model/checkpoints.py`

```
peek_checkpoint_backbone(checkpoint_path)
```

▶ `core/src/crackz/ai/model/checkpoints.py`

```
get_checkpoint_compatibility(project)
```

▶ `core/src/crackz/ai/model/checkpoints.py`

```
load_model_from_checkpoint(
    project,
    lr,
    checkpoint_name=None,
    *,
    on_incompatible="original",
)
```

`project`	[`Project`]()		*required*
`lr`	`float`		*required*
`checkpoint_name`	`str | None`		`None`
`on_incompatible`	`IncompatibleMode`	`"original"` `"untrained"` `"raise"` `CheckpointIncompatibleError`	`'original'`

[`CrackzModel`]()	

[`CheckpointIncompatibleError`]()	*on_incompatible* `"raise"`

▶ `core/src/crackz/ai/model/checkpoints.py`

```
create_checkpoint_callback(project)
```

`project`	[`Project`]()		*required*

`ModelCheckpoint`	

▶ `core/src/crackz/ai/model/checkpoints.py`

```
prune_checkpoints(project, keep)
```

`project`	[`Project`]		*required*
`keep`	`int`		*required*

`list[Path]`	

▶ `core/src/crackz/ai/model/checkpoints.py`

▶ `core/src/crackz/ai/pipeline/cancellation.py`

```
__init__()
```

▶ `core/src/crackz/ai/pipeline/cancellation.py`

```
cancel()
```

▶ `core/src/crackz/ai/pipeline/cancellation.py`

```
is_cancelled()
```

`bool`	

▶ `core/src/crackz/ai/pipeline/cancellation.py`

```
check_cancelled()
```

`CancellationError`	

▶ `core/src/crackz/ai/pipeline/cancellation.py`

`Handler`

▶ `core/src/crackz/log/crackz_logger.py`

`emit(record)`

▶ `core/src/crackz/log/crackz_logger.py`

`use_rich_console()`

`bool`	

▶ `core/src/crackz/log/crackz_logger.py`

`setup_logging(level=None, log_dir=None, *, use_rich=None)`

`level`	`str | None`		`None`

`log_dir`	`Path | None`		`None`
`use_rich`	`bool | None`		`None`

`Logger`	

▶ `core/src/crackz/log/crackz_logger.py`

```
get_logger(name, filename=None, log_dir='logs')
```

`logging` `filename` `InterceptHandler`

▶ `core/src/crackz/log/crackz_logger.py`

`TyperGroup`

▶ `core/src/crackz/cli/crackz_cli.py`

```
main(
    args=None,
    prog_name=None,
    complete_var=None,
    standalone_mode=True,
    windows_expand_args=True,
    **extra,
)
```

▶ `core/src/crackz/cli/crackz_cli.py`

```
list_commands(ctx)
```

▶ `core/src/crackz/cli/crackz_cli.py`

```
get_command(ctx, cmd_name)
```

▶ `core/src/crackz/cli/crackz_cli.py`

```
format_commands(ctx, formatter)
```

▶ `core/src/crackz/cli/crackz_cli.py`

```
root_callback(
    ctx,
    version=False,
    debug=False,
    quiet=False,
    log_dir=None,
)
```

▶ `core/src/crackz/cli/crackz_cli.py`

```
main()
```

▶ `core/src/crackz/cli/crackz_cli.py`

```
init_project(
    name=NAME_OPTION,
    image_path=IMAGE_PATH_OPTION,
    image_mask_path=IMAGE_MASK_PATH_OPTION,
    artifacts_path=ARTIFACTS_PATH_OPTION,
    description=DESCRIPTION_OPTION,
    log_dir=LOG_DIR_OPTION,
    *,
    force=FORCE_OPTION,
)
```

▶ `core/src/crackz/cli/project_cli.py`

```
export_project_config(
    cfg=CFG_OPTION,
    project_path=PROJECT_PATH_OPTION,
    out=EXPORT_OUTPUT_OPTION,
)
```

▶ `core/src/crackz/cli/project_cli.py`

```
migrate_project_config(
    cfg=CFG_OPTION,
```

```
    project_path=PROJECT_PATH_OPTION,
    *,
    dry_run=DRY_RUN_OPTION,
    no_backup=NO_BACKUP_OPTION,
    migrate_files=typer.Option(
        default=False,
        help="Also migrate artifacts directory structure (checkpoints to models, etc.)",
    ),
)
```

▶ `core/src/crackz/cli/project_cli.py`

```
clean_project(
    project_path=PROJECT_PATH_OPTION,
    *,
    dry_run=DRY_RUN_OPTION,
    logs_only=LOGS_ONLY_OPTION,
    keep_best_model=KEEP_BEST_MODEL_OPTION,
)
```

▶ `core/src/crackz/cli/project_cli.py`

```
import_images(
    cfg=CFG_OPTION,
    project_path=PROJECT_PATH_OPTION,
    source_path=SOURCE_PATH_OPTION,
    masks_source_path=MASKS_SOURCE_PATH_OPTION,
    image_type=TYPE_OPTION,
    image_size=SIZE_OPTION,
    quality=QUALITY_OPTION,
    target_size=TARGET_SIZE_OPTION,
    *,
    overwrite=OVERWRITE_OPTION,
    split=SPLIT_OPTION,
    train_ratio=TRAIN_RATIO_OPTION,
    val_ratio=VAL_RATIO_OPTION,
    test_ratio=TEST_RATIO_OPTION,
    random_seed=RANDOM_SEED_OPTION,
)
```

`cfg`	`Path | None`		`CFG_OPTION`
`project_path`	`Path | None`		`PROJECT_PATH_OPTION`
`source_path`	`Path`		`SOURCE_PATH_OPTION`

`masks_source_path`	`Path | None`	`source_path`	`MASKS_SOURCE_PATH_OPTION`
`image_type`	[`ImagesType`]()		`TYPE_OPTION`
`image_size`	`SizeType`		`SIZE_OPTION`
`quality`	`int`		`QUALITY_OPTION`
`target_size`	`int | None`		`TARGET_SIZE_OPTION`
`overwrite`	`bool`		`OVERWRITE_OPTION`
`split`	`bool`		`SPLIT_OPTION`
`train_ratio`	`float`		`TRAIN_RATIO_OPTION`
`val_ratio`	`float`		`VAL_RATIO_OPTION`
`test_ratio`	`float`		`TEST_RATIO_OPTION`
`random_seed`	`int | None`		`RANDOM_SEED_OPTION`

`BadParameter`	
`Exit`	

`None`	

▶ `core/src/crackz/cli/image_import_cli.py`

```
run_detection(
    cfg=CFG_OPTION,
    project_path=PROJECT_PATH_OPTION,
    model=CHECKPOINT_NAME_OPTION,
)
```

▶ `core/src/crackz/cli/detection_cli.py`

```
detection_train(
    cfg=CFG_OPTION,
    project_path=PROJECT_PATH_OPTION,
    epochs=EPOCHS_OPTION,
    batch_size=BATCH_SIZE_OPTION,
    resume=RESUME_FROM_OPTION,
)
```

`cfg`	`Path | None`	`project_path`	`CFG_OPTION`
`project_path`	`Path | None`	`cfg`	`PROJECT_PATH_OPTION`
`epochs`	`int | None`		`EPOCHS_OPTION`
`batch_size`	`int | None`		`BATCH_SIZE_OPTION`
`resume`	`str | None`		`RESUME_FROM_OPTION`

`BadParameter`	`cfg` `project_path`
`Exit`	

▶ `core/src/crackz/cli/detection_cli.py`

```
prune_checkpoints_cmd(
    keep=KEEP_CHECKPOINTS_OPTION,
    cfg=CFG_OPTION,
    project_path=PROJECT_PATH_OPTION,
)
```

▶ `core/src/crackz/cli/detection_cli.py`

	`CrackzModel` `CrackzDataset`	
	`MetricsLoggerCallback` `resolve_device_and_accelerator` `DetectionResult` `Pipeline`	

1.
2.
3. `DetectionResult`
4.
5. `evaluation.py`
6.
7.
8.
9.
10.
11.
12.
13.
14.
15.
16.
17.
18. `ProjectConfig`
19.
20.
21.
22. `Pipeline`

```
Project -> load_model_from_checkpoint -> CrackzModel.eval() -> DataLoader -> forward -> sigmoid -> threshold -> mask -> metrics(json)
```

`requirements.txt`

CLI Reference

app

```
 [OPTIONS] COMMAND [ARGS]...
```

```
 -v, --version          Show version and exit
 --debug                Enable debug logging
 --quiet                Only warnings and errors
 --log-dir DIRECTORY    Central log directory (precedence: env CRACKZ_LOG_DIR
                        > --log-dir > system temp). Created if missing.
                        Project logs are always under <project>/logs.
 --install-completion   Install completion for the current shell.
 --show-completion      Show completion for the current shell, to copy it or
                        customize the installation.
```

`dashboard`

```
crackz dashboard --project my-project
```

`http://localhost:8501`

```
crackz dashboard --project my-project --port 8080
```

```
crackz dashboard
```

1.
2.
3.
4.
5.
6.
7.
8.
9.
10.
11.
12.
13.
14.
15.
16.

17.
18.

```
crackz detect --project my-project
crackz dashboard --project my-project
# Navigate to "Detection Results" page
```

```
crackz train --project my-project --epochs 50
crackz dashboard --project my-project
# Navigate to "Training Metrics" page
```

```
crackz dashboard --project my-project
# Navigate to "GPS Map" page
```

```
crackz dashboard --project my-project
# Navigate to "Parameter Tuning" page
# Adjust thresholds and preview results
```

`clean`

```
crackz clean --project ./my_project
```

```
crackz clean -p ./my_project
```

- `*.log`
-
- `models/` `checkpoints/`
-
-

```
crackz clean --project ./my_project --dry-run
```

```
Dry run: The following would be cleaned:
  • /path/to/project/logs/crackz.log
  • /path/to/project/artifacts/training/tensorboard
  • /path/to/project/artifacts/models
  • /path/to/project/artifacts/detections

Re-run without --dry-run to apply changes.
```

```
crackz clean --project ./my_project --logs-only
```

`*.log`

`models/` `checkpoints/`

```
$ crackz clean --project ./my_project

Project cleaned! Removed 5 items (logs and artifacts).
```

`--logs-only`

```
$ crackz clean --project ./my_project --logs-only

Project cleaned! Removed 2 items (logs).
```

```
crackz clean -p ./my_project
```

```
crackz clean -p ./my_project --logs-only
```

```
crackz clean -p ./my_project --dry-run
```

`migrate`

`schema_version: 1` `ai.training.tensorboard` `false` `data.mask_threshold: 1` `crackz_version`

`checkpoints/` `models/` `training/tensorboard/` `runtime/tensorboard/` `training/lightning_logs/` `runtime/lightning_logs/` `detections/*.png` `detections/visualizations/`

```
crackz migrate --project ./my_project
```

```
crackz migrate --project ./my_project --migrate-files
```

```
crackz migrate --cfg ./my_project/my_project_project.yaml --migrate-files
```

```
crackz migrate -p ./my_project --migrate-files
crackz migrate -c ./my_project/my_project_project.yaml --migrate-files
```

```
# Schema changes only
crackz migrate --project ./my_project --dry-run

# Schema + file changes
crackz migrate --project ./my_project --migrate-files --dry-run
```

would

`.v{version}.{timestamp}.bak`

```
crackz migrate --project ./my_project --no-backup
```

```
[migrated] my_project schema 0 -> 1 (current=1) steps: 0->1: added tensorboard default + schema_version=1
```

```
[up-to-date] my_project schema 1 -> 1 (current=1) steps: none
```


`build.yml`	
`integration-tests.yml`	`pytest -m integration`
`release.yml`	
`docker-build.yml`	
`smoke-tests.yml`	
`benchmark-tests.yml`	`pytest -m benchmark`
`docs.yml`	

```
pytest -m integration -vv
```

```
pytest -m "not integration"
```

```
pytest -m benchmark --benchmark-only
```

`crackz/cli/*_cli.py`

`schema_version`

- `.bak`
- `CRACKZ_AUTO_MIGRATE=1`

```
crackz migrate --project <project-dir>
```

`--dry-run` `--no-backup`

`migration.md`

`model`

```
crackz model export --project ./my_project --out my_model.zip
```

```
crackz model export -p ./my_project -o my_model.zip
```

<project_name>_model.zip

```
crackz model export -p ./my_project -c final -o final_model.zip
```

`best` `last` `final`

```
crackz model import --project ./my_project --model my_model.zip
```

```
crackz model import -p ./my_project -m my_model.zip
```

```
crackz model import -p ./my_project -m my_model.zip --force
```

`--force`

```
# Export from training project
crackz model export -p ./training_project -o harbor_crack_v1.zip

# Import into deployment project
crackz model import -p ./deployment_project -m harbor_crack_v1.zip
```

```
# Export pre-trained model
crackz model export -p ./pretrained_project -o base_model.zip

# Import into new project for fine-tuning
crackz model import -p ./finetuning_project -m base_model.zip
```

```
# Export after successful training
crackz model export -p ./my_project -o backups/model_$(date +%Y%m%d).zip
```

`<checkpoint_name>.ckpt` `metadata.json`

```
{
  "project_name": "harbor_detection",
  "checkpoint_name": "best.ckpt",
  "model_config": {
    "encoder": "resnet34",
    "encoder_weights": "imagenet",
    "in_channels": 3,
    "num_classes": 1,
    "threshold": 0.5,
    "loss_name": "combo",
    "dice_weight": 0.7,
    "focal_weight": 0.3
  }
}
```

Model Evaluation

Model Evaluation

```
# Run evaluation on test dataset
crackz evaluate run --project my_harbor

# Specific metrics
crackz evaluate run --project my_harbor --metrics accuracy,iou

# Export results
crackz evaluate run --project my_harbor --output results.json
crackz evaluate run --project my_harbor --output results.csv
```

```
from pathlib import Path
from crackz.ai.evaluation import evaluate_model, export_json

# Run evaluation
results = evaluate_model(
    project_path=Path("my_harbor"),
    metrics=["accuracy", "precision", "recall", "iou"]
)

# Display results
print(results)

# Export to JSON
export_json(results, Path("evaluation_results.json"))
```

```
Accuracy = (TP + TN) / (TP + TN + FP + FN)
```

```
Precision = TP / (TP + FP)
```

```
Recall = TP / (TP + FN)
```

```
F1 = 2 * (Precision * Recall) / (Precision + Recall)
```

```
IoU = Area of Overlap / Area of Union
```

```
# Default threshold (0.5)
crackz evaluate run --project my_harbor

# Custom threshold
crackz evaluate run --project my_harbor --threshold 0.7
```

```
# All metrics (default)
crackz evaluate run --project my_harbor

# Specific metrics
crackz evaluate run --project my_harbor --metrics accuracy,precision,recall
crackz evaluate run --project my_harbor --metrics iou  # Only IoU
```

`accuracy` `precision` `recall` `f1` `iou`

```
Evaluation Results (500 images)
==================================================
Accuracy:     92.3%
Precision:    89.1%
Recall:       94.8%
F1 Score:     91.9%
Mean IoU:     0.847
Threshold:    0.5
```

```
{
  "num_images": 500,
  "threshold": 0.5,
  "metrics": {
    "accuracy": 0.923,
    "precision": 0.891,
    "recall": 0.948,
    "f1_score": 0.919,
    "iou": 0.847
  }
}
```

```
Metric,Value
Num Images,500
Threshold,0.5
Accuracy,0.9230
Precision,0.8910
Recall,0.9480
F1 Score,0.9190
IoU,0.8470
```

```
project/
├── images/
│   └── test/          # Test images
│       ├── img1.png
│       ├── img2.png
│       └── ...
└── masks/
    └── test/          # Ground truth masks
        ├── img1.png
```

```
        ├── img2.png
        └── ...
```

```
def evaluate_model(
    project_path: Path,
    test_images: list[Path] | None = None,
    test_masks: list[Path] | None = None,
    metrics: list[str] | None = None,
    threshold: float = 0.5,
) -> EvaluationResults:
    """Evaluate crack detection model on test dataset."""
```

`project_path` `test_images`
`test_masks` `metrics`
`threshold`

`EvaluationResults`

```
@dataclass
class EvaluationResults:
    accuracy: float
    precision: float
    recall: float
    f1_score: float
    iou: float
    num_images: int
    threshold: float
```

```
# Export to JSON
export_json(results, Path("results.json"))

# Export to CSV
export_csv(results, Path("results.csv"))
```

```
ValueError: No test images found
```

`<project>/images/test/`

```
ls <project>/images/test/
```

```
ValueError: Image/mask count mismatch: 100 vs 98
```

```
# Check counts
ls <project>/images/test/ | wc -l
ls <project>/masks/test/ | wc -l

# Find missing masks
diff <(ls <project>/images/test/) <(ls <project>/masks/test/)
```

```
FileNotFoundError: No trained model checkpoint found
```

```
crackz detect train --project <project>
```

MCP Integration

MCP (Model Context Protocol) Integration

`.github/copilot-instructions.md`

```
┌──────────────────────────┬───────────────────┬───────────────────┐
│ Development Agents       │ GitHub Copilot    │ End-User GUI      │
├──────────────────────────┼───────────────────┼───────────────────┤
│ □ 4 MCP Servers          │ × No MCP          │ × No MCP          │
│   • filesystem           │                   │                   │
│   • github               │ IDE context       │ Simple API        │
│   • python               │ Workspace files   │ Project info only │
│   • uv                   │                   │                   │
├──────────────────────────┼───────────────────┼───────────────────┤
│ Tools: Claude Desktop,   │ Tools: VS Code,   │ Tools: Crackz GUI │
│        Cline             │        IDE        │        Chat Panel │
└──────────────────────────┴───────────────────┴───────────────────┘
```

```
TOOL CALL: filesystem.readDirectory("src/")
→ returns 50 files in context

TOOL CALL: filesystem.readFile("src/file1.py")
→ returns full file in context

TOOL CALL: filesystem.readFile("src/file2.py")
→ returns full file in context
```

```
# Discover and filter in code
files = await filesystem.readDirectory("src/")
python_files = [f for f in files if f.endswith(".py")]

# Load only what's needed
for filename in python_files[:5]:  # Top 5 only
    content = await filesystem.readFile(f"src/{filename}")
    # Process in code, log summary only
    print(f"{filename}: {len(content)} lines")
```

1.
2.
3.
4.

`<type>/<scope>-<short-slug>`

```
feat/gui-live-preview       # New GUI feature
fix/pdf-rendering-crash     # Bug fix in PDF generation
perf/dataloader-cache       # Performance improvement
docs/api-reference-update   # Documentation update
refactor/config-cleanup     # Code refactoring
```

`<type>(<scope>): <description>`

```
feat(gui): add real-time detection preview
fix(cli): correct path handling in Windows
perf(dataloader): implement batch caching
docs(api): update training configuration reference
```

`<type>(<scope>): <description>`

```
feat(gui): add export panel for detection results
fix(pdf): handle missing hero image gracefully
perf(training): optimize data loading pipeline
```

```
Fix GUI bugs and add new feature
Update documentation
Add export functionality
```

`feat:` `feat(<scope>):` `fix:` `fix(<scope>):`
`perf:` `perf(<scope>):`

`docs:` `refactor:`
`test:` `chore:` `ci:` `build:`

`typer.secho()` `logger` `crackz.log`
`crackz.cli.options` `print()`

`uv run task tests`

`uv run task mypy`

```
crackz mcp init
```

`.mcp/config.json`

```
crackz mcp show
```

```
crackz mcp validate
```

`.mcp/config.json`

```
{
  "mcpServers": {
    "filesystem": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "."],
      "description": "Filesystem operations with progressive disclosure"
    },
    "github": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "${GITHUB_TOKEN}"
      },
      "description": "GitHub operations (issues, PRs, releases)"
    },
    "python": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-python"],
      "description": "Python code execution environment"
    },
    "uv": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-uv"],
      "description": "UV package manager operations"
    }
  }
}
```

```
{
  "metadata": {
    "project": "Crackz",
    "language": "python",
    "version": "3.13",
    "package_manager": "uv",
    "framework": "pytorch-lightning",
    "cli": "typer",
    "gui": "pyside6",
    "testing": "pytest",
    "ci": "github-actions",
    "release": "semantic-release",
    "code_execution": true,
    "progressive_disclosure": true
  }
}
```

```
{
  "patterns": {
    "code_execution": {
      "enabled": true,
      "description": "Use code execution to interact with MCP servers efficiently",
      "examples": [
        "Filter large datasets in code before returning results",
        "Loop over operations without alternating tool calls",
        "Compose multiple MCP operations in a single execution"
      ]
    },
    "progressive_disclosure": {
      "enabled": true,
      "description": "Load tool definitions on-demand",
      "workflow": [
        "List directories to discover modules",
        "Read specific files as needed",
        "Load documentation when relevant"
      ]
    }
  }
}
```

`--output` `-o` `.mcp/config.json` `--force` `-f`

```
# Create with defaults
crackz mcp init

# Create in custom location
crackz mcp init --output custom/mcp.json

# Overwrite existing config
crackz mcp init --force
```

`--config` `-c` `.mcp/config.json`

```
# Show default config
crackz mcp show

# Show custom config
crackz mcp show --config custom/mcp.json
```

`--config` `-c` `.mcp/config.json`

```
# Validate default config
crackz mcp validate

# Validate custom config
crackz mcp validate --config custom/mcp.json
```

`.mcp/config.json`

```
{
  "mcpServers": {
    "filesystem": { ... },
    "custom-api": {
      "type": "stdio",
      "command": "node",
      "args": ["./custom-mcp-server.js"],
      "description": "Custom MCP server for specialized operations"
    }
  }
}
```

```
{
  "metadata": {
    "project": "Crackz",
    "language": "python",
    "team": "ML Ops",
    "repository": "https://github.com/Anaxiatech-AB/crackz",
    "custom_field": "your_value"
  }
}
```

1.
2.
3.
4.
5.

```
# List Python files in src/
files = await filesystem.readDirectory("src/crackz")
py_files = [f for f in files if f.endswith(".py")]

# Read only the first 3 for initial analysis
for filename in py_files[:3]:
    content = await filesystem.readFile(f"core/src/crackz/{filename}")
    lines = len(content.split('\n'))
    print(f"{filename}: {lines} lines")
```

```
# Get recent PRs and filter in code
prs = await github.listPullRequests(repo="Anaxiatech-AB/crackz", state="open")
priority_prs = [pr for pr in prs if "feat:" in pr.title or "fix:" in pr.title]

print(f"Found {len(priority_prs)} priority PRs out of {len(prs)} total")
for pr in priority_prs[:5]:  # Show top 5 only
    print(f"#{pr.number}: {pr.title}")
```

```
# Install multiple packages in one operation
packages = ["pytest", "ruff", "mypy"]
for pkg in packages:
    await uv.install(package=pkg)
    print(f"✓ Installed {pkg}")
```

`logger`

`print()`

2. `CRACKZ_LOG_DIR` `--log-dir`

5. `<project>/logs/`

Info level - operational progress

Warning level - non-critical issues

Error level - failures with details

Debug level - detailed troubleshooting info

Tracing & Observability

Tracing with OpenTelemetry

```
uv sync --extra tracing
```

`opentelemetry-instrumentation-anthropic` `opentelemetry-sdk` `opentelemetry-exporter-otlp-proto-http`

`Ctrl+Shift+P` `Cmd+Shift+P`

```
AI Toolkit: Open Tracing
```

`ai-mlstudio.tracing.open`

```
export CRACKZ_TRACING_ENABLED=1
crackz-gui  # Tracing auto-enabled
```

```
from crackz.tracing import setup_tracing

# Initialize with default AI Toolkit endpoint
setup_tracing()

# Or custom endpoint
setup_tracing(
    endpoint="http://custom-collector:4318/v1/traces",
    service_name="crackz-production"
)
```

-
-
-
-
-

```
crackz
└── anthropic.messages.create
    ├── model: claude-sonnet-4-5-20250929
    ├── input_tokens: 1245
    ├── output_tokens: 382
    └── duration: 2.3s
```

`CRACKZ_TRACING_ENABLED`	`0`	`1` `0`

`http://localhost:4318/v1/traces` `http://localhost:4317`

```
setup_tracing(endpoint="http://your-collector:4318/v1/traces")
```

```
# Enable tracing for all CLI operations
export CRACKZ_TRACING_ENABLED=1
crackz detect run --project my_project

# Disable for specific run
export CRACKZ_TRACING_ENABLED=0
crackz detect run --project my_project
```

```
# In main.py or startup script
from crackz.tracing import setup_tracing

# Enable before starting GUI
if os.getenv("CRACKZ_TRACING_ENABLED") == "1":
    setup_tracing()

# Run GUI
from crackz_gui.qt.app import main
main()
```

```
from crackz.tracing import setup_tracing, shutdown_tracing

# Initialize tracing
setup_tracing(service_name="crackz-batch-processor")

# Your AI operations (automatically traced)
from crackz_gui.services.ai_chat import AIChatService
chat = AIChatService(...)
chat.send_message("Analyze this crack detection result")

# Clean shutdown (flushes pending spans)
shutdown_tracing()
```

```
uv sync --extra tracing
```

```
Ctrl+Shift+P → AI Toolkit: Open Tracing
```

`uv sync --extra tracing`
`CRACKZ_TRACING_ENABLED=1`

`shutdown_tracing()`

```
import atexit
from crackz.tracing import shutdown_tracing

atexit.register(shutdown_tracing)
```

`CRACKZ_TRACING_ENABLED=1`

```
from opentelemetry import trace

tracer = trace.get_tracer(__name__)

with tracer.start_as_current_span("custom-operation") as span:
    span.set_attribute("project.name", "harbor_alpha")
    span.set_attribute("image.count", 1024)
    # Your code here
```

```
from opentelemetry.sdk.trace.sampling import TraceIdRatioBased

# Sample 10% of traces
sampler = TraceIdRatioBased(0.1)

# Pass to setup_tracing (requires custom implementation)
```

Async Callbacks

Async Callback Architecture

```python
from crackz.ai.core.async_callbacks import CallbackDispatcher

dispatcher = CallbackDispatcher(maxsize=100)

# Create sync reporter for PyTorch Lightning (training thread)
progress_reporter = dispatcher.create_sync_progress_reporter()

# Use in training (synchronous context)
train(project, progress_reporter=progress_reporter)

# Consume updates asynchronously (main thread)
async for progress, message in dispatcher.iter_progress():
    update_progress_bar(progress, message)
```

```python
from crackz.ai.core.async_callbacks import (
    create_async_progress_consumer,
    create_async_metrics_consumer,
)

# Define callback
def update_ui(progress: float, message: str | None = None) -> None:
    progress_bar.set_value(progress)

# Create async consumer
consumer = create_async_progress_consumer(dispatcher, update_ui)

# Run in async context
await consumer()
```

```python
from crackz_gui.qt.async_bridge import (
    start_gui_callback_consumers,
    stop_gui_callback_consumers,
)

# Start consumers (manages event loop)
loop, tasks = start_gui_callback_consumers(
    dispatcher,
    progress_callback=update_progress,
    metrics_callback=update_metrics,
)

# ... run training ...
```

```
# Cleanup
stop_gui_callback_consumers(dispatcher, loop)
```

`TaskSignals` `async_bridge.py`

```
from crackz_gui.qt.async_bridge import TaskSignals, run_in_background

def _do_work(signals: TaskSignals) -> None:
    signals.progress.emit(0.5)   # Safe from any thread
    signals.status.emit("Processing...")
    result = expensive_operation()
    signals.data.emit(result)
    signals.finished.emit()

signals = run_in_background(_do_work)
signals.progress.connect(self._on_progress)
signals.finished.connect(self._on_complete)
signals.error.connect(self._on_error)
```

`threading.Lock` `after()`

`crackz_gui.qt.async_bridge`

```
def progress_callback(progress: float, message: str | None = None) -> None:
    print(f"Progress: {progress:.2%}")

train(project, progress_reporter=progress_callback)
```

```
dispatcher = CallbackDispatcher(maxsize=100)

# Create sync reporters
progress_reporter = dispatcher.create_sync_progress_reporter()
metrics_reporter = dispatcher.create_sync_metrics_reporter()

# Start training in background
training_thread = threading.Thread(
    target=train,
    args=(project,),
    kwargs={
        "progress_reporter": progress_reporter,
        "metrics_callback": metrics_reporter,
    }
)
training_thread.start()

# Consume updates asynchronously
async def consume_updates():
    async for progress, message in dispatcher.iter_progress():
        print(f"Progress: {progress:.2%} - {message}")

asyncio.run(consume_updates())
```

```
from crackz.ai.core.async_callbacks import CallbackDispatcher
from crackz_gui.qt.async_bridge import (
    start_gui_callback_consumers,
    stop_gui_callback_consumers,
)

dispatcher = CallbackDispatcher(maxsize=100)

# Start async consumers with Qt-thread-safe GUI updates
loop, tasks, emitters = start_gui_callback_consumers(
    dispatcher,
    progress_callback=self.update_progress_bar,
```

```
    metrics_callback=self.update_metrics_display,
)
self._qt_emitters = emitters  # Keep signal emitters alive

# Create reporters
progress_reporter = dispatcher.create_sync_progress_reporter()
metrics_reporter = dispatcher.create_sync_metrics_reporter()

try:
    # Run training in background thread
    training_thread = threading.Thread(
        target=train,
        args=(project,),
        kwargs={
            "progress_reporter": progress_reporter,
            "metrics_callback": metrics_reporter,
        }
    )
    training_thread.start()
    training_thread.join()
finally:
    stop_gui_callback_consumers(dispatcher, loop)
```

```
Training Loop:
  Process batch → Update metrics → **Wait for UI** → Next batch
                                    ↑ 1-5ms overhead
```

```
Training Loop:
  Process batch → Put in queue → Next batch (no wait!)
                       ↓ 0.1ms
  Async Consumer: Takes from queue → Updates UI
```

```
dispatcher = CallbackDispatcher(maxsize=100)

# ... use reporters ...

# Check progress queue stats
stats = dispatcher.progress_stats
print(f"Total items put: {stats.total_put}")
print(f"Total items retrieved: {stats.total_get}")
print(f"Total items dropped: {stats.total_dropped}")
print(f"Current queue size: {stats.current_size}")
print(f"Max queue size: {stats.max_size}")

# Check metrics queue stats
metrics_stats = dispatcher.metrics_stats
```

`total_dropped`

- `sync_q` `async_q`
- `threading.Event`
-
-

```
Training Thread          Async Event Loop Thread     GUI Main Thread
     |                            |                        |
     ├─ sync_q.put() ────────────>|                        |
     |  (thread-safe)             |                        |
     |                            ├─ async_q.get()         |
     |                            |                        |
```

```
     │                             ├─ Qt signal emit ─────>│
     │                             │                       │─ Update UI
```

```
# Before (still works)
train(project, progress_reporter=my_callback)

# After (optional upgrade for better performance)
dispatcher = CallbackDispatcher()
train(project, progress_reporter=dispatcher.create_sync_progress_reporter())
```

```
from crackz.ai.core.async_callbacks import CallbackDispatcher
from crackz_gui.qt.async_bridge import start_gui_callback_consumers

dispatcher = CallbackDispatcher(maxsize=100)
loop, tasks = start_gui_callback_consumers(
    dispatcher,
    progress_callback=update_progress,
    metrics_callback=update_metrics,
)

# Use dispatcher's sync reporters in training
train(
    project,
    progress_reporter=dispatcher.create_sync_progress_reporter(),
    metrics_callback=dispatcher.create_sync_metrics_reporter(),
)

stop_gui_callback_consumers(dispatcher, loop)
```

`crackz.ai.core.async_callbacks`

- `CallbackDispatcher`
- `QueueStats`
- `create_async_progress_consumer()`
- `create_async_metrics_consumer()`

`crackz_gui.qt.async_bridge`

- `AsyncEventLoop`
- `create_threadsafe_gui_callback()`
- `create_threadsafe_metrics_callback()`
- `start_gui_callback_consumers()`
- `stop_gui_callback_consumers()`

`help()`

```
from crackz.ai.core.async_callbacks import CallbackDispatcher
help(CallbackDispatcher)
```

[`docs/examples/async_callbacks_example.py`](docs/examples/async_callbacks_example.py)

Contributing

Contributing Guide

-
- `.github/copilot-instructions.md` `.github/workflows/copilot-setup-steps.yml`

`<type>/<scope>-<short-slug>`

```
# AI creates feature branch
git checkout -b feat/gui-export-panel

# AI makes changes with proper commit messages
git commit -m "feat(gui): add export panel component"
git commit -m "feat(gui): integrate export panel with main view"

# When creating PR, AI suggests proper title
# □ PR Title: "feat(gui): add export panel for detection results"
# This triggers a MINOR version release (e.g., 1.2.0 → 1.3.0)
```

```
# create & activate venv
uv venv
source .venv/bin/activate  # Windows: .venv\Scripts\activate

# install with dev dependencies
uv sync --dev

# run CLI help
uv run crackz --help

# run GUI (optional)
uv run crackz-gui
```

`pyproject.toml` `[tool.taskipy.tasks]`

```
uv run task lint        # ruff lint (auto-fix)
uv run task format      # ruff format
uv run task mypy        # static type check
uv run task tests       # unit tests only
uv run task integration # integration tests
uv run task all_tests   # full test matrix
uv run task docs        # live docs (mkdocs serve)
uv run task build_docs  # strict build
```

- `tests/unit` `core/tests/unit` `gui/tests/unit`
- `tests/integration` `@pytest.mark.integration`
- `backend/backend/tests/unit/crackz/cli/test_cli_benchmark.py` `@pytest.mark.benchmark`

- `tests/test_data`
-
- `core/tests` `gui/tests`

```
uv run task tests
```

```
uv run task all_tests
```

```
uv run task benchmark
```

`pyproject.toml` `fail_under = 75`

`tests/integration/crackz/gui/`

```
# Run all integration tests (includes GUI smoke tests)
uv run task integration

# Run only GUI smoke tests
uv run pytest tests/integration/crackz/gui/test_gui_smoke.py

# Run GUI in smoke mode (auto-close after initialization)
set CRACKZ_GUI_SMOKE=1  # Windows CMD
export CRACKZ_GUI_SMOKE=1  # Linux/macOS
uv run crackz-gui
```

`CRACKZ_GUI_SMOKE=1`

`CI` `GITHUB_ACTIONS`

```
tests/integration/crackz/gui/
├── test_gui_smoke.py        # Smoke tests for GUI initialization
└── ...                      # Future GUI integration tests
```

1.

```
env = os.environ.copy()
env["CRACKZ_GUI_SMOKE"] = "1"
subprocess.run([sys.executable, "gui/src/crackz_gui/crackz_gui.py"], env=env)
```

2.

```
@pytest.mark.skipif(not _has_display() or os.environ.get("CI"),
                    reason="Requires display")
def test_component(qapp):
    component = MyComponent()
    assert component is not None
```

3.

```
def test_no_unsupported_parameters():
    # Ensure widget initialization works without errors
    browser.toggle_toolbar_buttons(visible=True)  # Should not raise
```

`benchmark`

```
# Run benchmarks - compares to last saved baseline and fails on regression
uv run task benchmark
```

```
# Update the baseline after intentional performance-impacting changes
uv run task benchmark_save
```

```
# Manual benchmark with comparison
uv run pytest -m benchmark --benchmark-only --benchmark-autosave --benchmark-compare --benchmark-compare-fail=mean:20%
```

`.benchmarks/`

- `uv run task format` `lint`
-
-
- `loguru` `print`
-

```
<type>(<optional-scope>): <description>
```

`feat:` `feat(<scope>):` `fix:` `fix(<scope>):` `perf:` `perf(<scope>):`

`docs:` `refactor:` `test:` `chore:` `ci:` `build:` `style:`

```
feat(gui): add return-to-run-view functionality
fix(pdf): correct missing hero image
perf(dataloader): optimize batch processing
```

```
Fix GUI image display and add return-to-run-view functionality
Add new feature to handle user authentication
Update documentation for new feature
```

`feat:` `fix:` `Feat:` `Fix:` `:` `feat(gui):` `fix(cli):`

```
# Install pre-commit hooks (one-time setup)
pip install pre-commit  # or: uv tool install pre-commit
pre-commit install
pre-commit install --hook-type commit-msg
```

```
# Test with a valid message
echo "feat(gui): add export panel" > temp_msg.txt
cz check --commit-msg-file temp_msg.txt

# Test with an invalid message
echo "Fix bug" > temp_msg.txt
cz check --commit-msg-file temp_msg.txt
```

```
git commit --no-verify -m "your message"
```

```
uv run task docs  # serves at http://127.0.0.1:8000
```

```
uv run task build_docs
```

`scripts/export_pdf.py`

```
uv run task pdf
```

- `main` `cz bump`
  `pyproject.toml` `CHANGELOG.md` `main`

`logs/crackz.log`

`main`

<type>/<scope>-<short-slug>

`feat/gui-live-preview` `fix/pdf-rendering-crash` `perf/dataloader-optimization` `docs/api-reference-update` `refactor/config-module-cleanup`

`feat/gui-live-preview`
`feat(gui): add live preview panel` `fix/pdf-crash` `fix(pdf): handle null hero image`

- `main`

<type>/<scope>-<short-slug>

```
feat/gui-live-preview
fix/pdf-hero-image
refactor/logging-bootstrap
perf/dataloader-prefetch
chore/ci-release-step
hotfix/0.17.1-pdf-null-crash
exp/vector-db-poc
```

`feat` `fix` `perf` `refactor` `docs` `test` `chore` `ci` `build` `style` `hotfix` `exp` `wip`

`hotfix/*` `main` `fix:`

```
<type>(<optional-scope>): <imperative summary>

<body – optional>

<footer – BREAKING CHANGE / issue refs>
```

```
feat(gui): add real-time detection preview panel
fix(pdf): correct missing hero when logo == cover image
refactor(logging): unify central + project logger bootstrap
chore(ci): refine release workflow version verification
```

```
feat!: remove legacy --batch flag

BREAKING CHANGE: The --batch flag was replaced by --batch-size.
```

`feat` `fix` `!` `BREAKING CHANGE:`

-
- `feat(gui): add live preview`
- `fix(cli): correct path handling`
- `docs: update installation guide`
- `Fix GUI bugs`
- `Add new feature`
-
-
-
-

`<type>(<scope>): <description>` `feat:` `fix:` `docs:`

`Dev Build` `<base>.devYYYYMMDD+<shortsha>` `dev-<computed-dev-version>` `dev-`

1. `main`
2. `cz bump`
3.
4. `CHANGELOG.md` `vX.Y.Z` `main`
5.
6.
7. `:X.Y.Z` `:latest`
8.
9.
10.
11.
12. `v0.35.0` `latest`
13.
14.

1. `hotfix/<next-patch>-<short-desc>` `main`
2. `fix:`
3. `cz bump`

`TrainingConfig`

`tests/unit/crackz/project/test_training_config_schema.py` `.pre-commit-config.yaml`

```
pip install pre-commit  # or uv tool install pre-commit
pre-commit install
```

`test_project.py`

`docs/configuration.md` *TrainingConfig Contract & Compatibility* `CHANGELOG.md`

`ModelConfig` `DetectionConfig`

		`feat/cli-batch-export`
		`fix/train-metrics-path`
		`perf/dataloader-cache`
		`refactor/model-config`
		`docs/pdf-section`
		`test/cli-train-e2e`
		`chore/update-ruff`
		`ci/release-tag-fix`
		`build/multiarch-images`
		`style/ruff-cleanup`
		`hotfix/0.17.2-threshold-null`
		`exp/alt-model-head`
		`wip/gui-layout-pass1`

`feat(gui):`

`BREAKING CHANGE:`

-
-
- `feat/*`

`typer.secho`

```
# Success messages
typer.secho("□ Operation completed successfully", fg=typer.colors.GREEN)

# Error messages
typer.secho("× Operation failed", fg=typer.colors.RED)

# Warning messages
typer.secho("⚠ Warning message", fg=typer.colors.YELLOW)

# Info messages
typer.secho("ℹ Information", fg=typer.colors.CYAN)
```

`logger`

```
from crackz.log import logger

# Detailed progress, technical info, debugging
logger.info("Detailed operational information")
logger.warning("Warning about internal state")
logger.error("Error details for troubleshooting")
```

NEVER USE

- `print()`
- `typer.echo()` `typer.secho()`
- `console.print()`

-

`core/src/crackz/cli/options.py`

```
from crackz.cli.options import PROJECT_PATH_OPTION, FORCE_OPTION

def command_function(
    project_path: Path | None = PROJECT_PATH_OPTION,
    force: bool = FORCE_OPTION,
) -> None:
```

```
# Only for options not reused elsewhere
LOCAL_OPTION = typer.Option(
    None,
    "--local-flag",
    help="Module-specific option not reused elsewhere",
)
```

NEVER

- `FORCE_OPTION`
- `typer.Option()`
-

```
"""Module docstring explaining CLI command purpose."""

from __future__ import annotations

from pathlib import Path
import typer

from crackz.cli.exceptions import cli_failure
from crackz.cli.options import PROJECT_PATH_OPTION, FORCE_OPTION
from crackz.log import logger  # Always import logger

app = typer.Typer(no_args_is_help=True)

@app.command(name="command")
def command_function(
    project_path: Path | None = PROJECT_PATH_OPTION,
    force: bool = FORCE_OPTION,
) -> None:
    """Command description."""
    try:
        # Business logic here
        logger.info("Processing started")
        typer.secho("□ Operation completed", fg=typer.colors.GREEN)
    except Exception as e:
        cli_failure(e, "Operation name")
```

-
-
-
- `loguru` `print()`
-

-
-
-
-

-
- `tests/unit/`
- `tests/integration/`
- `@pytest.mark.integration` `@pytest.mark.benchmark`

1.
2. `options.py`
3.
4. `uv run task lint && uv run task format`
5. `uv run task mypy`
6. `uv run task tests`

1. `typer.secho()`
2. `logger`
3. `options.py`
4.
5.

1. **`print()`**
2. **`typer.echo()`** `typer.secho()`
3. `options.py`
4.
5.
6.
7.

Release Flow

Release Flow

`.github/workflows/release.yml`

`main`

1.
2.
3. `CHANGELOG.md` `vX.Y.Z`
4. `main`
5.
6.

- `workflow_run` `Integration Tests` `main`
- `workflow_dispatch` `confirm=YES`

```
uv run cz bump --dry-run
```

- `0`
- `21` `NoneIncrementExit`
-

- `feat:`
- `fix:`
- `perf:`

`docs:` `chore:` `refactor:` `test:` `ci:` `build:` `style:`

`version-bump`

-
- `commitizen`
-
-
- `cz bump --yes --changelog`

-
-

- `CHANGELOG.md`
-
-

```
uv lock
```

`uv.lock`

`main`

- `HEAD` `main`
-

`version-bump`

- `CHANGELOG.md`
- `vX.Y.Z`

- `build-wheels`
-
-
-
- `build-sdist`
- `.tar.gz`

`crackz`

`release`

-
- `dist/`
- `site/crackz-docs.pdf` `.0`
- `CHANGELOG.md`
- `SHA256SUMS.txt`
-
-
- `--draft=false`

-
- `cu118` `cu128`
-
- `CHANGELOG.md`
- `SHA256SUMS.txt`
- `.0`

`dist-combined`

- `ai-release-notes.yml`
-
-
- `pyinstaller-build.yml`
-
-
- `docker-build.yml`
-
-

-
-
-
-

- `feat:` `fix:` `perf:` `main`
-

-
-
-
- `.0`

```
gh run list --limit 10
gh run view <run-id>
gh run watch
```

PR Review Workflow

Pull Request Review Workflow with Claude Code

- `gh`
-
-

```
# List open PRs
gh pr list

# View PR details
gh pr view <number>

# View PR diff
gh pr diff <number>

# Check CI status
gh pr checks <number>

# Review PR
gh pr review <number> --approve
gh pr review <number> --comment -b "your comment"
gh pr review <number> --request-changes -b "needs changes"
```

```
# List all open PRs
gh pr list

# List PRs with specific labels
gh pr list --label "ready-for-review"

# List your own PRs
gh pr list --author @me
```

```
# View PR summary
gh pr view 254

# View PR with comments
gh pr view 254 --comments

# Open PR in browser
gh pr view 254 --web
```

```
fix(docs): resolve build failures and clarify release standards #254
Open • theresiasnow wants to merge 4 commits into main from fix/docs-build-and-standards

  This PR fixes critical documentation build failures...

  View this pull request on GitHub: https://github.com/owner/repo/pull/254
```

```
# View full diff
gh pr diff 254

# View only changed filenames
gh pr diff 254 --name-only

# View diff in browser
gh pr diff 254 --web
```

```
# View all checks
gh pr checks 254
```

```
# View only required checks
gh pr checks 254 --required

# Watch checks until completion
gh pr checks 254 --watch

# Open checks in browser
gh pr checks 254 --web
```

```
✓ Build and Test (ubuntu-latest) — pass
✓ Build and Test (windows-latest) — pass
✓ Integration Tests — pass
✓ Lint and Format — pass
```

```
# Checkout PR to review code locally
gh pr checkout 254

# This creates/switches to the PR branch
# Now you can:
# - Run tests: uv run task tests
# - Run linting: uv run task lint
# - Build docs: uv run task build_docs
# - Run the code locally
```

```
Review the changes in scripts/validate_commit_msg.py for:
- Code quality and readability
- Error handling
- Edge cases
- Documentation completeness
```

```
Review the changes in docs/contributing.md for:
- Clarity and completeness
- Correct formatting
- Broken links
- Consistency with existing docs
```

```
Check if the changes need additional tests and identify any edge cases not covered.
```

```
gh pr review 254 --approve -b "LGTM! Code quality looks good, tests pass."
```

```
gh pr review 254 --comment -b "Good work! Minor suggestion: consider adding error handling for X."
```

```
gh pr review 254 --request-changes -b "Please address the linting issues in validate_commit_msg.py before merging."
```

```
gh pr review 254 --approve -b "$(cat <<'EOF'
Great PR! Here's my review:

□ Code quality: Excellent
□ Tests: All passing
□ Documentation: Clear and complete
□ Linting: All checks pass

Minor suggestions:
- Consider adding a note about pre-commit hook in README
- The error messages are very helpful

Approved for merge.
EOF
)"
```

```
# After review, switch back to main
git checkout main
```

```
# List all PRs and store numbers
gh pr list

# Review each PR in sequence
for pr in 254 255 256; do
    echo "Reviewing PR #$pr"
    gh pr view $pr
    gh pr diff $pr --name-only
    gh pr checks $pr
    # Ask Claude Code to review changes
done
```

```
# Get PR data as JSON
gh pr view 254 --json title,body,author,createdAt,additions,deletions,files

# Get check status as JSON
gh pr checks 254 --json name,state,completedAt
```

```
# Continuously monitor checks until completion
gh pr checks 254 --watch --interval 30

# This will update every 30 seconds until all checks complete
```

Always Check CI Before Reviewing

```
# Wait for checks to complete before reviewing
gh pr checks 254 --watch
```

Review Locally for Complex Changes

```
# For complex PRs, checkout and test locally
gh pr checkout 254
uv run task lint
uv run task tests
uv run task build_docs
```

Use Claude Code for Deep Review

Provide Constructive Feedback

```
# Instead of just "needs work"
gh pr review 254 --request-changes -b "$(cat <<'EOF'
Thanks for the PR! A few items to address:

1. Linting: Please run `uv run task lint` to fix PLR2004 error
2. Tests: Add test case for empty commit message
3. Docs: Update contributing.md with hook installation steps

Otherwise looks great!
EOF
)"
```

Verify Release Triggers

```
# Check PR title format
gh pr view 254 --json title

# Should match: <type>(<scope>): <description>
# Examples:
```

```
# □ feat(gui): add export panel
# □ fix(docs): resolve build failures
# × Fix bugs
```

```
# 1. View the PR
gh pr view <number>

# 2. Check what files changed
gh pr diff <number> --name-only

# 3. View the actual changes
gh pr diff <number>

# 4. Checkout and test the fix
gh pr checkout <number>
# Test the fix...

# 5. Run all pre-commit checks (lint, format, mypy, tests)
uv run task pre_commit

# 6. Approve if all looks good
gh pr review <number> --approve -b "Bug fix verified, all pre-commit checks pass."
```

```
# 1. View the PR
gh pr view <number>

# 2. Checkout the branch
gh pr checkout <number>

# 3. Build docs to check for errors
uv run task build_docs

# 4. Review locally with live server
uv run task docs
# Opens at http://127.0.0.1:8000

# 5. Ask Claude Code to review
"Review the documentation changes for clarity, completeness, and broken links"

# 6. Approve or request changes
gh pr review <number> --approve
```

```
# 1. View PR and check size
gh pr view <number> --json additions,deletions,files

# 2. Check CI status
gh pr checks <number>

# 3. Checkout locally
gh pr checkout <number>

# 4. Run full test suite
uv run task all_tests

# 5. Test the feature manually
# ... manual testing ...

# 6. Ask Claude Code for code review
"Review this feature for:
- Code quality and maintainability
- Test coverage
- Documentation completeness
- Potential edge cases"

# 7. Provide detailed review
gh pr review <number> --approve -b "Feature works well, tests comprehensive."
```

1.
2.
3.
4.

```
 Release workflow completed successfully
 New release published: v0.52.1
 Artifacts built and uploaded
 CHANGELOG updated
 Git tag created
```

```
ℹ Release workflow completed - No release created
 Reason: No conventional commits found that trigger releases
 This is expected behavior for documentation and maintenance commits
```

```
gh auth login
```

```
# Make sure you're in the right repository
gh repo view

# Or specify repository explicitly
gh pr view 254 -R Anaxiatech-AB/crackz
```

```
# Stash local changes first
git stash

# Then checkout PR
gh pr checkout <number>

# Restore changes later
git stash pop
```

Distribution

Distribution & Commercial Delivery

`.github/workflows/docker-build.yml`

`.github/workflows/docker-build.yml`

`uv build --wheel`
`python scripts/build_cuda_wheel.py`
`uv build --sdist`
`uv run task build_exe`
`docker build -f Dockerfile -t crackz-cli:latest .`
`docker build -f Dockerfile.slim -t crackz-cli:slim .`
`docker build -f Dockerfile.gpu -t crackz-cli:gpu .`

```
# Build with CUDA 11.8 (default, most compatible for Windows)
python scripts/build_cuda_wheel.py

# Build with CUDA 12.1
python scripts/build_cuda_wheel.py --cuda-version cu121

# Build with CUDA 12.8 (default for Linux)
python scripts/build_cuda_wheel.py --cuda-version cu128

# Build without cleaning artifacts first
python scripts/build_cuda_wheel.py --no-clean
```

```
# Method 1: Install with specific CUDA index
pip install dist/crackz-*.whl --extra-index-url https://download.pytorch.org/whl/cu118

# Method 2: Install with CUDA extra (uses pyproject.toml config)
pip install dist/crackz-*.whl[cu128]  # For CUDA 12.8 on Linux

# Method 3: Install CPU-only version
pip install dist/crackz-*.whl[cpu]
```

`cu118` `cu121`
`cu128`

`--extra-index-url` `pyproject.toml`
`python -c "import torch; print(f'CUDA available: {torch.cuda.is_available()}')"`

```
# Build both CLI and GUI executables (CPU-only)
uv run task build_exe

# Build only CLI
uv run task build_exe_cli

# Build only GUI
uv run task build_exe_gui

# Clean and rebuild
uv run task build_exe_clean
```

```
# Build with CUDA 11.8 (default, recommended for Windows)
python scripts/build_cuda_exe.py
```

```
# Build with CUDA 12.8 (for newer GPUs/Linux)
python scripts/build_cuda_exe.py --cuda-version cu128

# Build only GUI with CUDA
python scripts/build_cuda_exe.py --gui-only

# Skip CUDA check if already installed
python scripts/build_cuda_exe.py --skip-cuda-check
```

`dist/crackz` `dist/crackz-gui`

```
# Use the build script directly for more control
python scripts/build_exe.py --help
python scripts/build_exe.py --clean  # Clean then build both
python scripts/build_exe.py --no-build  # Only clean, don't build

# CUDA build script
python scripts/build_cuda_exe.py --help
python scripts/build_cuda_exe.py --clean --cuda-version cu128
```

-
- `dist/` `v0.14.0`
- `upload_to_pypi = false`

1.
2.
3.
4.

```
# Install specific version from GitHub release
pip install https://github.com/OWNER/REPO/releases/download/v0.30.2/crackz-0.30.2-py3-none-any.whl

# Or download then install
wget https://github.com/OWNER/REPO/releases/download/v0.30.2/crackz-0.30.2-py3-none-any.whl
pip install crackz-0.30.2-py3-none-any.whl
```

```
# Set GitHub token
export GITHUB_TOKEN="ghp_xxxxxxxxxxxx" # gitleaks:allow
```

```
# Download using gh CLI
gh release download v0.30.2 --pattern '*.whl' --repo OWNER/REPO
pip install crackz-*.whl

# Or use curl with authentication
curl -L \
  -H "Authorization: token $GITHUB_TOKEN" \
  -o crackz.whl \
  https://github.com/OWNER/REPO/releases/download/v0.30.2/crackz-0.30.2-py3-none-any.whl
pip install crackz.whl
```

`PRIVATE_INDEX_URL`

`PRIVATE_INDEX_USER` `__token__` `PRIVATE_INDEX_PASSWORD`

```
pip install --index-url https://pypi.company.com/simple/ crackz
```

`.github/workflows/release.yml`

```
name: release
on:
  push:
    tags:
      - 'v*.*.*'
jobs:
  build-release:
    runs-on: ubuntu-latest
    permissions:
      contents: write   # allow releasing
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.13'
      - name: Install uv
        run: pip install uv
      - name: Sync deps (locked)
        run: uv sync --frozen
      - name: Build wheel + sdist
        run: uv build --wheel --sdist
      - name: Build documentation PDF
        run: |
          uv run mkdocs build --clean --strict
          uv run playwright install chromium
          uv run python scripts/export_pdf.py --outline \
            --cover-image 'assets/crackz-logo.png' \
            --logo 'assets/crackz-logo.png' \
            --out site/crackz-docs.pdf
          cp site/crackz-docs.pdf dist/
      - name: Upload to private index (twine)
        if: env.PRIVATE_INDEX_URL != ''
        env:
          TWINE_USERNAME: ${{ secrets.PRIVATE_INDEX_USER }}
          TWINE_PASSWORD: ${{ secrets.PRIVATE_INDEX_PASSWORD }}
        run: |
          pip install twine
          twine upload --repository-url ${{ secrets.PRIVATE_INDEX_URL }} dist/*.whl dist/*.tar.gz
      - name: Attach artifacts to Release
        uses: softprops/action-gh-release@v2
        with:
          files: dist/*
```

`publish-release.yml`

`crackz-gui`

`v0.35.0` `latest`

`PRIVATE_INDEX_URL`

`https://pypi.company.com/simple/` `PRIVATE_INDEX_USER` `__token__`

`PRIVATE_INDEX_PASSWORD`

`.github/workflows/docker-build.yml`

`.github/workflows/docker-build.yml`

```
docker login ghcr.io -u USER -p TOKEN
# Build variants
docker build -f Dockerfile       -t ghcr.io/anaxiatech-ab/crackz:latest .
docker build -f Dockerfile.slim  -t ghcr.io/anaxiatech-ab/crackz:slim   .
docker build -f Dockerfile.gpu   -t ghcr.io/anaxiatech-ab/crackz:gpu    .
# Push
for tag in latest slim gpu; do docker push ghcr.io/anaxiatech-ab/crackz:$tag; done
```

```
docker pull ghcr.io/anaxiatech-ab/crackz:slim
```

	`pip install --index-url https://pypi.company.com/simple/ crackz`
	`pip install https://github.com/OWNER/REPO/releases/download/vVERSION/crackz-VERSION-py3-none-any.whl`
	`pip install torch torchvision --index-url <TORCH_URL> && pip install <crackz-wheel-url>`
	`pip install --no-index --find-links wheelhouse crackz-VERSION-py3-none-any.whl`
	`pip install --require-hashes -r requirements.txt`

1.
   ```
   # Download Crackz wheel from GitHub Release first
   wget https://github.com/OWNER/REPO/releases/download/vVERSION/crackz-VERSION-py3-none-any.whl

   # Create wheelhouse directory and download all dependencies
   mkdir wheelhouse
   pip download crackz-VERSION-py3-none-any.whl -d wheelhouse
   ```
2.
   ```
   (cd wheelhouse && shasum -a 256 * > SHA256SUMS.txt)
   ```
3.
4.
   ```
   pip install --no-index --find-links wheelhouse crackz-VERSION-py3-none-any.whl
   ```

- `.asc`
- `pip install pip-audit` `cyclonedx-bom`
-

`~/.config/pip/pip.conf`

```
[global]
extra-index-url = https://TOKEN@pkg.company.com/simple
```

`%APPDATA%/pip/pip.ini`

-
- `escrow-*`

-
-
-

```
# Build wheel
uv build --wheel
# Build + attach release (manual example)
git tag v0.14.0 && git push origin v0.14.0
# Install from private index
pip install --index-url https://pypi.company.com/simple/ crackz
# Install from GitHub Release
pip install https://github.com/OWNER/REPO/releases/download/vVERSION/crackz-VERSION-py3-none-any.whl
# Offline install
pip install --no-index --find-links wheelhouse crackz-VERSION-py3-none-any.whl
```

Customer Guide

Customer Guide: Crackz for Infrastructure Management

What is a Mask?

```
 Original Image          Mask Image           Result
[photo of concrete] → [white lines on    = Trained model learns
                       black background]    to detect cracks
```

When Do You Need Masks?

Recommended Tools for Creating Masks

1. CVAT (Computer Vision Annotation Tool) - Best for Teams

```
# Self-hosted option (Docker)
docker run -d -p 8080:8080 cvat/server
# Or use cloud-hosted version at app.cvat.ai
```

2. Labelme - Best for Individual Users

```
pip install labelme
labelme
```

`labelme_json_to_dataset <json_file>` `label.png`

3. Photoshop / GIMP - Best for Fine Control

`masks/train/`
`masks/val/` `masks/test/`

4. Python Scripting - Best for Automation

```
# Example: Convert polygon annotations to masks
from PIL import Image, ImageDraw
import json

def polygon_to_mask(image_path, polygon_coords, output_path):
    # Load original image to get dimensions
    img = Image.open(image_path)
    width, height = img.size

    # Create black background
    mask = Image.new('L', (width, height), 0)
    draw = ImageDraw.Draw(mask)

    # Draw white polygon for crack
    draw.polygon(polygon_coords, fill=255)

    # Save mask
    mask.save(output_path)

# Usage
polygon_coords = [(100, 200), (150, 250), (200, 230), ...]
polygon_to_mask('image_01.jpg', polygon_coords, 'masks/train/image_01.png')
```

Best Practices for Mask Creation

Quality Guidelines

-
-
-
-
- `image_01.jpg` `image_01.png`

File Naming Convention

```
✓ CORRECT:
images/train/harbor_crack_001.jpg
masks/train/harbor_crack_001.png

✗ INCORRECT:
```

```
images/train/harbor_crack_001.jpg
masks/train/harbor_crack_001_mask.png  ← Extra suffix
masks/train/HarborCrack001.png         ← Different name
```

Organizational Tips

1.
2.
3.
4.
5.

Train/Val/Test Split

```
masks/
  train/    # 70% of labeled data (200-350 images)
  val/      # 15% of labeled data (30-50 images)
  test/     # 15% of labeled data (30-50 images)
```

`images/`

```
images/
  train/    # Same filenames as masks/train/
  val/      # Same filenames as masks/val/
  test/     # Same filenames as masks/test/
```

Using Crackz with Your Masks

`images/train/`

```
# Automatic mask detection (if masks are in source_folder/masks/)
crackz import-images \
  --project my-project \
  --src ./labeled-data \
  --type train \
  --size 512 512

# Explicit mask source
crackz import-images \
  --project my-project \
  --src ./images \
  --masks_src ./masks \
  --type train \
  --size 512 512
```

```
crackz import-images \
  --project my-project \
  --src ./labeled-data \
  --masks_src ./labeled-data/masks \
  --split \
  --train-ratio 0.7 \
  --val-ratio 0.15 \
  --test-ratio 0.15 \
  --size 512 512 \
  --random-seed 42
```

Troubleshooting Mask Issues

`masks/train/` `masks/`

Advanced: Semi-Automated Mask Generation

Time Estimates

Professional Labeling Services

```
inspection-campaign-2025/
 images/          # Organized by train/val/test/raw splits
 masks/           # Ground truth for training (if available)
 artifacts/       # Detection results, metrics, checkpoints
 logs/            # Complete audit trail
 project.yaml     # Configuration snapshot
```

Customer Access Management

1.
2.
3.
4.

1.
2. `templates/welcome-email.md`
3. `templates/access-revocation-email.md`
4. `templates/renewal-reminder-email.md`
5.
6.
7.
8.

1.
2.
3.

`docs/customer-access.md`

Customer Access Management

1.
2.
3. `pip install crackz`

`pip install crackz`

1.

```
# Via GitHub web UI:
# Settings > Teams > New Team
# Team name: "Customer-Acme-Corp"
# Privacy: Secret
# Repository access: crackz (Read)
```

2.

3.
4.
5.

```
Subject: Crackz Access - Welcome to [Customer Name]

Hi [Contact Name],

Thank you for your Crackz license purchase!

To access Crackz packages and Docker images, please:

1. Create a GitHub account (if you don't have one): https://github.com/signup
2. Reply with your GitHub username
3. We'll add you to the private repository

Once added, you can access:
- Releases: https://github.com/Anaxiatech-AB/crackz/releases
- Docker images: https://github.com/anaxiatech?tab=packages&repo_name=crackz
- Documentation: https://github.com/Anaxiatech-AB/crackz/tree/main/docs

License: [LICENSE-KEY or CONTRACT-REF]
Now the `release.yml` workflow will automatically publish to Azure Artifacts when configured!

Best regards,
Theresia
```

```
Subject: Crackz Access Confirmed

Hi [Contact Name],

You now have access to the Crackz repository!

Installation instructions:
https://github.com/Anaxiatech-AB/crackz/blob/main/docs/installation.md

Getting started:
https://github.com/Anaxiatech-AB/crackz/blob/main/docs/getting-started.md

Docker registry:
ghcr.io/anaxiatech-ab/crackz:latest

Need help? Email theresia.sofia.snow@gmail.com

Best regards,
Theresia
```

```
# Find latest version at: https://github.com/Anaxiatech-AB/crackz/releases
pip install https://github.com/Anaxiatech-AB/crackz/releases/download/v0.30.2/crackz-0.30.2-py3-none-any.whl
```

```
# Download from GitHub Releases page
wget https://github.com/Anaxiatech-AB/crackz/releases/download/v0.30.2/crackz-0.30.2-py3-none-any.whl

# Install locally
pip install crackz-0.30.2-py3-none-any.whl
```

```
# Install GitHub CLI: https://cli.github.com/
gh auth login

# Download latest release
gh release download --repo anaxiatech/crackz --pattern '*.whl'

# Install
pip install crackz-*.whl
```

```
# Generate GitHub Personal Access Token

1. Go to: https://github.com/settings/tokens/new
```

```
2. Note: "Crackz Docker Access"
3. Expiration: 1 year (or custom)
4. Scopes: Select ONLY:
   - □ read:packages
5. Click "Generate token"
6. Copy the token (starts with ghp_)
```

```
# Customer runs this once
export CR_PAT=ghp_xxxxxxxxxxxxxxxxxxx  # Their PAT # gitleaks:allow
echo $CR_PAT | docker login ghcr.io -u GITHUB_USERNAME --password-stdin

# Now they can pull
docker pull ghcr.io/anaxiatech-ab/crackz:latest
docker pull ghcr.io/anaxiatech-ab/crackz:slim
docker pull ghcr.io/anaxiatech-ab/crackz:gpu
```

```
# Use as normal
docker run --rm ghcr.io/anaxiatech-ab/crackz:latest --version
```

1.
2.
3.
4.

   ```
   # Via Azure DevOps UI:
   # Artifacts > Create Feed
   # Name: crackz-packages
   # Visibility: Private (organization)
   # Upstream sources: Enable PyPI (optional)
   ```

5.
6.
7.
   -
   -
8.

   ```
   https://pkgs.dev.azure.com/YOUR-ORG/_packaging/crackz-packages/pypi/simple/
   ```

```
# Settings > Secrets and variables > Actions > New repository secret

PRIVATE_INDEX_URL: https://pkgs.dev.azure.com/YOUR-ORG/_packaging/crackz-packages/pypi/upload
PRIVATE_INDEX_USER: azure
PRIVATE_INDEX_PASSWORD: <Azure-PAT-with-Packaging-Write-scope> # gitleaks:allow
```

`publish-release.yml`

```
# Option 1: Install with explicit index
pip install crackz \
  --index-url https://pkgs.dev.azure.com/YOUR-ORG/_packaging/crackz-packages/pypi/simple/

# Option 2: Configure pip globally
# Linux/macOS: ~/.config/pip/pip.conf
# Windows: %APPDATA%\pip\pip.ini
```

```
[global]
index-url = https://YOUR-PAT@pkgs.dev.azure.com/YOUR-ORG/_packaging/crackz-packages/pypi/simple/
```

```
pip install crackz
crackz --version
```

-
-
-

```
# Install devpi
pip install devpi-server devpi-web

# Initialize server
devpi-init

# Start server
devpi-server --start --host=0.0.0.0 --port=3141

# Create user for yourself
devpi use http://localhost:3141
devpi user -c admin password=SECRET # gitleaks:allow

# Create index
devpi index -c crackz bases=root/pypi
devpi use crackz
```

```
# Install devpi client
pip install devpi-client

# Login
devpi use http://YOUR-SERVER:3141
devpi login admin --password=SECRET # gitleaks:allow
devpi use admin/crackz

# Upload wheel
devpi upload dist/crackz-*.whl
```

```
# Customer configures pip
pip install --index-url http://YOUR-SERVER:3141/admin/crackz/+simple/ crackz
```

`docker run -p 3141:3141 muccg/devpi`

1.

2.
3.
4.
5.
6.
7.
8.
9.

```
Subject: Crackz License Expired - Access Revoked

Hi [Contact],

Your Crackz license expired on [DATE].
Repository and package access has been revoked.

To renew, please contact theresia.sofia.snow@gmail.com

Thank you for using Crackz.
```

1.
2.
3.

1.
2.
3.
4.

```
# Ask customer to verify:
1. Go to: https://github.com/Anaxiatech-AB/crackz
2. If you see "404 Not Found", you don't have access yet
3. Reply with your GitHub username for access
```

`read:packages`

```
# Ask customer to:
1. Generate new PAT: https://github.com/settings/tokens/new
2. Select ONLY: read:packages
3. Run: echo PAT | docker login ghcr.io -u USERNAME --password-stdin
4. Try again: docker pull ghcr.io/anaxiatech-ab/crackz:latest
```

```
# Recommend:
pip install https://github.com/Anaxiatech-AB/crackz/releases/download/v0.30.2/crackz-0.30.2-py3-none-any.whl
```

```
# You prepare:
pip download crackz -d wheelhouse
tar -czf crackz-offline-bundle.tar.gz wheelhouse/

# Customer installs:
tar -xzf crackz-offline-bundle.tar.gz
pip install --no-index --find-links wheelhouse crackz
```

1.

   ```
   # Customer pulls and retags
   docker pull ghcr.io/anaxiatech-ab/crackz:latest
   docker tag ghcr.io/anaxiatech-ab/crackz:latest registry.company.com/crackz:latest
   docker push registry.company.com/crackz:latest
   ```

2.

```
# Get release download counts
gh api repos/anaxiatech/crackz/releases \
  --jq '.[] | {name: .name, downloads: [.assets[].download_count] | add}'
```

```
{"name":"v0.30.2","downloads":47}
{"name":"v0.30.1","downloads":23}
```

```
# Requires GitHub GraphQL API (complex)
# Easier to use web UI for now
```

```
# Crackz Usage Report - October 2025

## Active Customers: 5

| Customer | License | Status | Last Access |
|----------|---------|--------|-------------|
| Acme Corp | LIC-001 | Active | 2025-10-08 |
| Beta Inc | LIC-002 | Active | 2025-10-05 |
| Gamma LLC | LIC-003 | Expired | 2025-09-30 |

## Download Statistics

- Total releases: 12
- Total downloads (wheels): 156
- Docker pulls (latest): 89
- Docker pulls (gpu): 34

## Top Versions
1. v0.30.2: 47 downloads
2. v0.30.1: 23 downloads
```

```
3. v0.30.0: 18 downloads

## Action Items
- [ ] Contact Gamma LLC for renewal
- [ ] Send v0.31.0 release announcement
```

1.
2. `PRIVATE_INDEX_*`
3.
4.
5.
6.
7.
8.
9.
10.
11.
12.

1.
2.
3. `read:packages`
4.
5.
6.
7.
8.
9.
10.
11.
12.

```
#!/bin/bash
# scripts/customer-access/onboard-customer.sh

set -euo pipefail

CUSTOMER_NAME="$1"
GITHUB_USERNAME="$2"
LICENSE_KEY="$3"

echo "□ Onboarding customer: $CUSTOMER_NAME"
echo "GitHub username: $GITHUB_USERNAME"
echo "License: $LICENSE_KEY"

# Create GitHub team (requires gh CLI with admin access)
TEAM_SLUG="customer-$(echo $CUSTOMER_NAME | tr '[:upper:]' '[:lower:]' | tr ' ' '-')"
gh api orgs/YOUR-ORG/teams -f name="Customer-$CUSTOMER_NAME" -f privacy=secret

# Add member to team
gh api orgs/YOUR-ORG/teams/$TEAM_SLUG/memberships/$GITHUB_USERNAME -X PUT

# Add team to repository
gh api orgs/YOUR-ORG/teams/$TEAM_SLUG/repos/YOUR-ORG/crackz -X PUT -f permission=pull

echo "□ Customer onboarded successfully!"
echo ""
echo "Next steps:"
echo "1. Send welcome email to customer"
echo "2. Add to tracking spreadsheet"
echo "3. Schedule renewal reminder for 1 year from now"
```

```
#!/bin/bash
# scripts/customer-access/revoke-customer.sh

set -euo pipefail

CUSTOMER_NAME="$1"
REASON="${2:-License expired}"

echo "□ Revoking access for: $CUSTOMER_NAME"
echo "Reason: $REASON"

TEAM_SLUG="customer-$(echo $CUSTOMER_NAME | tr '[:upper:]' '[:lower:]' | tr ' ' '-')"

# Remove team from repository
gh api orgs/YOUR-ORG/teams/$TEAM_SLUG/repos/YOUR-ORG/crackz -X DELETE

# Optionally delete team entirely
read -p "Delete team entirely? (y/n) " -n 1 -r
echo
if [[ $REPLY =~ ^[Yy]$ ]]; then
    gh api orgs/YOUR-ORG/teams/$TEAM_SLUG -X DELETE
    echo "□ Team deleted"
else
    echo "ℹ Team kept (access removed from repository only)"
fi

echo "□ Access revoked successfully!"
echo ""
echo "Next steps:"
echo "1. Send termination notice to customer"
echo "2. Update tracking spreadsheet"
echo "3. Archive customer records"
```

`PRIVATE_INDEX_*`

`pip install crackz`

Printable PDF

Printable / PDF Export

`site/crackz-docs.pdf`

The cover automatically injects the current project version from `pyproject.toml` and © Anaxiatech AB.

```
uv run task pdf
```

`mkdocs build --clean --strict` `python scripts/export_pdf.py`

```
site/crackz-docs.pdf
```

```
make docs-pdf
```

```
uv run task pdf_fast
```

`site/crackz-docs-fast.pdf`

```
pwsh scripts/windows/export_pdf.ps1 -Outline -CoverTitle "Crackz Documentation"
```

`-NoCover` `-NoHeader`

```
uv run mkdocs build --clean --strict
uv run playwright install chromium   # one time
uv run python scripts/export_pdf.py \
  --outline \
  --cover-image 'assets/crackz-logo.png' \
  --logo 'assets/crackz-logo.png' \
  --cover-title 'Crackz Documentation' \
  --cover-subtitle 'AI Image Analysis' \
  --out site/crackz-docs.pdf
```

```
uv run python scripts/export_pdf.py --out site/my-custom-docs.pdf
```

`nav:`

`--no-bookmarks`

`<article>`

```
- name: Build docs + PDF
  run: |
    uv sync --dev
    uv run mkdocs build --clean --strict
    uv run playwright install chromium
    uv run python scripts/export_pdf.py --outline --cover-image 'assets/crackz-logo.png' --logo 'assets/crackz-logo.png' --out site/crackz-docs.pdf
- uses: actions/upload-artifact@v4
  with:
    name: crackz-docs-pdf
    path: site/crackz-docs.pdf
```

```
https://<your-user>.github.io/crackz/crackz-docs.pdf
```

License

License